

SYDO: Algorithm Specifications

Version 1.0

Zongyang Zhang, Boyuan Gao, Pengce Li, Zihe Gao, Siyuan Qin,
Xiao Kou, Xiang Chen, Jianwei Liu, Ying Gao, Hua Guo, and Yanqing Yao

School of Cyber Science and Technology, Beihang University, China

Table of Contents

1	Introduction	1
1.1	Feature Statement	1
1.2	Relation to Prior Work	1
1.3	Organization	2
2	Preliminaries	2
2.1	Notations	2
2.2	Syndrome Decoding Problem	2
2.3	VOLE-in-the-Head Paradigm	3
2.4	Universal Hashing	5
3	Modeling of the RSD Problem	7
3.1	A Framework for Elementary-Vector Encoding	7
3.2	Polynomial Constraints for RSD	12
4	Algorithm Description	12
4.1	Main Algorithms	13
4.2	Subroutines	16
5	Design Rationale	31
5.1	Hardness Problem	31
5.2	Parameter Selection	32
5.3	Failure Probability	35
6	Theoretical Security Analysis	36
6.1	Preliminaries for Security Proofs	36
6.2	Properties of the VOLE Commitment	39
6.3	EU-F-KO	41
6.4	EU-F-CMA	49
6.5	QROM proof	51
7	Concrete Security Analysis	53
7.1	Attacks Against the SD problem	53
7.2	Other Known Attacks	54
8	Performance Evaluation	55
8.1	Benchmark Environment	55
8.2	Benchmark Results	56
8.3	Performance Analysis	57
A	Performance Comparison with Existing Post-Quantum Signatures	60
A.1	Implementation Methodology	60
A.2	Comparisons with Other Post-Quantum Signatures	61

1 Introduction

This document describes and specifies SYDO, a post-quantum signature scheme based on the hardness of the syndrome decoding (SD) problem. SYDO is designed to provide post-quantum security under conservative assumptions while achieving practical performance. In addition to symmetric-key primitives, its security relies solely on the hardness of the syndrome decoding problem for random linear codes over $\mathbb{F}_2$. This hardness assumption is among the oldest and most extensively studied in code-based cryptography, with decades of well-established cryptanalysis.

Construction overview. Let $\mathbf{H}$ be a random parity-check matrix and choose a low-weight vector $\mathbf{x}$, with $\mathbf{y} = \mathbf{H}\mathbf{x}$. A key pair (pk, sk) of SYDO is then given by $\text{pk} = (\mathbf{H}, \mathbf{y})$, with sk derived from $\mathbf{x}$. SYDO is based on a zero-knowledge argument of knowledge (ZKAoK) of $\mathbf{x}$. The ZKAoK is constructed using the recently proposed VOLE-in-the-Head (VOLEitH) paradigm [BBdSG⁺23b], and is converted into a signature scheme using the Fiat-Shamir transform [FS87]. The signature is the proof generated by the ZKAoK, and the verification procedure checks the proof's validity.

In VOLEitH, proving the Hamming weight of $\mathbf{x}$ and the relation $\mathbf{H}\mathbf{x} = \mathbf{y}$ amounts to proving that a system of polynomial constraints vanishes on the witness $\mathbf{w}$, which is included in sk. The resulting signature size scales linearly with two parameters of this representation: the witness length and the maximum degree of the polynomials. Thus, to obtain short VOLEitH-based signatures for SD, one must represent the SD relation with both a compact witness and low-degree constraints. We refer to this approach to representing a problem by polynomial constraints as *modeling*. The main technical contribution of SYDO is a new modeling method for the SD problem that is well suited to the VOLEitH paradigm. By reducing the witness length while keeping the constraint degree low, this method yields the smallest signature sizes among current SD-based signatures.

1.1 Feature Statement

Conservative hardness assumption. SYDO relies on the unstructured binary syndrome decoding problem for random linear codes. This is one of the oldest and most studied assumptions in code-based cryptography, with well-established cryptanalysis developed over decades.

Innovativeness. The main novelty of SYDO is the development of a new modeling method for SD instances that is well suited to the VOLE framework. This yields the smallest signature sizes among current SD-based signatures. For 160-bit classical security and 80-bit quantum security, SYDO has a signature size of about 5.3 KiB and a public key size of 80 bytes.

Simplicity. At the algorithmic level, SYDO only requires symmetric-key primitives and finite-field operations, which keeps the design simple and easy to implement. Both types of operations are standard in cryptographic software and have mature implementation support.

Flexibility. The parameter sets provide explicit tradeoffs between signature size and the speed of signing and verification. The short variants reduce signature size, while the fast variants improve signing and verification time. This gives a simple way to choose parameters according to whether bandwidth or computation is the main constraint.

Compatibility. SYDO has a modular design that allows the use of different symmetric primitives, including block ciphers and hash functions. These components are already common in cryptographic software and can be implemented using existing optimized libraries or platform support.

Extensibility. The same design approach supports several security levels and both short and fast parameter sets. Further variants can target different security levels, signature-size targets, or performance targets without changing the high-level structure of the scheme.

1.2 Relation to Prior Work

Relation to SDitH. SDitH [MBB⁺25] is a third-round candidate to the NIST call for additional digital signature schemes for the post-quantum cryptography standardization process [NIS22]. Like SYDO, SDitH is based on the hardness of the unstructured SD problem and uses the VOLE-in-the-Head paradigm. The main technical difference lies in the encoding of the low-weight solution $\mathbf{x}$. SYDO uses elementary-vector encoding, which yields more compact witness representations and lower-degree polynomial constraints in VOLEitH, thereby improving performance. As a result, SYDO has shorter signatures and faster running time than SDitH at the same security levels.

Relation to signatures based on RSD. Regular syndrome decoding (RSD) is a structured variant of syndrome decoding in which the solution is required to be a concatenation of elementary vectors. This additional structure has been used to obtain short signatures, including [CCJ23, CLY⁺24, OTX24]. The key distinction between SYDO and these signatures is that SYDO uses RSD only as a modeling tool rather than as an underlying hardness assumption. Concretely, we instantiate a ZKPoK for an RSD instance for which there exists a provable reduction to a secure unstructured SD instance. This ensures that our construction relies only on the standard SD problem while benefiting from the efficiency of modeling low-weight solutions with regular structure.

Relation to signatures from VOLE-in-the-Head. The VOLE-in-the-Head paradigm was introduced as a way to obtain publicly verifiable zero-knowledge proofs and digital signature schemes with short signatures. FAEST [BBB⁺25a], a submission to the NIST call for additional post-quantum digital signatures, is a representative signature scheme based on this paradigm. The main difference between FAEST and SYDO is the underlying hardness assumption; as a consequence, they target different points in the size-time tradeoff. At the same security levels, SYDO gives shorter signatures, whereas FAEST gives faster signing and verification.

1.3 Organization

The remainder of this document is organized as follows. Section 2 recalls the preliminaries used throughout the specification. Section 3 describes the modeling of the hardness problem, and Section 4 gives the SYDO algorithms. Section 5 explains the design rationale and parameter choices. Section 6 and Section 7 give the theoretical and concrete security analyses, respectively. Section 8 reports the benchmark results.

2 Preliminaries

2.1 Notations

$[n]$ denotes the set $\{1, \dots, n\}$, and $[m, n]$ denotes $\{m, m+1, \dots, n\}$ for non-negative integers $m < n$. For integers $a \leq b$, $[a..b]$ denotes the set $\{a, a+1, \dots, b\}$. We write $\mathbb{F}$ for a finite field, and $\mathbb{F}[X_1, \dots, X_\eta]_{\leq d}$ for the set of polynomials over $\mathbb{F}$ in variables $X_1, \dots, X_\eta$ with individual degree at most d . Bold lowercase letters denote vectors, and bold uppercase letters denote matrices. Given a vector $\mathbf{a}$, $\mathbf{a}[i..j]$ denotes the sub-vector from its i -th entry to its j -th entry, and $\mathbf{a}_i$ or $(\mathbf{a})_i$ denotes its i -th entry. The Hamming weight of a vector $\mathbf{x}$ is denoted by $\text{wt}(\mathbf{x})$. The symbols $\parallel$ and $\oplus$ denote string concatenation and bitwise XOR, respectively. The notation $x \leftarrow_r S$ means that x is sampled uniformly at random from S .

2.2 Syndrome Decoding Problem

This section recalls the definitions of the syndrome decoding problem and its regular syndrome decoding variant, which has a tight reduction to the SD problem.

Definition 1 (Syndrome Decoding Problem [BMvT78]). *Let q, n, k and w be positive integers such that $n > k$ and $n > w$. The syndrome decoding problem with parameters (q, n, k, w) is the following problem:*

Let $\mathbf{H}$, $\mathbf{x}$ and $\mathbf{y}$ be such that:

- 1. $\mathbf{H}$ is uniformly sampled from $\mathbb{F}_q^{(n-k) \times n}$,*
- 2. $\mathbf{x}$ is uniformly sampled from $\{\mathbf{x} \in \mathbb{F}_q^n : \text{wt}(\mathbf{x}) = w\}$,*
- 3. $\mathbf{y}$ is defined as $\mathbf{y} := \mathbf{H}\mathbf{x}$.*

From $(\mathbf{H}, \mathbf{y})$, find $\mathbf{x}$.

Definition 2 (Regular Syndrome Decoding Problem [AFS03]). *Let q, n, k and w be positive integers such that $n > k$, $n > w$, and $w \mid n$. The regular syndrome decoding problem with parameters (q, n, k, w) is the following problem:*

Let $\mathbf{H}$, $\mathbf{x}$ and $\mathbf{y}$ be such that:

1. $\mathbf{H}$ is uniformly sampled from $\mathbb{F}_q^{(n-k) \times n}$,
 2. $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_w) \in \mathbb{F}_q^n$ is uniformly sampled such that $\mathbf{x}_i \in \mathbb{F}_q^{n/w}$ and $\text{wt}(\mathbf{x}_i) = 1$ for all $i \in [w]$,
 3. $\mathbf{y}$ is defined as $\mathbf{y} := \mathbf{H}\mathbf{x}$.
- From $(\mathbf{H}, \mathbf{y})$, find $\mathbf{x}$.

SYDO uses RSD as a modeling tool rather than as an underlying hardness assumption. Concretely, we instantiate the ZKPoK for an RSD instance for which there exists a provable reduction to a secure unstructured SD instance. This ensures that our construction relies only on the standard SD problem while benefiting from the efficiency of modeling solutions with regular structure. We formally describe the reduction below and refer the reader to SDitH [MBB⁺25] for the detailed proof.

Theorem 3 ([MBB⁺25]). *Let $\mathbb{F}_q$ be a finite field. Let $\theta \in [1, \infty)$. Let $n, k_{\text{RSD}}, k_{\text{RSD}}, w$ be positive integers such that $k_{\text{SD}} \leq k_{\text{RSD}} \leq n, w < n, w \mid n$, and*

$$k_{\text{RSD}} \leq n - w \cdot \log_2 \left(\frac{n}{w} \right) - \log_2 \theta.$$

Let $\mathcal{A}_{\text{RSD}}$ be an algorithm solving a random $(q, n, k_{\text{RSD}}, w)$ -instance of the regular syndrome decoding problem in time t with success probability ε_{RSD} . Then there exists an algorithm $\mathcal{A}_{\text{SD}}$ solving a random (q, n, k_{SD}, w) -instance of the standard syndrome decoding problem in time t with probability ε_{SD} , where

$$\varepsilon_{\text{SD}} \geq e^{-1/\theta} \cdot \frac{\left(\frac{n}{w}\right)^w}{\binom{n}{w}} \cdot \varepsilon_{\text{RSD}}.$$

2.3 VOLE-in-the-Head Paradigm

This section recalls the construction of the VOLE-in-the-Head paradigm. We first describe how VOLEitH generates VOLE correlations from all-but-one vector commitments, and then explain how these correlations are used to prove polynomial constraints on the witness. We refer to [BBdSG⁺23b, BBB⁺25a] for a detailed description; here we only recall the aspects needed for our construction.

VOLE correlations. Let λ be the security parameter. A vector oblivious linear evaluation (VOLE) correlation of length R consists of random bits $\mathbf{u} = (u_1, \dots, u_R) \in \mathbb{F}_2^R$ and random VOLE tags $\mathbf{v} = (v_1, \dots, v_R) \in \mathbb{F}_{2^\lambda}^R$, together with a random global key $\Delta \in \mathbb{F}_{2^\lambda}$ and VOLE keys $\mathbf{q} = (q_1, \dots, q_R) \in \mathbb{F}_{2^\lambda}^R$ such that

$$q_i = u_i \Delta + v_i, \quad \forall i \in [R].$$

The vectors $\mathbf{u}$ and $\mathbf{v}$ are known only to the prover, while Δ and $\mathbf{q}$ are given to the verifier. To derive authenticated shares for a witness $\mathbf{w} \in \mathbb{F}_2^R$, the prover sends $\mathbf{c} = \mathbf{w} - \mathbf{u}$. The verifier then updates its VOLE keys by computing $q'_i := q_i + c_i \Delta$.

In this relation, the verifier's value q_i acts as a commitment to the prover's bit u_i , with v_i serving as the hiding randomness. Hiding follows from the randomness of v_i , while binding follows from the secrecy of Δ : if the prover opens the same q_i to some $u'_i \neq u_i$, then the corresponding tag $v'_i = q_i - u'_i \Delta$ determines $\Delta = (v_i - v'_i)/(u'_i - u_i)$. Since Δ is uniform in $\mathbb{F}_{2^\lambda}$, this can occur only with probability $2^{-\lambda}$; moreover, the linear relation $q_i = u_i \Delta + v_i$ gives the homomorphic property used by the zero-knowledge proof.

All-but-one vector commitments. Baum et al. [BBdSG⁺23b] show how to generate VOLE correlations in a publicly verifiable way using a special type of vector commitment, called an all-but-one vector commitment (AVC). The prover first samples the values u_i and v_i , and commits to them using the AVC. The commitment allows the verifier to later send the random key Δ to the prover, who then opens the commitment so that the verifier learns the corresponding values q_i .

Here we recall the concrete construction of AVCs used in previous work. This construction commits to a vector of N pseudo-random seeds by deriving them from a root seed using length-doubling PRGs. It starts from a random root seed r and expands a complete binary tree of nodes. At each internal node, the parent seed is evaluated with a length-doubling PRG to obtain two child seeds. After expanding the tree, the seed stored at each leaf node is used as randomness in a leaf commitment function to derive a committed seed sd_i and a commitment com_i . The seed sd_i is the i -th committed seed and will later be expanded as part of a small-field VOLE, while com_i serves as a commitment

to the seed. Finally, applying a hash function, modelled as a random oracle, to all com_i values gives a single commitment to the leaf seeds.

These works [BBM⁺24, BBB⁺25b] give several optimizations on the above construction, which we summarize below.

- Batch all-but- τ vector commitments (BAVCs) and rejection sampling for the opening [BBM⁺24]. The high-level idea is to replace τ separate small trees by one big tree, so that the paths opened for different unopened seeds may overlap and the opening can share tree nodes. Rejection sampling makes this optimization controllable: challenges whose opening size exceeds a threshold T_{open} are discarded, reducing the signature size at the cost of additional hash computations. The corresponding algorithms are given in Section 4.2.3.
- PRG-based leaf commitments [BBB⁺25b]. Instead of computing leaf commitments with a hash function, one can generate them using a PRG. This reduces the computational cost of opening and checking the tree, while relying on the collision resistance of the PRG output for distinct seeds. The concrete routine used by SYDO is specified in Section 4.2.3.

From BAVCs to VOLE. An all-but-one vector commitment of length $N = 2^k$ gives a VOLE commitment over $\mathbb{F}_{2^k}$ as follows. The prover expands the committed seeds $\text{sd}_0, \dots, \text{sd}_{N-1}$ with a PRG to obtain strings $\mathbf{r}_0, \dots, \mathbf{r}_{N-1} \in \{0, 1\}^R$. Using the indices as elements of $\mathbb{F}_{2^k}$, it computes

$$\mathbf{u} = \sum_{i=0}^{N-1} \mathbf{r}_i, \quad \mathbf{v} = \sum_{i=0}^{N-1} i \cdot \mathbf{r}_i.$$

During opening, the verifier learns all seeds except the one at index $j^* \in \{0, \dots, N-1\}$, again viewed as an element of $\mathbb{F}_{2^k}$. From the revealed seeds, it computes

$$\begin{aligned} \mathbf{q} &= \sum_{i=0}^{N-1} (j^* - i) \cdot \mathbf{r}_i \\ &= j^* \cdot \sum_{i=0}^{N-1} \mathbf{r}_i - \sum_{i=0}^{N-1} i \cdot \mathbf{r}_i \\ &= j^* \cdot \mathbf{u} - \mathbf{v} \end{aligned}$$

Thus $\mathbf{q} = j^* \mathbf{u} - \mathbf{v}$, which is the desired VOLE relation over $\mathbb{F}_{2^k}$.

From τ small-field VOLEs to one large-field VOLE. VOLEitH obtains the large-field correlation by packing τ small-field VOLE correlations. We use a field $\mathbb{F}_{2^k}$ with $\tau k = \lambda$, and write $N = 2^k$. The batch vector commitments first give the prover τ small-field pairs $(\mathbf{u}_i, \mathbf{V}_i)$. Before these pairs can be packed, their $\mathbf{u}_i$ components must be made identical. The prover sets the common value to $\mathbf{u} = \mathbf{u}_0$ and sends correction vectors

$$\mathbf{c}_i = \mathbf{u}_i - \mathbf{u}_0, \quad i = 1, \dots, \tau - 1.$$

The verifier uses these corrections after opening, so all small-field VOLE relations are written with the same vector $\mathbf{u}$.

For the corrected VOLEs, write the prover side as $(\mathbf{u}_i, \mathbf{V}_i)$ for $i = 0, \dots, \tau - 1$, and view each $\mathbf{V}_i$ as an $R \times k$ binary matrix. Opening the i -th small-field VOLE gives the verifier $(\Delta_i, \mathbf{Q}_i)$ with

$$\mathbf{Q}_i = \mathbf{V}_i + (\delta_0 \cdot \mathbf{u}_i \dots \delta_{k-1} \cdot \mathbf{u}_i)$$

where $(\delta_0, \dots, \delta_{k-1})$ is the bit decomposition of Δ_i . The final large-field VOLE is obtained by concatenating the τ blocks,

$$\mathbf{V} = (\mathbf{V}_0 \dots \mathbf{V}_{\tau-1}), \quad \mathbf{Q} = (\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1}).$$

Each row is then interpreted as an element of $\mathbb{F}_{2^\lambda}$, and the global key Δ is formed by concatenating the bit decompositions of the Δ_i . This gives

$$\mathbf{Q}[i] = \mathbf{V}[i] + \mathbf{u}[i] \cdot \Delta.$$

VOLE consistency check. The correction vectors must be checked, since an incorrect $\mathbf{c}_i$ would break the VOLE relation used by the ZK proof system. VOLEitH checks this by applying a random

linear universal hash function to the prover-side values $\mathbf{u}$ and $\mathbf{V}$. Writing this hash as multiplication by a compressing matrix $\mathbf{H}$, the prover sends

$$\tilde{\mathbf{u}} = \mathbf{H}\mathbf{u}, \quad \tilde{\mathbf{V}} = \mathbf{H}\mathbf{V}.$$

The verifier computes $\tilde{\mathbf{Q}} = \mathbf{H}\mathbf{Q}$ and checks the same VOLE relation on the compressed values $\tilde{\mathbf{u}}$, $\tilde{\mathbf{V}}$, and $\tilde{\mathbf{Q}}$. The hash family used for this check is described in Section 2.4.1, and the corresponding algorithmic step appears in Section 4.2.4.

A note on witness size. In the discussion above, the witness has length ℓ_w bits, and the first ℓ_w entries of the VOLE mask vector are used to commit to this witness. The VOLE commitment is run on a longer vector of length

$$\ell_v = \ell_w + (d+1)\lambda + B,$$

where d is the degree of the constraint polynomials described below. The final $\lambda + B$ entries are used to hide the value revealed by the VOLE consistency check. After this check, the remaining length $\ell_w + d\lambda$ VOLE is used in the ZK proof phase: the first ℓ_w entries commit to the witness, while the final $d\lambda$ entries provide the masks needed for proving degree- d polynomial constraints.

Proving polynomial constraints. We use the QuickSilver approach [YSWW21] to prove polynomial constraints from VOLE-authenticated witness bits. Let $F_1, \dots, F_t \in \mathbb{F}_2[X_1, \dots, X_R]_{\leq d}$ be a set of polynomials. Given the prover's authenticated witness representation $(\mathbf{w}, \mathbf{v})$ and the verifier's VOLE share $(\Delta, \mathbf{q}')$, the protocol allows the prover to prove knowledge of a witness $\mathbf{w} \in \mathbb{F}_2^R$ satisfying

$$F_j(\mathbf{w}) = 0, \quad \forall j \in [t].$$

For each $j \in [t]$, let $\mathcal{M}_j$ be the set of exponents $\alpha = (\alpha_1, \dots, \alpha_R)$ whose monomial $\prod_{i=1}^R X_i^{\alpha_i}$ appears in F_j with coefficient $c_j^{(\alpha)}$. We write

$$F_j(X_1, \dots, X_R) = \sum_{\alpha \in \mathcal{M}_j} c_j^{(\alpha)} \prod_{i=1}^R X_i^{\alpha_i}.$$

The protocol handles these polynomial constraints by viewing each pair (w_i, v_i) as the coefficients of the degree-1 polynomial

$$p_i(X) = w_i X + v_i.$$

In this way, the condition $F_j(\mathbf{w}) = 0$ is equivalent to requiring the leading coefficient of

$$P_j(X) = \sum_{\alpha \in \mathcal{M}_j} c_j^{(\alpha)} X^{d-(\alpha_1+\dots+\alpha_R)} \prod_{i=1}^R p_i(X)^{\alpha_i}$$

to be zero. To verify this, the verifier samples random scalars $\gamma \in \mathbb{F}_{2^\lambda}^t$ and asks the prover to reveal the polynomial $\tilde{P}(X) = \sum_{j=1}^t \gamma_j P_j(X)$. The verifier checks the validity of $\tilde{P}(X)$ by computing $\tilde{P}(\Delta)$ from its VOLE share $(\Delta, \mathbf{q}')$ and checking whether $\sum_{j=1}^t \gamma_j \tilde{P}_j(\Delta) = \tilde{P}(\Delta)$. If any constraint is unsatisfied, the verifier accepts a cheating prover with probability at most $d \cdot 2^{-\lambda}$. For zero knowledge, the protocol additionally generates $\lambda(d-1)$ random VOLEs and uses them to mask the coefficients of $\tilde{P}(X)$ before revealing them.

2.4 Universal Hashing

The universal hash functions used in SYDO satisfy the almost uniform and hiding properties, which are defined as follows.

Definition 4 (Almost Universal and Almost Uniform Hash Families [BBB⁺25a]). A family of linear hash functions is a family of matrices $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$. The family $\mathcal{H}$ is ε -almost universal if, for every nonzero $\mathbf{x} \in \mathbb{F}_q^n$,

$$\Pr_{\mathbf{H} \leftarrow \mathcal{H}} [\mathbf{H}\mathbf{x} = 0] \leq \varepsilon.$$

The family $\mathcal{H}$ is ε -almost uniform if, for every nonzero $\mathbf{x} \in \mathbb{F}_q^n$ and every $\mathbf{v} \in \mathbb{F}_q^r$,

$$\Pr_{\mathbf{H} \leftarrow \mathcal{H}} [\mathbf{H}\mathbf{x} = \mathbf{v}] \leq \varepsilon.$$

Definition 5 (Hiding Hash Families [BBB⁺25a]). A matrix $\mathbf{H} \in \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$ -hiding if the distribution of $\mathbf{H}\mathbf{v}$ is independent of $\mathbf{v}[0..n-1]$ when $\mathbf{v}[n..n+h-1] \leftarrow \mathbb{F}_q^h$. A hash family $\mathcal{H} \subseteq \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$ -hiding if every $\mathbf{H} \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding.

Proposition 6 ([BBB⁺25a]). Let $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$ be an ε -almost uniform hash family. Let $\mathcal{H}' \subseteq \mathbb{F}_q^{r \times (n+r)}$ be the family $\{[\mathbf{H} \ I_r] : \mathbf{H} \in \mathcal{H}\}$, where I_r is the $r \times r$ identity matrix. Then $\mathcal{H}'$ is ε -almost universal and $\mathbb{F}_q^n$ -hiding.

Proof ([BBB⁺25a]). For $\mathbf{x} = (\mathbf{x}_0, \mathbf{x}_1) \in \mathbb{F}_q^n \times \mathbb{F}_q^r$ nonzero, a collision under $[\mathbf{H} \ I_r]$ means $\mathbf{H}\mathbf{x}_0 + \mathbf{x}_1 = 0$. If $\mathbf{x}_0 = 0$, then this cannot hold because $\mathbf{x}_1 \neq 0$. Otherwise, it holds with probability at most ε by almost uniformity of $\mathcal{H}$. The hiding property follows because the last r input coordinates are added through I_r and therefore perfectly mask the first n coordinates.

2.4.1 VOLE Universal Hash

The construction is the VOLE universal hash of FAEST [BBB⁺25a, Section 4.3.3], with notation adapted to SYDO. It is used for the VOLE consistency check.

Let the hash seed be $\mathbf{sd} \in \{0,1\}^{5\lambda+64}$. It is parsed into λ -bit strings r_0, r_1, r_2, r_3, s and a 64-bit string t . We view r_0, r_1, r_2, r_3, s as elements of $\mathbb{F}_{2^\lambda}$, and view t as an element of $\mathbb{F}_{2^{64}}$. For an input vector $\mathbf{x} \in \mathbb{F}_2^{\ell_v}$, parse $\mathbf{x} = \mathbf{x}_0 \| \mathbf{x}_1$, where $\mathbf{x}_0 \in \mathbb{F}_2^{\ell_w+d\lambda}$ and $\mathbf{x}_1 \in \mathbb{F}_2^{\lambda+B}$. After padding $\mathbf{x}_0$ to $\ell_{\text{pad}} = \lambda \cdot \lceil (\ell_w + d\lambda) / \lambda \rceil$ bits, parse it as both $\hat{\mathbf{y}} \in \mathbb{F}_{2^\lambda}^{\ell_{\text{pad}}/\lambda}$ and $\bar{\mathbf{y}} \in \mathbb{F}_{2^{64}}^{\ell_{\text{pad}}/64}$. Compute

$$h_0 = \sum_{i=0}^{\ell_{\text{pad}}/\lambda-1} s^{\ell_{\text{pad}}/\lambda-1-i} \hat{\mathbf{y}}[i] \in \mathbb{F}_{2^\lambda}, \quad h_1 = \sum_{i=0}^{\ell_{\text{pad}}/64-1} t^{\ell_{\text{pad}}/64-1-i} \bar{\mathbf{y}}[i] \in \mathbb{F}_{2^{64}}.$$

View h_1 as an element $h'_1 \in \mathbb{F}_{2^\lambda}$ by zero-padding its bit representation. Let $(h_2, h_3) = (r_0 h_0 + r_1 h'_1, r_2 h_0 + r_3 h'_1) \in \mathbb{F}_{2^\lambda}^2$. The hash is

$$\text{VOLE.Hash}(\mathbf{sd}, \mathbf{x}) = (\text{FieldsToBits}(h_2, \lambda) \| \text{FieldsToBits}(h_3, \lambda)[0..B-1]) \oplus \mathbf{x}_1.$$

For a matrix input $\mathbf{M}$, VOLE.Hash applies this vector hash to each column of $\mathbf{M}$.

Lemma 7 ([BBB⁺25a]). The hash family Hash maps $\mathbb{F}_2^{\ell_v}$ to $\mathbb{F}_2^{\lambda+B}$. It is an ε_v -almost universal hash family, for $\varepsilon_v = 2^{-\lambda-B}(1 + 2^{B-50})$, if $\ell_{\text{pad}} \leq 2^{13}$. Furthermore, it is $\mathbb{F}_2^{\ell_w+d\lambda}$ -hiding.

Proof ([BBB⁺25a]). This follows the FAEST proof with the above notation. It suffices to analyze the hash that outputs the first $\lambda + B$ bits of (h_2, h_3) before xoring with $\mathbf{x}_1$; adding $\mathbf{x}_1$ gives almost universality for the final hash and the hiding property. The first part, which computes h_0 and h_1 , is a concatenation of two polynomial hashes over $\mathbb{F}_{2^\lambda}$ and $\mathbb{F}_{2^{64}}$. These hashes are $\mathbb{F}_2$ -linear, and the FAEST analysis bounds their concatenation by $\varepsilon_0 \leq \ell_{\text{pad}}^2 / 2^{\lambda+76} \leq 2^{-\lambda-50}$ when $\ell_{\text{pad}} \leq 2^{13}$. The second part starts with the 2×2 matrix hash defined by r_0, r_1, r_2, r_3 , which is $2^{-2\lambda}$ -uniform. After truncating to $\lambda + B$ bits, the resulting hash is ε_1 -uniform for $\varepsilon_1 = 2^{-\lambda-B}$. Combining the two parts as in FAEST gives $\varepsilon_v = \varepsilon_0 + \varepsilon_1 \leq 2^{-\lambda-B}(1 + 2^{B-50})$.

2.4.2 ZK Universal Hash

The ZK universal hash used in SYDO is adapted from the universal hash of FAEST [BBB⁺25a, Section 4.3.4]. The change is that SYDO adds the term $\sum_{i=0}^{\beta-1} g_i \mathbf{x}_0[i]$ for folding the packed RSD linear relation before applying the original FAEST hash. This folding reduces the number of field multiplications needed to check the RSD linear relation, since the $\beta = \lceil (n-k)/\lambda \rceil$ packed syndrome blocks are combined into one field equation before the QuickSilver constraint is hashed.

Let the hash seed be $\mathbf{sd} \in \{0,1\}^{\beta\lambda+3\lambda+64}$. It is parsed into λ -bit strings $g_0, \dots, g_{\beta-1}, r_0, r_1, s$ and a 64-bit string t . We view $g_0, \dots, g_{\beta-1}, r_0, r_1, s$ as elements of $\mathbb{F}_{2^\lambda}$, and view $t \| 0^{\lambda-64}$ as an element of $\mathbb{F}_{2^\lambda}$. For input $(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2) \in \mathbb{F}_{2^\lambda}^\beta \times \mathbb{F}_{2^\lambda}^m \times \mathbb{F}_{2^\lambda}$, the hash is

$$\text{ZKHash}(\mathbf{sd}, \mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2) = \sum_{i=0}^{\beta-1} g_i \mathbf{x}_0[i] + r_0 \sum_{i=0}^{m-1} s^{m-1-i} \mathbf{x}_1[i] + r_1 \sum_{i=0}^{m-1} t^{m-1-i} \mathbf{x}_1[i] + \mathbf{x}_2.$$

Table 3.1. Main notations used in this section

Notation	Meaning
$\mathcal{B}_\mu$	Standard basis of $\mathbb{F}_2^\mu$
$\mathbf{e}_i$	The i -th elementary vector in $\mathcal{B}_\mu$
μ	Length of the reconstructed elementary vector
S	Representative set for elementary vectors
ι	Embedding from S to a binary vector space
η	Length of the embedded binary representation
ϕ	Reconstruction bijection from S to $\mathcal{B}_\mu$
$\hat{\phi}$	Reconstruction map from embedded representatives to $\mathcal{B}_\mu$
f_j	Polynomial reconstructing the j -th output coordinate
d	Degree bound for reconstruction polynomials
$\mathcal{H}_{\eta,r}$	Set of binary vectors in $\mathbb{F}_2^\eta$ of weight at most r
$\mathcal{E}_{\eta,r}$	Set of binary vectors in $\mathbb{F}_2^\eta$ of weight exactly r
ℓ	Number of blocks
η_j	Length of the j -th block
r_j	Weight parameter of the j -th block
$\rho_{j,T}$	Polynomial enforcing the weight upper bound of the j -th block
σ_j	Polynomial enforcing the weight lower bound of the j -th block

In the RSD proof, $\mathbf{x}_0$ contains the coefficients obtained from the folded RSD linear relation, while $\mathbf{x}_1$ contains the remaining QuickSilver constraint coefficients. The final term x_2 is the masking field element supplied by the VOLE masks, exactly as in the FAEST hash.

Lemma 8. *The hash family ZKHash is an ε_{zk} -almost universal hash family from $\mathbb{F}_{2^\lambda}^{\beta+m+1}$ to $\mathbb{F}_{2^\lambda}$, for $\varepsilon_{\text{zk}} = 2^{-\lambda}(1 + 2^{-38})$, if $m + 1 \leq 2^{13}$. Furthermore, ZKHash is $\mathbb{F}_{2^\lambda}^{\beta+m}$ -hiding.*

Proof. The proof follows the proof in FAEST, with one additional linear hash in front of the original ZK universal hash. Let two distinct inputs differ by $(\mathbf{z}_0, \mathbf{z}_1, \mathbf{z}_2)$. If $\mathbf{z}_0 \neq 0$, then the term $\sum_{i=0}^{\beta-1} g_i \mathbf{z}_0[i]$ must equal a fixed field element determined by the remaining terms. This term is uniform in $\mathbb{F}_{2^\lambda}$ because the g_i are uniform and independent. Hence the collision probability is exactly $2^{-\lambda}$. If $\mathbf{z}_0 = 0$, the condition is precisely the collision condition for the FAEST ZK universal hash on $(\mathbf{x}_1, x_2)$. The FAEST analysis gives collision probability at most $2^{-\lambda}(1 + 2^{-38})$ when $m + 1 \leq 2^{13}$. Therefore the full hash family is ε_{zk} -almost universal with $\varepsilon_{\text{zk}} = 2^{-\lambda}(1 + 2^{-38})$. The hiding property follows because the final input x_2 is added with coefficient 1. If x_2 is uniform in $\mathbb{F}_{2^\lambda}$, then the output is uniform and independent of $(\mathbf{x}_0, \mathbf{x}_1)$.

3 Modeling of the RSD Problem

SYDO uses RSD as a modeling tool rather than as an underlying hardness assumption. Although this section develops polynomial constraints for the RSD relation proved by the ZKAoK, the security of the resulting signature is based on the unstructured SD problem through the reduction in Theorem 3. This lets the proof system benefit from the compact representation of regular solutions without changing the underlying hardness assumption.

In this section, we describe an efficient approach for modeling the RSD problem as polynomial constraints to obtain a short signature. We first introduce a framework for elementary-vector encoding, which generalizes the existing approaches and allows for more efficient encoding of the witness. Then, we describe how to model the RSD problem using this framework, and show that the resulting constraint system builds upon our modeling approach.

3.1 A Framework for Elementary-Vector Encoding

As recalled in Section 2, a solution to an RSD can be seen as a concatenation of multiple elementary vectors. Instead of directly including each elementary vector in the witness, recent constructions [OTX24, BBGK24, MBB⁺25] use *elementary-vector encoding* techniques to compress it into

a shorter representation. The encoded representation then later reconstructs the original elementary vector using low-degree polynomials. Prior works instantiate this encoding in two seemingly different ways: a Lagrange-based approach described using polynomials [OTX24, BBGK24], and a tensor-product-based approach described using vectors [MBB⁺25]. This section develops a unified framework for elementary-vector encoding using low-degree polynomials, and shows that the existing approaches are special cases of this framework. For convenience, Table 3.1 summarizes the main notation used throughout this section.

3.1.1 Generalizing Elementary-Vector Encoding and Reconstruction

Before describing the framework, we first introduce some notations. Let $\mathcal{B}_\mu = \{\mathbf{e}_i\}_{i \in [\mu]}$ be the standard basis of $\mathbb{F}_2^\mu$, where $\mathbf{e}_i$ is the i -th elementary vector. For simplicity, we assume that the length μ of the elementary vectors is a power-of-two number. Let S be a representative set with $|S| = \mu$, where each element $s \in S$ corresponds to a unique elementary vector $\mathbf{e}_i \in \mathcal{B}_\mu$. Assume an embedding $\iota : S \hookrightarrow \mathbb{F}_2^\eta$, where $\lceil \log_2 \mu \rceil \leq \eta \leq \mu$. This embedding provides a binary representation for elements in S using η bits. This size- η vector is included in the witness in signature schemes. Define a public bijection $\phi : S \rightarrow \mathcal{B}_\mu$ that maps each element of S to its corresponding elementary vector. Note that ϕ exactly describes the reconstruction procedure. To describe ϕ explicitly, define another map $\hat{\phi} : \mathbb{F}_2^\eta \rightarrow \mathcal{B}_\mu$. By definition, we have for every $s \in S$, $\hat{\phi}(\iota(s)) = \phi(s)$. Under this view, different encoding approaches lead to different choices of S and ϕ (and hence $\hat{\phi}$).

The following theorem summarizes the main result established in this subsection.

Theorem 9. *For any parameters $(\eta_j, r_j)_{j \in [\ell]}$ satisfying $\sum_j \eta_j = \eta$, $\sum_j r_j = d$, and $\eta_j \geq r_j$, there exists a representative set S of size*

$$|S| = \prod_{j=1}^{\ell} \sum_{t=0}^{r_j} \binom{\eta_j}{t}$$

such that a bijection $\phi : S \rightarrow \mathcal{B}_{|S|}$ can be realized by Boolean polynomials of degree at most d .

Moreover, if $r_j \leq d - 1$ for every $j \in [\ell]$, then there exists a map $\varphi : \mathbb{F}_2^\eta \rightarrow \mathbb{F}_2^n$ for some n , realized by Boolean polynomials of degree at most d , such that

$$\varphi(\iota(s)) = \mathbf{0} \iff s \in S.$$

Constructing the reconstruction polynomials. Our starting point is to generally define the reconstruction bijection ϕ and corresponding reconstruction polynomials. Suppose there is a representative set S of size μ with an embedding $\iota : S \hookrightarrow \mathbb{F}_2^\eta$. Define μ degree- d binary polynomials $f_1, \dots, f_\mu \in \mathbb{F}_2[X_1, \dots, X_\eta]_{\leq d}$, where d is a small constant. Let $\mathbf{s}_i \in \iota(S)$ be the witness vector corresponding to $\mathbf{e}_i$. Then, we define $\hat{\phi}$ as

$$\hat{\phi}(\mathbf{s}_i) = (f_1(\mathbf{s}_i), \dots, f_\mu(\mathbf{s}_i)) = \mathbf{e}_i. \quad (1)$$

We next show how to represent the functions $f_1, \dots, f_\mu$ as polynomials. Specifically, we obtain these polynomials by expanding them in the degree- $\leq d$ monomial basis and solving their coefficients via a linear system so that the evaluations on S give the desired elementary vectors.

Let $g^{(1)}, \dots, g^{(k)}$ denote all multilinear monomials with a total degree of at most d in η variables over $\mathbb{F}_2$. By definition, we have $k = \sum_{t=0}^d \binom{\eta}{t}$. Then, for each $j \in [\mu]$, we can write $f_j(X_1, \dots, X_\eta)$ as

$$f_j(X_1, \dots, X_\eta) = \sum_{h=1}^k \alpha_j^{(h)} \cdot g^{(h)}(X_1, \dots, X_\eta), \quad (2)$$

where $\alpha_j^{(h)} \in \mathbb{F}_2$ is the coefficient of $g^{(h)}$ in f_j . However, there may be no valid coefficients in Equation (2) that satisfy Equation (1) for all $i \in [\mu]$. We next discuss under what conditions Equation (1) can hold.

Define a matrix $\mathbf{G}_{S,d} \in \mathbb{F}_2^{\mu \times k}$ such that $(\mathbf{G}_{S,d})_{i,h} = g^{(h)}(\mathbf{s}_i)$, and let $\mathbf{A} \in \mathbb{F}_2^{k \times \mu}$ be the coefficient matrix whose j -th column is $(\alpha_j^{(1)}, \dots, \alpha_j^{(k)})^\top$. Then the conditions $\hat{\phi}(\mathbf{s}_i) = \mathbf{e}_i$ for all $i \in [\mu]$ are

equivalent to the following linear system

$$\underbrace{\begin{bmatrix} g^{(1)}(\mathbf{s}_1) & g^{(2)}(\mathbf{s}_1) & \cdots & g^{(k)}(\mathbf{s}_1) \\ g^{(1)}(\mathbf{s}_2) & g^{(2)}(\mathbf{s}_2) & \cdots & g^{(k)}(\mathbf{s}_2) \\ \vdots & \vdots & \ddots & \vdots \\ g^{(1)}(\mathbf{s}_\mu) & g^{(2)}(\mathbf{s}_\mu) & \cdots & g^{(k)}(\mathbf{s}_\mu) \end{bmatrix}}_{\mathbf{G}_{S,d}} \underbrace{\begin{bmatrix} \alpha_1^{(1)} & \alpha_2^{(1)} & \cdots & \alpha_\mu^{(1)} \\ \alpha_1^{(2)} & \alpha_2^{(2)} & \cdots & \alpha_\mu^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{(k)} & \alpha_2^{(k)} & \cdots & \alpha_\mu^{(k)} \end{bmatrix}}_{\mathbf{A}} = \begin{bmatrix} e_1^T \\ e_2^T \\ \vdots \\ e_\mu^T \end{bmatrix}. \quad (3)$$

From linear algebra, we know that this system has a solution if and only if $\mathbf{G}_{S,d}$ has full row rank, namely $\text{rank}(\mathbf{G}_{S,d}) = \mu$. Recall that monomials $g^{(h)}$ are fixed and public. Subsequently, this means that achieving full row rank requires appropriate choices of $\mathbf{s}_1, \dots, \mathbf{s}_\mu$, or equivalently, an appropriate construction of S .

Specifying the representative set. We now specify the representative set S . Recall that in SDitH [MBB⁺25], the elements of S are partitioned into blocks of sub-vectors of Hamming weight one. However, one limitation is that a binary vector of length η provides only $\binom{\eta}{1}$ choices. If high-weight sub-vectors are allowed, say of weight at most r , then a binary vector of length η can represent many more choices, namely $\sum_{t=0}^r \binom{\eta}{t} = \Theta(\eta^r)$ for fixed r . Intuitively, this means we can represent longer elementary vectors with the same witness length η , or equivalently, to use a smaller η than a weight-one representation when representing the same standard basis elements. Since the length- η string is stored as part of the witness, this leads to a smaller witness.

Based on this intuition, we allow the sub-vectors to have a Hamming weight of at most r . The main challenge is to ensure that the corresponding evaluation matrix $\mathbf{G}_{S,d}$ has full row rank.

Let $\mathcal{H}_{\eta,r} = \{\mathbf{x} \in \mathbb{F}_2^\eta \mid \text{wt}(\mathbf{x}) \leq r\}$ be the Hamming ball of radius r in $\mathbb{F}_2^\eta$. We partition S to ℓ parts by choosing integers $(\eta_j)_{j \in [\ell]}$ and $(r_j)_{j \in [\ell]}$ and define

$$S = \mathcal{H}_{\eta_1, r_1} \times \cdots \times \mathcal{H}_{\eta_\ell, r_\ell}.$$

We show that this high-weight block-wise choice of S satisfies the full-rank requirements for polynomial reconstruction, as long as the blocks of S have a total Hamming weight of at most d (i.e., $\sum_{j \in [\ell]} r_j \leq d$), which is the degree bound of $f_j(X_1, \dots, X_\eta)$ as shown in Equation (2).

Lemma 10. *Let $S = \mathcal{H}_{\eta_1, r_1} \times \cdots \times \mathcal{H}_{\eta_\ell, r_\ell} \subseteq \mathbb{F}_2^\eta$ with $\sum_{j \in [\ell]} r_j \leq d$. Then the matrix $\mathbf{G}_{S,d}$ in Equation (3) has full row rank.*

Proof. Let $S = \{\mathbf{s}_1, \dots, \mathbf{s}_\mu\}$. For each point $\mathbf{s}_q$, define a support monomial polynomial $m_q(X_1, \dots, X_\eta) = \prod_{u \in \text{supp}(\mathbf{s}_q)} X_u$. Since $\mathbf{s}_q \in S$, we have $\text{wt}(\mathbf{s}_q) \leq \sum_{j \in [\ell]} r_j \leq d$. Hence, $m_q(X_1, \dots, X_\eta)$ is a η -variate multilinear monomial of degree at most d , and therefore appears in one of the columns of $\mathbf{G}_{S,d}$.

Let $\mathbf{B} \in \mathbb{F}_2^{\mu \times \mu}$ be the sub-matrix of $\mathbf{G}_{S,d}$ formed by the columns indexed by $m_1, \dots, m_\mu$, where the q -th selected column is the column indexed by m_q . By the definition of the evaluation matrix $\mathbf{G}_{S,d}$, the (p, q) -entry of $\mathbf{B}$ is the value of m_q evaluated at the point $\mathbf{s}_p$. Hence, we have $\mathbf{B}_{p,q} = m_q(\mathbf{s}_p) = \prod_{u \in \text{supp}(\mathbf{s}_q)} (\mathbf{s}_p)_u$. Since all coordinates are binary, this product is equal to 1 exactly when $(\mathbf{s}_p)_u = 1$ for every $u \in \text{supp}(\mathbf{s}_q)$. This condition is equivalent to saying that every nonzero coordinate of $\mathbf{s}_q$ is also a nonzero coordinate of $\mathbf{s}_p$, namely $\text{supp}(\mathbf{s}_q) \subseteq \text{supp}(\mathbf{s}_p)$. Therefore, $\mathbf{B}_{p,q} = 1$ if and only if $\text{supp}(\mathbf{s}_q) \subseteq \text{supp}(\mathbf{s}_p)$.

Now we re-order the points $\mathbf{s}_1, \dots, \mathbf{s}_\mu$ so that their Hamming weights are non-decreasing: $\text{wt}(\mathbf{s}_1) \leq \text{wt}(\mathbf{s}_2) \leq \cdots \leq \text{wt}(\mathbf{s}_\mu)$. We order the selected columns of $\mathbf{B}$ in the same way, meaning that the q -th column is still the column corresponding to the monomial m_q associated with the point $\mathbf{s}_q$.

We claim that, under this ordering, $\mathbf{B}$ is lower triangular with all diagonal entries equal to 1. First, consider a diagonal entry $\mathbf{B}_{q,q}$. By the containment characterization above, $\mathbf{B}_{q,q} = 1 \Leftrightarrow \text{supp}(\mathbf{s}_q) \subseteq \text{supp}(\mathbf{s}_q)$, which is always true. Hence every diagonal entry of $\mathbf{B}$ is equal to 1.

It remains to show that all entries above the diagonal are zero. We prove this by contradiction. Take any pair $p < q$. Suppose, for contradiction, that $\mathbf{B}_{p,q} = 1$. By the containment characterization, this means that $\text{supp}(\mathbf{s}_q) \subseteq \text{supp}(\mathbf{s}_p)$. Therefore, $\text{wt}(\mathbf{s}_q) = |\text{supp}(\mathbf{s}_q)| \leq |\text{supp}(\mathbf{s}_p)| = \text{wt}(\mathbf{s}_p)$. On the other hand, since $p < q$ and the points were ordered by non-decreasing Hamming weight, we also have $\text{wt}(\mathbf{s}_p) \leq \text{wt}(\mathbf{s}_q)$. Thus the two weights must be equal: $\text{wt}(\mathbf{s}_p) = \text{wt}(\mathbf{s}_q)$. However, we also have $\text{supp}(\mathbf{s}_q) \subseteq \text{supp}(\mathbf{s}_p)$. A containment between two finite sets of the same size must be equality, so $\text{supp}(\mathbf{s}_q) = \text{supp}(\mathbf{s}_p)$. Since the vectors are binary, equality of supports implies $\mathbf{s}_q = \mathbf{s}_p$, contradicting

the fact that $\mathbf{s}_1, \dots, \mathbf{s}_\mu$ are distinct points of S . Therefore our assumption was false, and $\mathbf{B}_{p,q} = 0$ for every $p < q$.

We conclude that $\mathbf{B}$ has zeros above the diagonal and ones on the diagonal. That is, $\mathbf{B}$ is a lower triangular matrix with unit diagonal. Such a matrix is non-singular. Therefore, $\mathbf{B}$ has rank μ .

Finally, recall that $\mathbf{B}$ is a $\mu \times \mu$ sub-matrix of $\mathbf{G}_{S,d}$. Since $\mathbf{G}_{S,d}$ contains a nonsingular $\mu \times \mu$ submatrix, the row rank of $\mathbf{G}_{S,d}$ is at least μ . But $\mathbf{G}_{S,d}$ has exactly μ rows, so its row rank is exactly μ . Hence $\mathbf{G}_{S,d}$ has full row rank.

The proof above also implies that not all columns of $\mathbf{G}_{S,d}$, that is, not all monomials, are needed for polynomial reconstruction. Instead, it suffices to use the μ columns indexed by the supporting monomials

$$m_q(X_1, \dots, X_\eta) = \prod_{u \in \text{supp}(\mathbf{s}_q)} X_u, \quad q \in [\mu],$$

which form the non-singular matrix $\mathbf{B}$ constructed above. Hence, for each $j \in [\mu]$, there exist coefficients $c_{1,j}, \dots, c_{\mu,j} \in \mathbb{F}_2$ such that the polynomial

$$f_j(X_1, \dots, X_\eta) = \sum_{q=1}^{\mu} c_{q,j} \cdot m_q(X_1, \dots, X_\eta) \quad (4)$$

satisfies $f_j(\mathbf{s}_i) = 1$ if $i = j$, and $f_j(\mathbf{s}_i) = 0$ otherwise. Therefore, we have $(f_1(\mathbf{s}_i), \dots, f_\mu(\mathbf{s}_i)) = \mathbf{e}_i$ for every $i \in [\mu]$. Equivalently, in the full coefficient matrix $\mathbf{A}$, the coefficients of all monomials other than these μ supporting monomials would be set to zero.

Putting everything together. Let $S = \mathcal{H}_{\eta_1, r_1} \times \dots \times \mathcal{H}_{\eta_\ell, r_\ell}$, where $\sum_{j \in [\ell]} \eta_j = \eta$, $\sum_{j \in [\ell]} r_j \leq d$ and $r_j \geq 1$. Let ι map each $(\mathbf{s}_i^{(1)}, \dots, \mathbf{s}_i^{(\ell)}) \in S$ to the concatenated vector $\mathbf{s}_i = (\mathbf{s}_i^{(1)} \parallel \dots \parallel \mathbf{s}_i^{(\ell)})$. By the full-row-rank property in Lemma 10, there exist Boolean polynomials $f_1, \dots, f_\mu \in \mathbb{F}_2[X_1, \dots, X_\eta]_{\leq d}$ whose evaluations on $\mathbf{s}_i$ output the desired elementary vectors. Then we can define the reconstruction function $\hat{\phi}$ as

$$\hat{\phi} : \mathbf{s}_i \mapsto (f_1(\mathbf{s}_i), \dots, f_\mu(\mathbf{s}_i)) = \mathbf{e}_i.$$

This yields a degree- d reconstruction from a η -bit witness to an elementary vector of length

$$\mu = \prod_{j=1}^{\ell} \sum_{t=0}^{r_j} \binom{\eta_j}{t}. \quad (5)$$

Note that by definition of Hamming balls, we have $\eta_j \geq r_j$ for each j . Since $\sum_{t=0}^{r_j} \binom{\eta_j}{t}$ is increasing in r_j , using the entire degree budget maximizes the number of representatives μ . Hence we take $\sum_{j \in [\ell]} r_j = d$ by default.

We also note that it is necessary to additionally prove the membership condition $\mathbf{s}_i^{(j)} \in \mathcal{H}_{\eta_j, r_j}$ for every sub-vector. To handle this, we introduce additional polynomials of degree $r_j + 1$. Let $\binom{[n]}{m}$ denote the collection of all m -element subsets of $[n]$. For each $j \in [\ell]$ and each set $T \in \binom{[\eta_j]}{r_j+1}$, define a polynomial

$$\rho_{j,T}(X_1, \dots, X_{\eta_j}) = \prod_{t \in T} X_t. \quad (6)$$

Then the bound $\text{wt}(\mathbf{s}_i^{(j)}) \leq r_j$ for every sub-vector is characterized by $\rho_{j,T}(\mathbf{s}_i^{(j)}) = 0$ for all $T \in \binom{[\eta_j]}{r_j+1}$. If we set $r_j + 1 \leq d$ for every j , then the membership constraints have degree at most d .

Sub-vectors from Hamming spheres. We also consider the exact-weight variant, where each sub-vector has a fixed Hamming weight rather than weight at most r_j . Let $\mathcal{E}_{\eta, r} = \{\mathbf{x} \in \mathbb{F}_2^\eta : \text{wt}(\mathbf{x}) = r\}$ be the Hamming sphere of weight r . We set the representative set to $S = \mathcal{E}_{\eta_1, r_1} \times \dots \times \mathcal{E}_{\eta_\ell, r_\ell}$.

The advantage of using Hamming spheres is that the reconstruction polynomials $\{f_j(X_1, \dots, X_\eta)\}_{j \in [\mu]}$ have a relatively simple form: each of them is a single monomial, rather than a linear combination of multiple monomials as in Equation (4). For a set $U \in \binom{[\eta]}{r}$, let $\mathbf{e}_U$ denote the binary vector whose support is exactly U . Consider the monomial $m_U(X_1, \dots, X_\eta) = \prod_{t \in U} X_t$. For any $\mathbf{x} \in \mathcal{E}_{\eta, r}$, the value $m_U(\mathbf{x})$ equals 1 if and only if all coordinates indexed by U equal 1. Since $\mathbf{x}$ has exactly r nonzero

coordinates and $|U| = r$, this happens if and only if $\text{supp}(\mathbf{x}) = U$, or equivalently $\mathbf{x} = \mathbf{e}_U$. Therefore, m_U equals 1 on the single point $\mathbf{e}_U$, and equals 0 on all other points of $\mathcal{E}_{\eta,r}$.

Now consider the representative set $S = \mathcal{E}_{\eta_1, r_1} \times \cdots \times \mathcal{E}_{\eta_\ell, r_\ell}$. Each representative $\mathbf{s}_j \in S$ can be written as $\mathbf{s}_j = (\mathbf{e}_{U_{j,1}}, \dots, \mathbf{e}_{U_{j,\ell}})$, where $U_{j,k} \in \binom{[\eta_k]}{r_k}$. The reconstruction polynomial for this representative is then $f_j(X_1, \dots, X_\eta) = \prod_{k=1}^\ell \prod_{t \in U_{j,k}} X_{k,t}$, where $X_{k,t}$ denotes the t -th variable in the k -th block. This polynomial evaluates to 1 on $\mathbf{s}_j$ and to 0 on every other point of S . Hence, the evaluation of f_j over S is exactly the elementary vector $\mathbf{e}_j$.

The degree of f_j is $\deg(f_j) = \sum_{k=1}^\ell r_k$. Therefore, when $\sum_{k=1}^\ell r_k = d$, all reconstruction polynomials are homogeneous monomials of degree d . This should be contrasted with the Hamming-ball construction above: over $\mathcal{H}_{\eta,r} = \{\mathbf{x} : \text{wt}(\mathbf{x}) \leq r\}$, the monomial $\prod_{t \in U} X_t$ only checks that $U \subseteq \text{supp}(\mathbf{x})$, and thus may be 1 on many different points. Consequently, the reconstruction polynomials in the Hamming-ball construction are obtained by interpolation and may involve linear combinations of several monomials. In the Hamming sphere construction, the exact-weight constraint removes this ambiguity, so each f_j is a single monomial.

We also impose the membership constraints on Hamming spheres $S = \mathcal{E}_{\eta_1, r_1} \times \cdots \times \mathcal{E}_{\eta_\ell, r_\ell}$. To obtain the exact weight r_j , we enforce that the weight is at most r_j , and add one equation to rule out all cases where the weight is smaller than r_j . The constraints $\rho_{j,T}(X_1, \dots, X_{\eta_j}) = 0$ for all T defined in Equation (6) enforce a weight of at most r_j . To exclude weights smaller than r_j , define

$$\sigma_j(X_1, \dots, X_{\eta_j}) = 1 + \sum_{T \in \binom{[\eta_j]}{r_j}} \prod_{t \in T} X_t. \quad (7)$$

The equation $\sigma_j(X_1, \dots, X_{\eta_j}) = 0$ excludes all weights smaller than r_j . Therefore, membership in $\mathcal{E}_{\eta_j, r_j}$ is characterized by $\rho_{j,T}(X_1, \dots, X_{\eta_j}) = 0$ for all $T \in \binom{[\eta_j]}{r_j+1}$, together with $\sigma_j(X_1, \dots, X_{\eta_j}) = 0$. These constraints have a degree of at most d whenever $r_j + 1 \leq d$.

Following the same idea as in SDitH, we can further compress the length- η_j sub-vector using the first $\eta_j - 1$ bits, since the weight of this sub-vector is fixed. For a valid exact-weight block, if the stored coordinates are $(x_1, \dots, x_{\eta_j-1})$, then the last coordinate can be recovered by $x_{\eta_j} \equiv r_j - \sum_{t=1}^{\eta_j-1} x_t \pmod{2}$. Thus, elementary vectors of length

$$\mu = \prod_{j=1}^\ell \binom{\eta_j}{r_j} \quad (8)$$

can be encoded using $\eta' = \sum_j \eta_j - \ell$ bits.

3.1.2 Previous Constructions as Special Cases

Our framework provides a unified way to view existing encoding approaches. In this view, an encoding is specified by the number of blocks, the size of each block, and the prescribed weight in each block. Different choices of parameters in our framework explain different approaches from prior work well. Table 3.2 summarizes these choices and shows how they fit into our framework. We next discuss these instantiations in more detail.

Table 3.2. Summary of existing modeling approaches under the unified framework

Modeling	# sub-vectors	Sub-vector sizes	Sub-vector weights
	ℓ	η or $(\eta_1, \dots, \eta_\ell)$	r or $(r_1, \dots, r_\ell)^\dagger$
OTX24 [OTX24], BBG+24 [BBGK24]	η	$(1, 1, \dots, 1)$	$\leq (1, 1, \dots, 1)$
SDitH [MBB ⁺ 25]	d	$(\eta_1, \dots, \eta_d)$	$= (1, 1, \dots, 1)$
CLY+24 [CLY ⁺ 24]	1	η	$= 1$

[†] $\leq$ denotes the Hamming ball case, while $=$ denotes the Hamming sphere case.

Lagrange-basis encoding. Our construction generalizes the Lagrange-basis encoding used in [OTX24, BBGK24]. To see this, set $\ell = \eta$ and $\eta_j = r_j = 1$ for every $j \in [\ell]$. Then $S = \mathcal{H}_{1,1} \times \cdots \times$

$\mathcal{H}_{1,1} \cong \mathbb{F}_2^\eta$. These parameters give reconstruction degree $d = \eta$ and elementary-vector length $\mu = \prod_{j=1}^\eta \sum_{t=0}^1 \binom{1}{t} = 2^\eta$. Moreover, for each coordinate $j \in [\mu]$, the condition $f_j(\mathbf{s}_i) = (\mathbf{e}_i)_j$ forces f_j to be the indicator function of a single point in $\mathbb{F}_2^\eta$. Therefore, the Lagrange-basis encoding follows naturally from our framework under this parameter choice.

Tensor-product-based encoding. The tensor-product-based encoding from SDitH [MBB⁺25] is recovered from our Hamming sphere construction by setting $\ell = d$ and $r_t = 1$ for every $t \in [d]$. Then $S = \mathcal{E}_{\eta_1,1} \times \cdots \times \mathcal{E}_{\eta_d,1}$ and $\mu = \prod_{t=1}^d \binom{\eta_t}{1} = \prod_{t=1}^d \eta_t$. Each element $s \in S$ determines a unique tuple $(j_1, \dots, j_d) \in [\eta_1] \times \cdots \times [\eta_d]$, where the t -th block is nonzero exactly at coordinate j_t . We identify the coordinates of $\mathbb{F}_2^\mu$ with tuples in $[\eta_1] \times \cdots \times [\eta_d]$. Under this identification, the coordinate of $\phi(s)$ indexed by $(j_1, \dots, j_d)$ is the indicator of the condition that $s_{j_t}^{(t)} = 1$ for every $t \in [d]$, and hence is given by the polynomial $\prod_{t=1}^d s_{j_t}^{(t)}$. Therefore, the reconstruction map obtained from our construction under this parameter choice is exactly the one used in the tensor-product-based encoding of SDitH. It has witness length $\sum_{t=1}^d (\eta_t - 1)$ and reconstruction degree d .

Identical encoding. Our framework also captures the construction of [CLY⁺24], which takes elementary vectors themselves as witnesses and proves their validity using polynomial constraints. This corresponds to setting $\ell = 1$, taking $S = \mathcal{B}_\mu$ as the domain, and setting ϕ to be the identity map with $\phi(\mathbf{e}_i) = \mathbf{e}_i$. The only difference lies in how membership in $\mathcal{B}_\mu$ is proved. To prove that the representation has weight 1, they use the sketching technique of [BGI16], whereas we use a simpler polynomial constraint to enforce the same condition.

3.2 Polynomial Constraints for RSD

We now use the encoding framework above to write the RSD relation as polynomial constraints. Let $\mu = n/w$, and let $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ and $\mathbf{y} \in \mathbb{F}_2^{n-k}$ be the public instance. The RSD relation consists of vectors $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_w)$ such that each block $\mathbf{x}_i \in \mathcal{B}_\mu$ is an elementary vector and $\mathbf{H}\mathbf{x} = \mathbf{y}$. Using the public reconstruction map $\phi: S \rightarrow \mathcal{B}_\mu$ defined in Section 3.1, each block $\mathbf{x}_i$ can be written uniquely as $\phi(s_i)$ for some $s_i \in S$. Recall from Section 3.1.1 that S is a product of Hamming balls $\mathcal{H}_{\eta_j, r_j}$. For $s_i \in S$, write $\iota(s_i) = \mathbf{s}_i \in \mathbb{F}_2^\eta$ and let $\mathbf{s}_i^{(j)}$ be its j -th block. For $S = \mathcal{H}_{\eta_1, r_1} \times \cdots \times \mathcal{H}_{\eta_\ell, r_\ell}$, the witness contains all coordinates of $\mathbf{s}_1, \dots, \mathbf{s}_w$, hence has size $w \sum_{j=1}^\ell \eta_j = w\eta$.

We impose the following polynomial constraints.

- *Membership in S .* For every $i \in [w]$, the representative $\mathbf{s}_i$ must belong to S . For $S = \mathcal{H}_{\eta_1, r_1} \times \cdots \times \mathcal{H}_{\eta_\ell, r_\ell}$, this means that each block $\mathbf{s}_i^{(j)}$ must satisfy

$$\rho_{j,T}(\mathbf{s}_i^{(j)}) = 0 \quad \text{for all } T \in \binom{[\eta_j]}{r_j + 1},$$

where $\rho_{j,T}$ is defined in Equation (6).

- *Syndrome equation.* The reconstructed vector must satisfy

$$\mathbf{H}(\phi(\mathbf{s}_1), \dots, \phi(\mathbf{s}_w)) = \mathbf{y}.$$

The membership constraints have degree d whenever $r_j + 1 \leq d$. The syndrome constraints also have degree at most d , because each component of ϕ is a degree- $\leq d$ polynomial.

4 Algorithm Description

This section gives the algorithms and subroutines for SYDO. We first specify the public-key signature interface, namely key generation, signing, and verification. We then give the supporting routines for data conversion, symmetric primitives, BAVC, VOLE, and RSD.

We first fix the parameter tuples used by key generation, signing, and verification. The first tuple contains the security parameter, the BAVC/VOLE parameters, the grinding parameter, and the lengths needed by the VOLE and ZK proof parts of the signature:

$$\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v).$$

The second tuple contains the RSD parameters used to sample the secret solution and form the witness:

$$\text{param}_{\text{RSD}} := (n, k, w, d, l, (\eta_j)_{j \in [0..l-1]}, (r_j)_{j \in [0..l-1]}).$$

The notation in these tuples is summarized in Table 4.1.

Table 4.1. Parameter notation used by the algorithms

Notation	Meaning
λ	Security parameter
τ	Number of repetitions
N	Number of leaves in each GGM tree row
L	Total number of leaves in the BAVC tree
w_{grind}	Number of grinding bits
T_{open}	Opening-size bound for GGM tree openings
B	Additional length used in the VOLE hash
ℓ_w	Witness length
ℓ_v	Length generated by the VOLE commitment
n	Code length
k	Code dimension
w	Hamming weight of the RSD solution
d	Degree of the constraint polynomials
l	Number of blocks in the witness
(η_j, r_j)	Size and weight bound of the j -th block

4.1 Main Algorithms

This subsection defines the three main algorithms of SYDO. SYDO.KeyGen generates a public key and a secret key, SYDO.Sign produces a signature on a message using the secret key, and SYDO.Verify checks a signature using the public key and the message.

4.1.1 Key Generation

Algorithm 1 presents the key generation algorithm SYDO.KeyGen. It takes no input and outputs a secret key sk and a public key pk . It samples the secret-key seed seed_{sk} and public-matrix seed seed_{pk} . RSD.ExpandWitness expands seed_{sk} into the RSD solution vector $\mathbf{x}$ and witness $\mathbf{w}$. RSD.ExpandPartialH computes the syndrome vector $\mathbf{y}$ from seed_{pk} and $\mathbf{x}$. The public key is $\text{pk} = (\text{seed}_{\text{pk}}, \mathbf{y})$, and the secret key is $\text{sk} = (\text{pk}, \mathbf{w})$.

Algorithm 1 SYDO.KeyGen(; param, param_{RSD})

Input: $\emptyset$

Output: $\text{sk} \in (\{0, 1\}^\lambda \times \mathbb{F}_2^{n-k}) \times \{0, 1\}^\lambda \times \{0, 1\}^{w \sum_{j=0}^{l-1} \eta_j}$ and $\text{pk} \in \{0, 1\}^\lambda \times \mathbb{F}_2^{n-k}$

1 : Parse $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$

2 : Parse $\text{param}_{\text{RSD}} := (n, k, w, d, l, (\eta_j)_j, (r_j)_j)$

3 : $\text{seed}_{\text{sk}} \leftarrow_r \{0, 1\}^\lambda$

4 : $\text{seed}_{\text{pk}} \leftarrow_r \{0, 1\}^\lambda$

5 : $(\mathbf{x}, \mathbf{w}) \leftarrow \text{RSD.ExpandWitness}(\text{seed}_{\text{sk}})$

6 : $\mathbf{y} \leftarrow \text{RSD.ExpandPartialH}(\text{seed}_{\text{pk}}, \mathbf{x})$

7 : $\text{pk} := (\text{seed}_{\text{pk}}, \mathbf{y})$

8 : $\text{sk} := (\text{pk}, \mathbf{w})$

9 : **return** (sk, pk)

4.1.2 Signing

Algorithm 2 SYDO.[Sign](#)(msg, sk; param, param_{RSD})

Input: msg $\in \{0, 1\}^*$ and sk $\in (\{0, 1\}^\lambda \times \mathbb{F}_2^{n-k}) \times \{0, 1\}^{w \sum_{j=0}^{l-1} \eta_j}$

Output: $\sigma \in \{0, 1\}^*$

```

    ▷ Phase 0: Initialization
1 : Parse param := (λ, τ, N, L, wgrind, Topen, B, ℓw, ℓv)
2 : Parse paramRSD := (n, k, w, d, l, (ηj)j, (rj)j)
3 : depth := log N
4 : β := ⌈(n - k)/λ⌉
5 : Parse sk := (pk, w)
6 : ctr := 0
7 : ρ ←r {0, 1}λ
8 : μ ← Hash20(pk||msg; 2λ)
9 : (root, ivpre) ← Hash3(sk||μ||ρ; 2λ)
10 : iv ← Hash4(ivpre)
    ▷ Phase 1: VOLE commitment and consistency check
11 : (com, decomp, c1, ..., cτ-1, u, V) ← VOLE.Commit(root, iv, ℓv)
12 : chall1 ← Hash21(μ||com||c1||...||cτ-1||ivpre; 5λ + 64)
13 : (ũ, Ṽ) ← VOLE.Hash(chall1, (u, V))
14 : d := w ⊕ u[0..ℓw - 1]
15 : chall2 ← Hash22(chall1||ũ||Ṽ||d; βλ + 3λ + 64)
    ▷ Phase 2: RSD relation proof
16 : u := u[ℓw..ℓw + (d - 1)λ - 1]
17 : V := V[0..ℓw + (d - 1)λ - 1]
18 : (πRSD, chkRSD) ← RSD.Prove(w, u, V, pk, chall2)
    ▷ Phase 3: Grinding and opening
19 : decompI := ⊥
20 : repeat
21 :   chall3 ← Hash23(chall2||chkRSD||πRSD||BitDec(ctr, 32); λ)
22 :   if chall3[λ - wgrind..λ - 1] ≠ 0wgrind then
23 :     ctr := ctr + 1
24 :     continue
25 :   I ← VOLE.DecodeAllChall(chall3[0..λ - wgrind - 1])
26 :   decompI ← BAVC.Open(decomp, I)
27 :   if decompI = ⊥ then ctr := ctr + 1
28 : until decompI ≠ ⊥
    ▷ Phase 4: Signature output
29 : σ := (c1, ..., cτ-1, ũ, d, πRSD, decompI, chall3, ivpre, ctr)
30 : return σ

```

The main phases of SYDO.[Sign](#) are summarized as follows.

- **Phase 0: Initialization.** The signer parses the parameter tuple **param**, the RSD parameter tuple **param_{RSD}**, and the secret key **sk** = (pk, w), where pk is the public key and w is the witness. It sets the tree depth **depth** = log N, the number β of packed syndrome field elements, and the 32-bit grinding counter **ctr** = 0, samples the signing randomness ρ, computes the message digest μ = Hash₂⁰(pk||msg; 2λ), derives the root seed root of GGM tree and preliminary IV iv^{pre}, and computes iv (cf. lines 1–10).
- **Phase 1: VOLE commitment and consistency check.** The signer calls VOLE.[Commit](#) on (root, iv, ℓ_v) to obtain the root commitment com, the full decommitment decomp, the correction

vectors $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$, the VOLE mask vector $\mathbf{u}$, and the VOLE matrix $\mathbf{V}$. It computes the first challenge chall_1 , calls `VOLE.Hash` to obtain the VOLE consistency proof $\tilde{\mathbf{u}}$ and hashed VOLE matrix $\tilde{\mathbf{V}}$, sets the masked witness $\mathbf{d} = \mathbf{w} \oplus \mathbf{u}[0..\ell_w - 1]$, and computes the second challenge chall_2 (cf. lines 11–15).

- **Phase 2: RSD relation proof.** The signer replaces the VOLE mask vector $\mathbf{u}$ by the slice $\mathbf{u}[\ell_w..\ell_w + \text{depth} \cdot \lambda - 1]$ and keeps the rows $\mathbf{V}[0..\ell_w + \text{depth} \cdot \lambda - 1]$ of the VOLE matrix $\mathbf{V}$. It calls `RSD.Prove` on $(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2)$ to obtain the RSD proof π_{RSD} and RSD check value chk_{RSD} (cf. lines 16–18).
- **Phase 3: Grinding and opening.** The signer initializes the selected opening $\text{decom}_I = \perp$ and repeats the grinding loop. In each trial, it computes the final challenge chall_3 , increases the grinding counter ctr and continues if the grinding condition fails, otherwise decodes the hidden indices I from $\text{chall}_3[0..\lambda - w_{\text{grind}} - 1]$ and calls `BAVC.Open` on (decom, I) ; if the selected opening decom_I is $\perp$, it increases ctr and repeats (cf. lines 19–28).
- **Phase 4: Signature output.** The signer sets σ to the tuple shown in line 29 and outputs σ (cf. lines 29–30).

4.1.3 Verification

Algorithm 3 `SYDO.Verify`(msg, pk, σ ; param, param_{RSD})

Input: msg $\in \{0, 1\}^*$, pk $\in \{0, 1\}^\lambda \times \mathbb{F}_2^{n-k}$, and $\sigma \in \{0, 1\}^*$

Output: accept $\in \{\text{true}, \text{false}\}$

```

    ▷ Phase 0: Parsing and grinding check
    1 : Parse param := ( $\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v$ )
    2 : Parse paramRSD := ( $n, k, w, d, l, (\eta_j)_j, (r_j)_j$ )
    3 :  $\beta := \lceil (n - k) / \lambda \rceil$ 
    4 : Parse  $\sigma := (\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \tilde{\mathbf{u}}, \mathbf{d}, \pi_{\text{RSD}}, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$ 
    5 :  $\mu \leftarrow \text{Hash}_2^0(\text{pk} \parallel \text{msg}; 2\lambda)$ 
    6 :  $\text{iv} \leftarrow \text{Hash}_4(\text{iv}^{\text{pre}})$ 
    7 : if  $\text{chall}_3[\lambda - w_{\text{grind}}..\lambda - 1] \neq 0^{w_{\text{grind}}}$  then
    8 :   return false
    ▷ Phase 1: VOLE reconstruction
    9 : rec  $\leftarrow \text{VOLE.Reconstruct}(\text{chall}_3[0..\lambda - w_{\text{grind}} - 1], \text{decom}_I, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv})$ 
    10 : if rec =  $\perp$  then
    11 :   return false
    12 : Parse rec := (com,  $\mathbf{Q}$ )
    13 :  $\text{chall}_1 \leftarrow \text{Hash}_2^1(\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}; 5\lambda + 64)$ 
    ▷ Phase 2: VOLE consistency check
    14 :  $\tilde{\mathbf{Q}} \leftarrow \text{VOLE.Hash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda+B) \times \lambda}$ 
    15 : Parse  $\text{chall}_3[0..\lambda - 1]$  as  $(\delta_0, \dots, \delta_{\lambda-1}) \in \{0, 1\}^\lambda$ 
    16 :  $\tilde{\mathbf{V}} := \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda-1} \cdot \tilde{\mathbf{u}}]$ 
    17 :  $\text{chall}_2 \leftarrow \text{Hash}_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}} \parallel \mathbf{d}; \beta\lambda + 3\lambda + 64)$  ▷ Hash  $\tilde{\mathbf{V}}$  in column-major order
    ▷ Phase 3: RSD verification and final challenge
    18 :  $\text{chk}_{\text{RSD}} \leftarrow \text{RSD.Verify}(\mathbf{d}, \mathbf{Q}, \text{pk}, \text{chall}_2, \text{chall}_3, \pi_{\text{RSD}})$ 
    19 :  $\text{chall}'_3 \leftarrow \text{Hash}_3^3(\text{chall}_2 \parallel \text{chk}_{\text{RSD}} \parallel \pi_{\text{RSD}} \parallel \text{BitDec}(\text{ctr}, 32); \lambda)$ 
    20 : if  $\text{chall}_3 = \text{chall}'_3$  then
    21 :   return true
    22 : return false

```

The main phases of `SYDO.Verify` are summarized as follows.

- **Phase 0: Parsing and grinding check.** The verifier parses the parameter tuple param, the RSD parameter tuple param_{RSD}, and the signature σ . The signature contains the correction vectors

$\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$, the VOLE consistency proof $\tilde{\mathbf{u}}$, the masked witness $\mathbf{d}$, the RSD proof π_{RSD} , the selected opening decom_I , the final challenge chall_3 , the preliminary IV iv^{pre} , and the grinding counter ctr . It computes the message digest $\mu = \text{Hash}_2^0(\text{pk} \parallel \text{msg}; 2\lambda)$ and $\text{iv} = \text{Hash}_4(\text{iv}^{\text{pre}})$ from the preliminary IV iv^{pre} , and returns **false** if the grinding condition on the final challenge chall_3 fails (cf. lines 1–8).

- **Phase 1: VOLE reconstruction.** The verifier calls VOLE.Reconstruct on the challenge prefix $\text{chall}_3[0..\lambda - w_{\text{grind}} - 1]$, the selected opening decom_I , the correction vectors $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$, and iv . It rejects if reconstruction returns $\perp$, otherwise parses the root commitment com and reconstructed VOLE keys $\mathbf{Q}$, and recomputes the first challenge chall_1 (cf. lines 9–13).
- **Phase 2: VOLE consistency check.** The verifier computes the hashed VOLE keys $\tilde{\mathbf{Q}} = \text{VOLE.Hash}(\text{chall}_1, \mathbf{Q})$, parses the first λ bits of the final challenge chall_3 as $(\delta_0, \dots, \delta_{\lambda-1})$, reconstructs the hashed VOLE matrix $\tilde{\mathbf{V}}$, and computes the second challenge chall_2 (cf. lines 14–17).
- **Phase 3: RSD verification and final challenge.** The verifier calls RSD.Verify to obtain the RSD check value chk_{RSD} . It recomputes the candidate final challenge chall'_3 and returns **true** exactly when $\text{chall}_3 = \text{chall}'_3$ (cf. lines 18–22).

4.2 Subroutines

This subsection defines the auxiliary routines used by the main algorithms above.

4.2.1 Data Type Conversions

The following subroutines involve conversions between different data types.

- **BitsToFields** takes the input bit string $x \in \{0, 1\}^{nk}$, where n is the number of blocks, and the field degree $k \in \mathbb{N}$ as input, parses blocks of k bits into the field vector $\mathbf{x} \in \mathbb{F}_{2^k}^n$, and outputs $\mathbf{x}$.
- **FieldsToBits** takes the field vector $\mathbf{x} \in \mathbb{F}_{2^k}^n$, where n is the number of field elements, and the field degree $k \in \mathbb{N}$ as input, serializes the field elements into the bit string $x \in \{0, 1\}^{nk}$, and outputs x .
- **BitDec** takes the integer $i \in [0, 2^d)$ and the bit length $d \in \mathbb{N}$ as input, converts the integer into the little-endian bits $(b_0, \dots, b_{d-1}) \in \{0, 1\}^d$, and outputs $(b_0, \dots, b_{d-1})$.
- **IntRec** takes the little-endian bits $(b_0, \dots, b_{d-1}) \in \{0, 1\}^d$ and the bit length $d \in \mathbb{N}$ as input, reconstructs the integer $i \in [0, 2^d)$, and outputs i .

Algorithm 4 $\text{BitsToFields}(x, k)$

Input: $x \in \{0, 1\}^{nk}$ and $k \in \mathbb{N}$

Output: $\mathbf{x} \in \mathbb{F}_{2^k}^n$

1 : $n := |x|/k$

2 : **let** $g_k \in \mathbb{F}_{2^k}$

▷ The generator of $\mathbb{F}_{2^k}$

3 : **for** $i \in [0..n - 1]$

4 : $\mathbf{x}[i] := \sum_{j=0}^{k-1} x[ki + j] \cdot g_k^j$

5 : **return** $\mathbf{x}$

Algorithm 5 [FieldsToBits](#)(x, k)

Input: $x \in \mathbb{F}_{2^k}^n$ and $k \in \mathbb{N}$ **Output:** $x \in \{0, 1\}^{nk}$

```
1 :  $n := |x|$ 
2 : for  $i \in [0..n - 1]$ 
3 :   Parse  $x_0 + x_1g_k + \dots + x_{k-1}g_k^{k-1} = x[i]$  for  $x_j \in \mathbb{F}_2$ 
4 :    $x_i := x_0 \parallel \dots \parallel x_{k-1} \in \{0, 1\}^k$ 
5 :    $x := x \parallel x_i$ 
6 : return  $x \in \{0, 1\}^{nk}$ 
```

Algorithm 6 [BitDec](#)(i, d)

Input: $i \in [0, 2^d)$ and $d \in \mathbb{N}$ **Output:** $(b_0, \dots, b_{d-1}) \in \{0, 1\}^d$

```
1 : for  $j \in [0..d - 1]$ 
2 :    $b_j := i \bmod 2$ 
3 :    $i := (i - b_j)/2$ 
4 : return  $(b_0, \dots, b_{d-1})$ 
```

Algorithm 7 [IntRec](#)($(b_0, \dots, b_{d-1}), d$)

Input: $(b_0, \dots, b_{d-1}) \in \{0, 1\}^d$ and $d \in \mathbb{N}$ **Output:** $i \in [0, 2^d)$

```
1 : return  $\sum_{j=0}^{d-1} b_j \cdot 2^j$ 
```

4.2.2 Symmetric Primitives

Algorithm 8 [PRG](#)($sd, iv, twk; m$)

Input: $sd \in \{0, 1\}^\lambda$, $iv \in \{0, 1\}^\lambda$, $twk \in \{0, 1\}^{32}$, and $m \in \mathbb{N}$ **Output:** $s \in \{0, 1\}^m$

```
1 :  $\beta := \lceil m/\lambda \rceil$ 
2 :  $m' := m - \lambda \cdot (\beta - 1)$ 
3 :  $sd := sd \parallel 0^{\kappa-\lambda}$   $\triangleright \kappa = 192$  for  $\lambda = 160$ ;  $\kappa = \lambda$  for  $\lambda \in \{256, 512\}$ 
4 :  $\bar{k} \leftarrow \text{KeyExpansion}(sd)$   $\triangleright$  Use Rijndael for  $\lambda \in \{160, 256\}$  and BLAKE2s for  $\lambda = 512$ 
5 :  $iv \leftarrow \text{AddToUpperWord}(iv, \text{IntRec}(twk, 32))$ 
6 : for  $i \in [0..\beta - 1]$ 
7 :    $t_i \leftarrow \text{Encrypt}(\bar{k}, \text{AddToLowerWord}(iv, i) \parallel 0^{\kappa-\lambda})$   $\triangleright$  Same primitive as in line 4
8 :    $s_i := t_i[0..\lambda - 1]$ 
9 : return  $s_0 \parallel \dots \parallel s_{\beta-2} \parallel s_{\beta-1}[0..m' - 1]$ 
```

Algorithm 9 `AddToUpperWord`($\text{iv} = (\text{iv}_0, \dots, \text{iv}_{\lambda/32-1}), \alpha$)

Input: $\text{iv} = (\text{iv}_0, \dots, \text{iv}_{\lambda/32-1})$ with $\text{iv}_i \in \{0, 1\}^{32}$ for $i \in [0.. \lambda/32 - 1]$, and $\alpha \in [0..2^{32} - 1]$

Output: $\text{iv}' \in \{0, 1\}^\lambda$

1 : $\text{iv}_{\lambda/32-1} := \text{IntRec}(\text{iv}_{\lambda/32-1}, 32) + \alpha \bmod 2^{32}$ $\triangleright \text{iv}_{\lambda/32-1}$ is the last 32-bit word in little-endian order
 2 : **return** $(\text{iv}_0, \dots, \text{iv}_{\lambda/32-2}, \text{BitDec}(\text{iv}_{\lambda/32-1}))$

Algorithm 10 `AddToLowerWord`($\text{iv} = (\text{iv}_0, \dots, \text{iv}_{\lambda/32-1}), \alpha$)

Input: $\text{iv} = (\text{iv}_0, \dots, \text{iv}_{\lambda/32-1})$ with $\text{iv}_i \in \{0, 1\}^{32}$ for $i \in [0.. \lambda/32 - 1]$, and $\alpha \in [0..2^{32} - 1]$

Output: $\text{iv}' \in \{0, 1\}^\lambda$

1 : $\text{iv}_0 := \text{IntRec}(\text{iv}_0, 32) + \alpha \bmod 2^{32}$ $\triangleright \text{iv}_0$ is the first 32-bit word in little-endian order
 2 : **return** $(\text{BitDec}(\text{iv}_0), \text{iv}_1, \dots, \text{iv}_{\lambda/32-1})$

This subsection describes the symmetric primitives used in SYDO, including pseudorandom generators and hash functions. The following summarizes the interfaces of the symmetric primitives.

- **PRG** : $\{0, 1\}^\lambda \times \{0, 1\}^\lambda \times \{0, 1\}^{32} \rightarrow \{0, 1\}^*$, a pseudorandom generator taking as input a λ -bit seed, a λ -bit initialization vector, and a 32-bit tweak.
- **SeedExp** : $\{0, 1\}^\lambda \times \{0, 1\}^\nu \rightarrow \{0, 1\}^\nu$, an ideal cipher with a λ -bit key and a ν -bit block.
- **Hash₀** : $\{0, 1\}^\lambda \rightarrow \{0, 1\}^{3\lambda}$, a hash function for commitment keys.
- **Hash₁** : $\{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, a collision-resistant hash for vector commitments.
- **Hash₂^j** : $\{0, 1\}^* \rightarrow \{0, 1\}^*$ for $j \in \{0, 1, 2, 3\}$, a hash function for the message digest and Fiat–Shamir challenges, modeled as a random oracle.
- **Hash₃** : $\{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, a hash function for secret randomness derivation.
- **Hash₄** : $\{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, a hash function for IV derivation.

IV-based PRG. The output length of the IV-based PRG **PRG** is specified by the requested length m . For $\lambda \in \{160, 256\}$, the IV-based PRG **PRG** is instantiated using Rijndael- κ , the Rijndael block cipher with κ -bit blocks and κ -bit keys. We set $\kappa = 192$ when $\lambda = 160$ and $\kappa = 256$ when $\lambda = 256$. To reduce the number of calls for key expansion, **PRG** computes the expanded key material once and reuses it across the output blocks. The algorithm uses the following two interfaces.

- **KeyExpansion** : $\{0, 1\}^\kappa \rightarrow \{0, 1\}^{r\kappa}$, a key expansion algorithm taking as input a κ -bit seed and producing expanded key material.
- **Encrypt** : $\{0, 1\}^{r\kappa} \times \{0, 1\}^\kappa \rightarrow \{0, 1\}^\kappa$, an encryption algorithm taking as input expanded key material and a κ -bit block.

For $\lambda = 512$, **PRG** uses the internal block cipher of BLAKE2s [SA15].

The routines **AddToUpperWord** and **AddToLowerWord** view the IV as $\lambda/32$ words in little-endian order. **AddToUpperWord** adds the input integer modulo 2^{32} to the last 32-bit word, while **AddToLowerWord** adds it modulo 2^{32} to the first 32-bit word.

SeedExp. **SeedExp** uses standard AES-192 and AES-256 in CTR mode for the 160- and 256-bit security levels, respectively. For the 512-bit security level, **SeedExp** uses the same internal block cipher of BLAKE2s with a 512-bit key and a 512-bit input block. The block-size symbol in the interface above is $\nu = 128$ for $\lambda \in \{160, 256\}$ and $\nu = 512$ for $\lambda = 512$.

Hash Functions. We instantiate the hash functions using the pseudo-XOF function provided by NGCC. Its interface is **pseudoXOF** : $\{0, 1\}^* \rightarrow \{0, 1\}^*$. For domain separation, the input is suffixed with one byte: i for **Hash_i** and $8 + j$ for **Hash₂^j**. The output length is specified by the parameter m .

- **Hash_i**($\text{msg}; m$) := **pseudoXOF**($\text{msg} \parallel i, m$), if $i \in \{0, 1, 3, 4\}$.
- **Hash₂^j**($\text{msg}; m$) := **pseudoXOF**($\text{msg} \parallel (8 + j), m$), if $j \in \{0, 1, 2, 3\}$.

4.2.3 BAVC Subroutines

The main BAVC routines are the commitment, opening, and reconstruction algorithms.

Algorithm 11 presents the BAVC commitment algorithm **BAVC.Commit**. The input is the root seed $\text{root} \in \{0, 1\}^\lambda$ and $\text{iv} \in \{0, 1\}^\lambda$. The output consists of the root commitment $\text{com} \in \{0, 1\}^{2\lambda}$, the decommitment $\text{decom} \in \{0, 1\}^{(2L-1)\lambda+2\lambda\tau}$, and the leaf seeds $(\text{sd}_{i,j})_{i \in [0..\tau-1], j \in [0..N-1]}$.

- **Phase 0: Tree expansion.** The algorithm parses the parameter tuple param , sets the root node to root , and expands the GGM tree with **PRG** (cf. lines 1–4).
- **Phase 1: Leaf commitments.** For each row index i and leaf index j , the algorithm maps (i, j) to the corresponding tree leaf, calls **BAVC.LeafCommit**, and hashes the row commitments into h_i (cf. lines 5–9).
- **Phase 2: Root commitment and decommitment.** The algorithm hashes $h_0, \dots, h_{\tau-1}$ into com , forms decom from the GGM nodes and leaf commitments, and outputs com , decom , and the leaf seeds (cf. lines 10–12).

Algorithm 11 **BAVC.Commit**($\text{root}, \text{iv}; \text{param}$)

Input: $\text{root} \in \{0, 1\}^\lambda$ and $\text{iv} \in \{0, 1\}^\lambda$

Output: $\text{com} \in \{0, 1\}^{2\lambda}$, $\text{decom} \in \{0, 1\}^{(2L-1)\lambda+2\lambda\tau}$, and $\text{sd} \in \{0, 1\}^{L\lambda}$

```

  ▷ Phase 0: Tree expansion
  1 : Parse  $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$ 
  2 :  $\text{node}_0 := \text{root}$ 
  3 : for  $\alpha \in [0..L-2]$ 
  4 :    $(\text{node}_{2\alpha+1}, \text{node}_{2\alpha+2}) \leftarrow \text{PRG}(\text{node}_\alpha, \text{iv}, \alpha; 2\lambda)$ 
  ▷ Phase 1: Leaf commitments
  5 : for  $i \in [0..\tau-1]$ 
  6 :   for  $j \in [0..N-1]$ 
  7 :      $\text{idx} \leftarrow \text{BAVC.GetLeafIdx}(i, j)$ 
  8 :      $(\text{sd}_{i,j}, \text{com}_{i,j}) \leftarrow \text{BAVC.LeafCommit}(\text{node}_{\text{idx}}, \text{iv}, i + L - 1)$ 
  9 :    $h_i \leftarrow \text{Hash}_1(\text{com}_{i,0} \parallel \dots \parallel \text{com}_{i,N-1}; 2\lambda)$ 
  ▷ Phase 2: Root commitment and decommitment
  10 :  $\text{com} \leftarrow \text{Hash}_1(h_0 \parallel \dots \parallel h_{\tau-1}; 2\lambda)$ 
  11 :  $\text{decom} := ((\text{node}_\alpha)_{\alpha \in [0..2L-2]}, (\text{com}_{i,j})_{i \in [0..\tau-1], j \in [0..N-1]})$ 
  12 : return  $(\text{com}, \text{decom}, (\text{sd}_{i,j})_{i \in [0..\tau-1], j \in [0..N-1]})$ 

```

Algorithm 12 presents the BAVC opening algorithm **BAVC.Open**. The input is the decommitment $\text{decom} \in \{0, 1\}^{(2L-1)\lambda+2\lambda\tau}$ and the hidden index vector $I = (\Delta_0, \dots, \Delta_{\tau-1}) \in [0..N-1]^\tau$. The output is the selected opening $\text{decom}_I \in \{0, 1\}^{2\lambda\tau+T_{\text{open}}\lambda} \cup \{\perp\}$.

- **Phase 0: Hidden path marking.** The algorithm parses decom into the GGM nodes and leaf commitments, then marks the paths from the unopened leaves indexed by I to the root (cf. lines 1–9).
- **Phase 1: Opening derivation.** The algorithm collects the sibling nodes needed to reveal all non-hidden leaves, returns $\perp$ if the opening exceeds the threshold, and otherwise outputs decom_I from the hidden leaf commitments and collected nodes (cf. lines 10–19).

Algorithm 12 BAVC.Open(decom, $I = (\Delta_0, \dots, \Delta_{\tau-1})$; param)

Input: $\text{decom} \in \{0, 1\}^{(2L-1)\lambda + 2\lambda\tau}$ and $I \in [0..N-1]^\tau$ **Output:** $\text{decom}_I \in \{0, 1\}^{2\lambda\tau + T_{\text{open}}\lambda} \cup \{\perp\}$

▷ Phase 0: Hidden path marking

```
1 : Parse param := ( $\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v$ )
2 : Parse decom := (( $\text{node}_\alpha$ ) $_{\alpha \in [0..2L-2]}$ , ( $\text{com}_{i,j}$ ) $_{i \in [0..\tau-1], j \in [0..N-1]}$ )
3 : hide :=  $0^{2L-1} \in \{0, 1\}^{2L-1}$                                 ▷ Nodes on hidden paths use value 1
4 : for  $i \in [0..\tau-1]$                                               ▷ Mark paths from unopened leaves to the root
5 :   idx ← BAVC.GetLeafIdx( $i, \Delta_i$ )
6 :   hide[idx] := 1
7 :   while idx ≠ 0
8 :     idx :=  $\lfloor (\text{idx} - 1) / 2 \rfloor$ 
9 :     hide[idx] := 1
```

▷ Phase 1: Opening derivation

```
10 : nodes :=  $\varepsilon$ 
11 : for  $\alpha = L - 2$  to 0                                           ▷ Append sibling nodes
12 :   if hide[ $2\alpha + 1$ ] = 1 and hide[ $2\alpha + 2$ ] = 0 then
13 :     nodes := nodes ||  $\text{node}_{2\alpha+2}$ 
14 :   if hide[ $2\alpha + 1$ ] = 0 and hide[ $2\alpha + 2$ ] = 1 then
15 :     nodes := nodes ||  $\text{node}_{2\alpha+1}$ 
16 : if |nodes| >  $T_{\text{open}}\lambda$  then                                     ▷ Reject openings that exceed the threshold
17 :   return  $\perp$ 
18 :  $\text{decom}_I := (\text{com}_{0,\Delta_0}, \dots, \text{com}_{\tau-1,\Delta_{\tau-1}}) || \text{nodes} || 0^{T_{\text{open}}\lambda - |\text{nodes}|}$ 
19 : return  $\text{decom}_I$ 
```

Algorithm 13 presents the BAVC reconstruction algorithm BAVC.Reconstruct. The input is the selected opening $\text{decom}_I \in \{0, 1\}^{2\lambda\tau + T_{\text{open}}\lambda}$, the hidden index vector $I = (\Delta_0, \dots, \Delta_{\tau-1}) \in [0..N-1]^\tau$, and $\text{iv} \in \{0, 1\}^\lambda$. The output is the reconstructed root commitment $\text{com} \in \{0, 1\}^{2\lambda}$ and the revealed seeds $(\text{sd}_{i,j})_{i \in [0..\tau-1], j \in [0..N-1] \setminus \{\Delta_i\}}$, or $\perp$.

- **Phase 0: Opening parsing.** The algorithm parses decom_I , marks the hidden paths for I , and reads the sibling nodes from the opening, returning $\perp$ if the opening is malformed (cf. lines 1–22).
- **Phase 1: Commitment reconstruction.** The algorithm expands the revealed subtrees, recomputes the leaf commitments and row hashes, reconstructs com , and returns com together with the revealed seeds (cf. lines 23–35).

Algorithm 13 BAVC.Reconstruct($\text{decom}_I, I = (\Delta_0, \dots, \Delta_{\tau-1}), \text{iv}; \text{param}$)

Input: $\text{decom}_I \in \{0, 1\}^{2\lambda\tau + T_{\text{open}}\lambda}$, $I \in [0..N-1]^\tau$, and $\text{iv} \in \{0, 1\}^\lambda$

Output: $\text{com} \in \{0, 1\}^{2\lambda}$ and $L - \tau$ seeds in $\{0, 1\}^\lambda$

▷ Phase 0: Opening parsing

```

1 : Parse param := ( $\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v$ )
2 : Parse  $\text{decom}_I := (\text{com}_{0, \Delta_0}, \dots, \text{com}_{\tau-1, \Delta_{\tau-1}}, \text{nodes})$ 
3 :  $\text{hide} := 0^{2L-1} \in \{0, 1\}^{2L-1}$                                 ▷ Nodes on hidden paths use value 1
4 : for  $i \in [0.. \tau - 1]$                                            ▷ Mark paths from unopened leaves to the root
5 :    $\text{idx} \leftarrow \text{BAVC.GetLeafIdx}(i, \Delta_i)$ 
6 :    $\text{hide}[\text{idx}] := 1$ 
7 :   while  $\text{idx} \neq 0$ 
8 :      $\text{idx} := \lfloor (\text{idx} - 1) / 2 \rfloor$ 
9 :      $\text{hide}[\text{idx}] := 1$ 
10 : for  $\alpha = L - 2$  to 0                                           ▷ Read sibling nodes
11 :   if  $\text{hide}[2\alpha + 1] = 1$  and  $\text{hide}[2\alpha + 2] = 0$  then
12 :     if nodes is empty then
13 :       return  $\perp$ 
14 :     Parse nodes := ( $\text{node}_{2\alpha+2}, \text{nodes}'$ )
15 :     nodes := nodes'
16 :   if  $\text{hide}[2\alpha + 1] = 0$  and  $\text{hide}[2\alpha + 2] = 1$  then
17 :     if nodes is empty then
18 :       return  $\perp$ 
19 :     Parse nodes := ( $\text{node}_{2\alpha+1}, \text{nodes}'$ )
20 :     nodes := nodes'
21 : if nodes is not all zeros then
22 :   return  $\perp$ 

```

▷ Phase 1: Commitment reconstruction

▷ Expand the revealed subtrees

```

23 : for  $\alpha \in [0..L - 2]$ 
24 :   if  $\text{hide}[\alpha] = 0$  then
25 :     ( $\text{node}_{2\alpha+1}, \text{node}_{2\alpha+2}$ )  $\leftarrow \text{PRG}(\text{node}_\alpha, \text{iv}, \alpha; 2\lambda)$ 
26 : for  $i \in [0.. \tau - 1]$ 
27 :   for  $j \in [0..N - 1]$ 
28 :      $\text{idx} \leftarrow \text{BAVC.GetLeafIdx}(i, j)$ 
29 :     if  $\text{hide}[\text{idx}] = 1$  then
30 :        $h_{i,j} := \text{com}_{i, \Delta_i}$ 
31 :     else
32 :       ( $\text{sd}_{i,j}, h_{i,j}$ )  $\leftarrow \text{BAVC.LeafCommit}(\text{node}_{\text{idx}}, \text{iv}, i + L - 1)$ 
33 :    $h_i \leftarrow \text{Hash}_1(h_{i,0} \parallel \dots \parallel h_{i,N-1}; 2\lambda)$ 
34 :    $\text{com} \leftarrow \text{Hash}_1(h_0 \parallel \dots \parallel h_{\tau-1}; 2\lambda)$ 
35 : return ( $\text{com}, (\text{sd}_{i,j})_{i \in [0.. \tau-1], j \in [0..N-1] \setminus \{\Delta_i\}}$ )

```

The remaining routines specify the helper subroutines used by the BAVC commitment, opening, and reconstruction algorithms.

- BAVC.[GetLeafIdx](#) takes the row index $i \in [0..\tau - 1]$ and the leaf index $j \in [0..N - 1]$ as input, maps the pair (i, j) to the tree index $\text{idx} \in [L - 1..2L - 2]$, and outputs idx .
- BAVC.[LeafCommit](#) takes the leaf seed $\text{sd} \in \{0, 1\}^\lambda$, $\text{iv} \in \{0, 1\}^\lambda$, and the 32-bit tweak $\text{twk} \in \{0, 1\}^{32}$ as input, computes the leaf commitment $\text{com} \in \{0, 1\}^{2\lambda}$ by calling [PRG](#), and outputs (sd, com) .

Algorithm 14 BAVC.[GetLeafIdx](#)(i, j)

Input: $i \in [0..\tau - 1]$ and $j \in [0..N - 1]$
Output: $\text{idx} \in [L - 1..2L - 2]$
1 : Parse $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$
2 : **return** $L - 1 + \tau j + i$

Algorithm 15 BAVC.[LeafCommit](#)($\text{sd}, \text{iv}, \text{twk}$)

Input: $\text{sd} \in \{0, 1\}^\lambda$, $\text{iv} \in \{0, 1\}^\lambda$, and $\text{twk} \in \{0, 1\}^{32}$
Output: $\text{sd} \in \{0, 1\}^\lambda$ and $\text{com} \in \{0, 1\}^{2\lambda}$
1 : $\text{com} \leftarrow \text{PRG}(\text{sd}, \text{iv}, \text{twk}; 2\lambda)$
2 : **return** (sd, com)

4.2.4 VOLE Subroutines

Algorithm 16 VOLE.[Commit](#)($\text{root}, \text{iv}, \ell_v; \text{param}$)

Input: $\text{root} \in \{0, 1\}^\lambda$, $\text{iv} \in \{0, 1\}^\lambda$, and $\ell_v \in \mathbb{N}_{>0}$
Output: $\text{com} \in \{0, 1\}^{2\lambda}$, $\text{decom} \in \{0, 1\}^{(2L-1)\lambda+2\lambda\tau}$, $\mathbf{c}_i, \mathbf{u} \in \{0, 1\}^{\ell_v}$, and $\mathbf{V} \in \{0, 1\}^{\ell_v \times \lambda}$

▷ Phase 0: BAVC commitment

1 : Parse $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$
2 : $\text{depth} := \log N$
3 : $(\text{com}, \text{decom}, (\text{sd}_{i,j})_{i \in [0..\tau-1], j \in [0..N-1]}) \leftarrow \text{BAVC.Commit}(\text{root}, \text{iv}; \text{param})$
▷ Phase 1: Row expansion

4 : **for** $i \in [0..\tau - 1]$
5 : $(\mathbf{u}_i, \mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,\text{depth}-1}) \leftarrow \text{VOLE.BuildVOLE}(\text{sd}_{i,0}, \dots, \text{sd}_{i,N-1}, \text{iv}, i + 2^{31}, \ell_v)$
6 : $\mathbf{V}_i := [\mathbf{v}_{i,0} \ \dots \ \mathbf{v}_{i,\text{depth}-1}] \in \mathbb{F}_2^{\ell_v \times \text{depth}}$
7 : $\mathbf{V} := [\mathbf{V}_0 \ \dots \ \mathbf{V}_{\tau-1} \ \mathbf{0}] \in \mathbb{F}_2^{\ell_v \times \lambda}$
▷ Phase 2: Correction vectors

8 : $\mathbf{u} := \mathbf{u}_0$
9 : **for** $i \in [1..\tau - 1]$
10 : $\mathbf{c}_i := \mathbf{u} \oplus \mathbf{u}_i$
11 : **return** $(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V})$

Algorithm 16 presents the VOLE committing algorithm VOLE.[Commit](#). The input is the root seed $\text{root} \in \{0, 1\}^\lambda$, $\text{iv} \in \{0, 1\}^\lambda$, and the output length $\ell_v \in \mathbb{N}_{>0}$. The output is the root commitment $\text{com} \in \{0, 1\}^{2\lambda}$, the decommitment $\text{decom} \in \{0, 1\}^{(2L-1)\lambda+2\lambda\tau}$, the correction vectors $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1} \in \{0, 1\}^{\ell_v}$, the vector $\mathbf{u} \in \{0, 1\}^{\ell_v}$, and the VOLE matrix $\mathbf{V} \in \{0, 1\}^{\ell_v \times \lambda}$.

- **Phase 0: BAVC commitment.** The algorithm parses the parameter tuple, sets $\text{depth} = \log N$, and calls BAVC.[Commit](#) on the root seed and IV (cf. lines 1–3).

- **Phase 1: Row expansion.** For each row index i , the algorithm calls `VOLE.BuildVOLE` on the row seeds, obtains $\mathbf{u}_i$ and the tag vectors, and places the tag vectors into the corresponding block of $\mathbf{V}$ (cf. lines 4–7).
- **Phase 2: Correction vectors.** The algorithm sets $\mathbf{u}$ to the first row value, computes each correction vector $\mathbf{c}_i = \mathbf{u} \oplus \mathbf{u}_i$ for $i \in [1..\tau - 1]$, and outputs $(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V})$ (cf. lines 8–11).

Algorithm 17 `VOLE.Reconstruct`($\text{chall}, \text{decom}_I, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}; \text{param}$)

Input: $\text{chall} \in \{0, 1\}^{\tau \log_2 N}$, $\text{decom}_I \in \{0, 1\}^{2\lambda\tau + T_{\text{open}}\lambda}$, $\mathbf{c}_i \in \{0, 1\}^{\ell_v}$, and $\text{iv} \in \{0, 1\}^\lambda$

Output: $\text{com} \in \{0, 1\}^{2\lambda}$ and $\mathbf{Q} \in \{0, 1\}^{\ell_v \times \lambda}$

```

  ▷ Phase 0: BAVC reconstruction
  1 : Parse  $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$ 
  2 :  $\text{depth} := \log N$ 
  3 :  $I = (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{VOLE.DecodeAllChall}(\text{chall}; \text{param})$ 
  4 :  $\text{rec} \leftarrow \text{BAVC.Reconstruct}(\text{decom}_I, I, \text{iv}; \text{param})$ 
  5 : if  $\text{rec} = \perp$  then
  6 :   return  $\perp$ 
  7 : Parse  $\text{rec} := (\text{com}, (\text{sd}_{i,j})_{i \in [0..\tau-1], j \in [0..N-1] \setminus \{\Delta_i\}})$ 
  ▷ Phase 1: VOLE key reconstruction
  8 : for  $i \in [0..\tau - 1]$ 
  9 :   for  $j \in [1..N - 1]$ 
  10 :     $\text{sd}'_{i,j} := \text{sd}_{i,j \oplus \Delta_i}$  ▷ XOR  $j$  and  $\Delta_i$  as bit strings
  11 :     $(\mathbf{u}'_i, \mathbf{q}_{i,0}, \dots, \mathbf{q}_{i,\text{depth}-1}) \leftarrow \text{VOLE.BuildVOLE}(\perp, \text{sd}'_{i,1}, \dots, \text{sd}'_{i,N-1}, \text{iv}, i + 2^{31}, \ell_v)$ 
  12 :     $(\delta_{i,0}, \dots, \delta_{i,\text{depth}-1}) \leftarrow \text{BitDec}(\Delta_i, \text{depth})$ 
  13 :    if  $i = 0$  then
  14 :       $\mathbf{Q}_i := [\mathbf{q}_{i,0} \ \dots \ \mathbf{q}_{i,\text{depth}-1}] \in \mathbb{F}_2^{\ell_v \times \text{depth}}$ 
  15 :    else
  16 :       $\mathbf{Q}_i := [\mathbf{q}_{i,0} \ \dots \ \mathbf{q}_{i,\text{depth}-1}] + [\delta_{i,0} \cdot \mathbf{c}_i \ \dots \ \delta_{i,\text{depth}-1} \cdot \mathbf{c}_i] \in \mathbb{F}_2^{\ell_v \times \text{depth}}$ 
  17 :  $\mathbf{Q} := [\mathbf{Q}_0 \ \dots \ \mathbf{Q}_{\tau-1} \ \mathbf{0}_{\ell_v, \lambda - \tau \cdot \text{depth}}] \in \mathbb{F}_2^{\ell_v \times \lambda}$ 
  18 : return  $(\text{com}, \mathbf{Q})$ 

```

Algorithm 17 presents the VOLE reconstruction algorithm `VOLE.Reconstruct`. The input is the challenge $\text{chall} \in \{0, 1\}^{\tau \log_2 N}$, the selected opening $\text{decom}_I \in \{0, 1\}^{2\lambda\tau + T_{\text{open}}\lambda}$, the correction vectors $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1} \in \{0, 1\}^{\ell_v}$, and $\text{iv} \in \{0, 1\}^\lambda$. The output is the root commitment $\text{com} \in \{0, 1\}^{2\lambda}$ and VOLE keys $\mathbf{Q} \in \{0, 1\}^{\ell_v \times \lambda}$, or $\perp$.

- **Phase 0: BAVC reconstruction.** The algorithm parses the parameter tuple param , sets the tree depth $\text{depth} = \log N$, decodes the challenge chall into the hidden index vector $I = (\Delta_0, \dots, \Delta_{\tau-1})$, and calls `BAVC.Reconstruct` on the selected opening decom_I , the hidden index vector I , and iv . If BAVC reconstruction fails, it returns $\perp$; otherwise, it obtains the root commitment com and the revealed seeds $\text{sd}_{i,j}$ (cf. lines 1–7).
- **Phase 1: VOLE key reconstruction.** For each row index i , the algorithm reorders the revealed seeds according to the hidden index Δ_i , calls `VOLE.BuildVOLE` with the hidden seed set to $\perp$, and forms the row block $\mathbf{Q}_i$. For $i \neq 0$, it applies the correction vector $\mathbf{c}_i$ according to the bits of Δ_i . It concatenates the row blocks with zero padding into the VOLE keys $\mathbf{Q}$ and returns $(\text{com}, \mathbf{Q})$ (cf. lines 8–18).

The remaining routines specify the VOLE subroutines used by signing and verification.

- `VOLE.DecodeAllChall` takes the challenge $\text{chall} \in \{0, 1\}^{\tau \log_2 N}$ as input, parses it into the hidden indices $I = (\Delta_0, \dots, \Delta_{\tau-1}) \in [0..N - 1]^\tau$, and outputs I .
- `VOLE.Hash` takes the string $\text{sd} \in \{0, 1\}^{5\lambda + 64}$ and the matrix $X \in \{0, 1\}^{(\ell_w + 3\lambda + B) \times m}$ as input, hashes the columns of X into $\mathbf{H} \in \{0, 1\}^{(\lambda + B) \times m}$, and outputs $\mathbf{H}$; the case $m = 1$ is the single-column case.

- **VOLE.BuildVOLE** takes the N BAVC row seeds $(\mathbf{sd}_0, \dots, \mathbf{sd}_{N-1}) \in (\{0, 1\}^\lambda)^N$, $\mathbf{iv} \in \{0, 1\}^\lambda$, the 32-bit tweak $\mathbf{twk} \in \{0, 1\}^{32}$, and the output length $\ell_v \in \mathbb{N}_{>0}$ as input, expands them into ℓ_v random bits $\mathbf{u}$ and tag vectors $\mathbf{v}_0, \dots, \mathbf{v}_{\text{depth}-1} \in \{0, 1\}^{\ell_v}$, where $\text{depth} = \log N$, and outputs $(\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{\text{depth}-1})$; if one seed is set to $\perp$, the corresponding expansion is replaced by zero.

Algorithm 18 **VOLE.DecodeAllChall**(chall; param)

Input: challenge $\text{chall} \in \{0, 1\}^{\tau \log_2 N}$

Output: $(\Delta_0, \dots, \Delta_{\tau-1}) \in [0..N-1]^\tau$

```

1 : Parse param := ( $\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v$ )
2 : depth :=  $\log N$ 
3 : for  $i \in [0..\tau-1]$ 
4 :   lo :=  $i \cdot \text{depth}$ 
5 :   hi :=  $(i+1) \cdot \text{depth} - 1$ 
6 :    $b_i := \text{chall}[lo..hi]$ 
7 :    $\Delta_i \leftarrow \text{IntRec}(b_i, \text{depth})$ 
8 : return  $(\Delta_0, \dots, \Delta_{\tau-1})$ 

```

Algorithm 19 **VOLE.Hash**(sd, X ; param)

Input: $\mathbf{sd} \in \{0, 1\}^{5\lambda+64}$ and $X \in \{0, 1\}^{\ell_v \times m}$

Output: $H \in \{0, 1\}^{(\lambda+B) \times m}$

```

1 : Parse param := ( $\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v$ )
2 : Parse  $\mathbf{sd} := (r_0 \| r_1 \| r_2 \| r_3 \| s \| t)$  ▷  $r_i, s$  are  $\lambda$ -bit strings;  $t$  is a 64-bit string
3 :  $r_i \leftarrow \text{BitsToFields}(r_i, \lambda)$  for  $i \in [0..3]$ 
4 :  $s \leftarrow \text{BitsToFields}(s, \lambda)$ 
5 :  $t \leftarrow \text{BitsToFields}(t, 64)$ 
6 :  $\ell_{\text{pad}} := \lambda \cdot \lceil (\ell_v - (\lambda + B)) / \lambda \rceil$ 
7 : for  $j \in [0..m-1]$ 
8 :   Parse  $X_{*,j} := \mathbf{x}_0 \| \mathbf{x}_1$  with  $\mathbf{x}_0 \in \{0, 1\}^{\ell_v - (\lambda + B)}$  and  $\mathbf{x}_1 \in \{0, 1\}^{\lambda + B}$ 
9 :    $\mathbf{x}_0 := \mathbf{x}_0 \| 0^{\ell_{\text{pad}} - (\ell_v - (\lambda + B))}$  ▷ Pad to a multiple of  $\lambda$ 
10 :   $\hat{\mathbf{y}} \leftarrow \text{BitsToFields}(\mathbf{x}_0, \lambda)$ 
11 :   $\bar{\mathbf{y}} \leftarrow \text{BitsToFields}(\mathbf{x}_0, 64)$ 
12 :   $h_0 := \sum_{i=0}^{\ell_{\text{pad}}/\lambda-1} s^{\ell_{\text{pad}}/\lambda-1-i} \cdot \hat{\mathbf{y}}[i]$  in  $\mathbb{F}_{2^\lambda}$ 
13 :   $h_1 := \sum_{i=0}^{\ell_{\text{pad}}/64-1} t^{\ell_{\text{pad}}/64-1-i} \cdot \bar{\mathbf{y}}[i]$  in  $\mathbb{F}_{2^{64}}$ 
14 :   $h'_1 \leftarrow \text{BitsToFields}(\text{FieldsToBits}(h_1, 64) \| 0^{\lambda-64}, \lambda)$ 
15 :   $(h_2, h_3) := (r_0 h_0 + r_1 h'_1, r_2 h_0 + r_3 h'_1)$ 
16 :   $H_{*,j} := (\text{FieldsToBits}(h_2, \lambda) \| \text{FieldsToBits}(h_3, \lambda)[0..B-1]) \oplus \mathbf{x}_1$ 
17 : return  $H$ 

```

Algorithm 20 `VOLE.BuildVOLE`($\text{sd}_0, \dots, \text{sd}_{N-1}, \text{iv}, \text{twk}, \ell_v; \text{param}$)

Input: $(\text{sd}_0, \dots, \text{sd}_{N-1}) \in (\{0, 1\}^\lambda)^N$, $\text{iv} \in \{0, 1\}^\lambda$, $\text{twk} \in \{0, 1\}^{32}$, and $\ell_v \in \mathbb{N}_{>0}$

Output: $\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{\text{depth}-1} \in \{0, 1\}^{\ell_v}$

```
1 : Parse  $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$ 
2 :  $\text{depth} := \log N$ 
3 : for  $i \in [0..N-1]$ 
4 :   if  $\text{sd}_i = \perp$  then
5 :      $\mathbf{r}_{0,i} := \mathbf{0}^{\ell_v}$ 
6 :   else
7 :      $\mathbf{r}_{0,i} \leftarrow \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \ell_v)$ 
8 :  $\mathbf{v}_0 := \dots := \mathbf{v}_{\text{depth}-1} := \mathbf{0}^{\ell_v}$ 
9 : for  $j \in [0..\text{depth}-1]$ 
10 :  for  $i \in [0..N/2^{j+1}-1]$ 
11 :     $\mathbf{v}_j := \mathbf{v}_j \oplus \mathbf{r}_{j,2i+1}$ 
12 :     $\mathbf{r}_{j+1,i} := \mathbf{r}_{j,2i} \oplus \mathbf{r}_{j,2i+1}$ 
13 :  $\mathbf{u} := \mathbf{r}_{\text{depth},0}$ 
14 : return  $(\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{\text{depth}-1})$ 
```

4.2.5 RSD Subroutines

Algorithm 21 presents the RSD proving algorithm `RSD.Prove`. The input consists of

$$\mathbf{w} \in \{0, 1\}^{w \sum_{j=0}^{l-1} \eta_j}, \quad \mathbf{u} \in \mathbb{F}_2^{(d-1)\lambda}, \quad \mathbf{V} \in \mathbb{F}_2^{(\ell_w + (d-1)\lambda) \times \lambda},$$

together with the public key $\mathbf{pk} \in \{0, 1\}^\lambda \times \mathbb{F}_2^{n-k}$ and the challenge $\text{chall} \in \{0, 1\}^{\beta\lambda + 3\lambda + 64}$. The output is the RSD proof $\pi_{\text{RSD}} \in \{0, 1\}^{(d-1)\lambda}$ and RSD check value $\text{chk}_{\text{RSD}} \in \{0, 1\}^\lambda$. The notation used in the RSD proof follows the polynomial encoding framework in Section 3.1 and Section 3.1.1. In particular, each witness block is parsed as sub-vectors $\mathbf{s}_q^{(j)}$, the reconstruction polynomials f_t map these sub-vectors to elementary-vector entries, and the membership polynomials $\rho_{j,T}$ are defined in Equation (6). The element g_λ is the generator used by `BitsToFields`; thus powers of g_λ are used below in the same field representation. For a bit x and a field element $v \in \mathbb{F}_{2^\lambda}$, the notation $\langle x \rangle = (x, v)$ represents the linear polynomial $v + xX$. Thus, evaluating $\langle x \rangle$ at $X = \delta$ gives $v + x\delta$, which is the value reconstructed by the verifier from the corresponding VOLE receiver value. When a polynomial such as f_t or $\rho_{j,T}$ is applied to values in angle brackets, it means that these linear polynomials are substituted into it and the resulting polynomial is expanded into coefficients.

- **Phase 0: RSD parameters.** The prover parses the common parameter tuple, the RSD parameter tuple, and the public key $\mathbf{pk} = (\text{seed}_{pk}, \mathbf{y})$. It then fixes the quantities used by the proof: the constraint degree $d = \sum_{j=0}^{l-1} r_j$, the number $\beta = \lceil (n-k)/\lambda \rceil$ of packed syndrome field elements, and the block length $m = n/w$ for each elementary-vector block (cf. lines 1–6).
- **Phase 1: VOLE-authenticated witness.** The witness bits are converted into linear polynomials $\langle w_i \rangle = (w_i, V_i)$, representing $V_i + w_iX$, where V_i is obtained by applying `BitsToFields` to the corresponding row of $\mathbf{V}$. The remaining rows of $\mathbf{u}$ and $\mathbf{V}$ are interpreted in the same field representation and combined into $b_0, \dots, b_{d-1}$. These values are the mask coefficients later added to the degree coefficients of the constraint polynomials before hashing (cf. lines 7–14).
- **Phase 2: Public relation and folding challenge.** The challenge chall is parsed so that its first $\beta\lambda$ bits define $\alpha_0, \dots, \alpha_{\beta-1} \in \mathbb{F}_{2^\lambda}$, which are used by `ZKHash` to fold the β packed syndrome coordinates. The prover also expands seed_{pk} into the packed public columns $\mathbf{h}_0, \dots, \mathbf{h}_{n-1}$ and parses $\mathbf{y}$, padded to a multiple of λ , as $(y_0, \dots, y_{\beta-1})$ (cf. lines 15–18).
- **Phase 3: RSD constraints.** For each nonzero position block q , the authenticated witness is sliced into the sub-vectors $\langle \mathbf{s}_q^{(j)} \rangle$ used by the encoding framework in Section 3.1.1. For every block j and every subset T of size $r_j + 1$, the prover forms the polynomial $\rho_{j,T}(\langle \mathbf{s}_q^{(j)} \rangle)$ and records all its degree coefficients in the vectors $\mathbf{a}_0, \dots, \mathbf{a}_{d-1}$. The same authenticated sub-vectors are then mapped by the reconstruction polynomials f_t to authenticated elementary-vector entries $\langle e_{q,t} \rangle$. Finally, for each packed syndrome coordinate b , the prover writes the relation $\sum_{q,t} \mathbf{h}_{qm+t}[b] \langle e_{q,t} \rangle + y_b$ as a degree- d polynomial and stores its coefficients in $\mathbf{z}_0, \dots, \mathbf{z}_{d-1}$ (cf. lines 19–31).

- **Phase 4: Proof output.** For each degree r , **ZKHash** hashes the packed-syndrome coefficient vector $\mathbf{z}_r$, the collected membership-coefficient vector $\mathbf{a}_r$, and the mask coefficient b_r under the same challenge string. The hash for degree 0 is the check value chk_{RSD} , and the hashes for degrees $1, \dots, d-1$ are concatenated as π_{RSD} (cf. lines 32–35).

Algorithm 21 **RSD.Prove**($\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}; \text{param}, \text{param}_{\text{RSD}}$)

Input: $\mathbf{w} \in \{0,1\}^{w \sum_{j=0}^{l-1} \eta_j}$, $\mathbf{u} \in \mathbb{F}_2^{(d-1)\lambda}$, $\mathbf{V} \in \mathbb{F}_2^{(\ell_w + (d-1)\lambda) \times \lambda}$, $\text{pk} \in \{0,1\}^\lambda \times \mathbb{F}_2^{n-k}$, and $\text{chall} \in \{0,1\}^{\beta\lambda + 3\lambda + 64}$; **param**, **param**_{RSD}

Output: $\pi_{\text{RSD}} \in \{0,1\}^{(d-1)\lambda}$ and $\text{chk}_{\text{RSD}} \in \{0,1\}^\lambda$

▷ Phase 0: RSD parameters

1 : Parse **param** := $(\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$

2 : Parse **param**_{RSD} := $(n, k, w, d, l, (\eta_j)_j, (r_j)_j)$

3 : Parse **pk** := $(\text{seed}_{\text{pk}}, \mathbf{y})$

4 : $d := \sum_{j=0}^{l-1} r_j$

5 : $\beta := \lceil (n-k)/\lambda \rceil$

6 : $m := n/w$

▷ Phase 1: VOLE-authenticated witness

7 : **for** $i \in [0..\ell_w - 1]$ **do** $\langle w_i \rangle := (\mathbf{w}[i], \text{BitsToFields}(\mathbf{V}_{i,*}, \lambda))$

▷ Represents $V_i + w_i X$

8 : Let g_λ be the generator of $\mathbb{F}_{2^\lambda}$ used by **BitsToFields**

9 : **for** $j \in [0..d-2]$ **do**

10 : $u_j^* := \text{BitsToFields}(\mathbf{u}[j\lambda..(j+1)\lambda-1], \lambda)$

11 : $v_j^* := \sum_{i=0}^{\lambda-1} \text{BitsToFields}(\mathbf{V}_{\ell_w+j\lambda+i,*}, \lambda) \cdot g_\lambda^i$

▷ Power-basis packing

12 : $b_0 := v_0^*$

13 : $b_j := u_{j-1}^* + v_j^*$ for $j \in [1..d-2]$

14 : $b_{d-1} := u_{d-2}^*$

▷ Phase 2: Public relation and folding challenge

15 : Parse **chall** := $\alpha_0 \| \dots \| \alpha_{\beta-1} \| r_0 \| r_1 \| s \| t$

▷ α_i, r_i, s are λ -bit strings; t is a 64-bit string

16 : $(\mathbf{h}_0, \dots, \mathbf{h}_{n-1}) \leftarrow \text{RSD.ExpandPackedH}(\text{seed}_{\text{pk}})$

17 : $\alpha_b \leftarrow \text{BitsToFields}(\alpha_b, \lambda)$ for $b \in [0..\beta-1]$

▷ Folding challenge

18 : Parse $\mathbf{y} \| 0^{\lambda\beta - (n-k)}$ as $(y_0, \dots, y_{\beta-1}) \in \mathbb{F}_{2^\lambda}^\beta$

▷ Packed syndrome

▷ Phase 3: RSD constraints

19 : $\mathbf{a}_r := \epsilon$ for $r \in [0..d-1]$

20 : **for** $q \in [0..w-1]$ **do**

21 : **for** $j \in [0..l-1]$ **do**

22 : $o_{q,j} := q \sum_{h=0}^{l-1} \eta_h + \sum_{h=0}^{j-1} \eta_h$

23 : $\langle \mathbf{s}_q^{(j)} \rangle := (\langle w_{o_{q,j}+r} \rangle)_{r \in [0..\eta_j-1]}$

▷ Authenticated sub-vector

24 : **for** $T \in \binom{[\eta_j]}{r_{j+1}}$ **do**

25 : Parse $\rho_{j,T}(\langle \mathbf{s}_q^{(j)} \rangle) = \sum_{r=0}^{d-1} a_r^{(q,j,T)} X^r$

▷ Coefficient expansion

26 : Append $a_r^{(q,j,T)}$ to $\mathbf{a}_r$ for every $r \in [0..d-1]$

27 : **for** $t \in [0..m-1]$ **do** $\langle e_{q,t} \rangle := f_t(\langle \mathbf{s}_q^{(0)} \rangle, \dots, \langle \mathbf{s}_q^{(l-1)} \rangle)$

▷ Reconstruction polynomial

28 : **for** $b \in [0..\beta-1]$ **do**

29 : $\langle Z_b \rangle := \sum_{q=0}^{w-1} \sum_{t=0}^{m-1} \mathbf{h}_{qm+t}[b] \cdot \langle e_{q,t} \rangle + y_b$

▷ Packed syndrome relation

30 : Parse $\langle Z_b \rangle = \sum_{r=0}^{d-1} z_{r,b} X^r$

31 : $\mathbf{z}_r := (z_{r,0}, \dots, z_{r,\beta-1})$ for $r \in [0..d-1]$

▷ Phase 4: Proof output

32 : $\tilde{a}_r := \text{ZKHash}(\text{chall}, \mathbf{z}_r, \mathbf{a}_r, b_r)$ for $r \in [0..d-1]$

33 : $\pi_{\text{RSD}} := \tilde{a}_1 \| \dots \| \tilde{a}_{d-1}$

34 : $\text{chk}_{\text{RSD}} := \tilde{a}_0$

35 : **return** $(\pi_{\text{RSD}}, \text{chk}_{\text{RSD}})$

Algorithm 22 presents the RSD verification algorithm `RSD.Verify`. The input consists of

$$\mathbf{d} \in \{0, 1\}^{\ell_w}, \quad \mathbf{Q} \in \mathbb{F}_2^{\ell_v \times \lambda}, \quad \mathbf{pk} \in \{0, 1\}^\lambda \times \mathbb{F}_2^{n-k},$$

together with the challenge $\mathbf{chall} \in \{0, 1\}^{\beta\lambda+3\lambda+64}$, the final challenge $\mathbf{chall}_3 \in \{0, 1\}^\lambda$, and the RSD proof $\pi_{\text{RSD}} \in \{0, 1\}^{(d-1)\lambda}$. The output is the recomputed RSD check value $\text{chk}_{\text{RSD}} \in \{0, 1\}^\lambda$. The verification algorithm is the evaluation counterpart of `RSD.Prove`: instead of constructing the coefficient vectors of the degree- d constraint polynomials, it evaluates the same constraints at the point δ derived from $\mathbf{chall}_3$. It uses the same sub-vector parsing, reconstruction polynomials f_t , and membership polynomials $\rho_{j,T}$ as in Section 3.1.1.

- **Phase 0: RSD and challenge values.** The verifier parses the common parameter tuple, the RSD parameter tuple, and the public key $\mathbf{pk} = (\text{seed}_{pk}, \mathbf{y})$. It recomputes $d = \sum_{j=0}^{l-1} r_j$, $\beta = \lceil (n-k)/\lambda \rceil$, and $m = n/w$, and parses $\mathbf{chall}_3$ as the field element $\delta \in \mathbb{F}_{2^\lambda}$. This point δ is used to evaluate the same degree- d polynomial relations whose coefficients were committed to by the prover (cf. lines 1–7).
- **Phase 1: Evaluated authenticated witness.** For every witness coordinate, the verifier combines the masked bit $\mathbf{d}[i]$ with the corresponding row of $\mathbf{Q}$ to obtain $w_i^* = Q_i + \mathbf{d}[i]\delta$, which is the value of the authenticated witness polynomial at $X = \delta$. The final $(d-1)\lambda$ rows of $\mathbf{Q}$ are parsed in the field representation of `BitsToFields`. Their power-basis combination gives q_j^* for each nonconstant degree, and $q^* = \sum_j q_j^* \delta^j$ is the evaluated mask term used in the final hash (cf. lines 8–12).
- **Phase 2: Public relation and folding challenge.** The verifier parses $\mathbf{chall}$ exactly as in `RSD.Prove`. The first $\beta\lambda$ bits give the folding field elements $\alpha_0, \dots, \alpha_{\beta-1}$ used by `ZKHash`, while the remaining bits define its hash seed. The verifier also expands seed_{pk} into the packed public columns $\mathbf{h}_0, \dots, \mathbf{h}_{n-1}$ and parses the padded syndrome as $(y_0, \dots, y_{\beta-1})$ (cf. lines 13–16).
- **Phase 3: Evaluated RSD constraints.** The verifier slices the values w_i^* into the same sub-vectors $\mathbf{s}_q^{(j),*}$ used in the proving algorithm. For each membership constraint, it evaluates $\rho_{j,T}(\mathbf{s}_q^{(j),*})$ and appends the result to $\mathbf{a}^*$. For the reconstruction part, it evaluates $f_t(\mathbf{s}_q^{(0),*}, \dots, \mathbf{s}_q^{(l-1),*})$ to obtain $e_{q,t}^*$. The packed syndrome equations are then evaluated as $z_b^* = \sum_{q,t} \mathbf{h}_{qm+t}[b] e_{q,t}^* + y_b$, giving $\mathbf{z}^* = (z_0^*, \dots, z_{\beta-1}^*)$ (cf. lines 17–27).
- **Phase 4: Check value reconstruction.** The verifier hashes the evaluated packed-syndrome vector $\mathbf{z}^*$, the evaluated membership vector $\mathbf{a}^*$, and the evaluated mask term q^* to obtain $\tilde{q}$. The proof π_{RSD} contains the values $\tilde{a}_1, \dots, \tilde{a}_{d-1}$ for degrees $1, \dots, d-1$. Therefore the verifier subtracts $\sum_{r=1}^{d-1} \tilde{a}_r \delta^r$ from $\tilde{q}$ and outputs the resulting constant term as chk_{RSD} (cf. lines 28–32).

Algorithm 22 $\text{RSD.Verify}(d, Q, \text{pk}, \text{chall}, \text{chall}_3, \pi_{\text{RSD}}; \text{param}, \text{param}_{\text{RSD}})$

Input: $d \in \{0, 1\}^{\ell_w}$, $Q \in \mathbb{F}_2^{\ell_v \times \lambda}$, $\text{pk} \in \{0, 1\}^\lambda \times \mathbb{F}_2^{n-k}$, $\text{chall} \in \{0, 1\}^{\beta\lambda+3\lambda+64}$, $\text{chall}_3 \in \{0, 1\}^\lambda$, and $\pi_{\text{RSD}} \in \{0, 1\}^{(d-1)\lambda}$; $\text{param}, \text{param}_{\text{RSD}}$

Output: $\text{chk}_{\text{RSD}} \in \{0, 1\}^\lambda$

▷ Phase 0: RSD and challenge values

- 1 : Parse $\text{param} := (\lambda, \tau, N, L, w_{\text{grind}}, T_{\text{open}}, B, \ell_w, \ell_v)$
- 2 : Parse $\text{param}_{\text{RSD}} := (n, k, w, d, l, (\eta_j)_j, (r_j)_j)$
- 3 : Parse $\text{pk} := (\text{seed}_{\text{pk}}, \mathbf{y})$
- 4 : $d := \sum_{j=0}^{l-1} r_j$
- 5 : $\beta := \lceil (n-k)/\lambda \rceil$
- 6 : $m := n/w$
- 7 : $\delta \leftarrow \text{BitsToFields}(\text{chall}_3[0..\lambda-1], \lambda)$ ▷ Evaluation point
- ▷ Phase 1: Evaluated authenticated witness
- 8 : **for** $i \in [0..\ell_w-1]$ **do** $w_i^* := \text{BitsToFields}(Q_{i,*}, \lambda) + d[i] \cdot \delta$ ▷ Evaluate $V_i + w_i X$ at $X = \delta$
- 9 : Let g_λ be the generator of $\mathbb{F}_{2^\lambda}$ used by BitsToFields
- 10 : **for** $j \in [0..d-2]$ **do**
- 11 : $q_j^* := \sum_{i=0}^{\lambda-1} \text{BitsToFields}(Q_{\ell_w+j\lambda+i,*}, \lambda) \cdot g_\lambda^i$ ▷ Power-basis packing
- 12 : $q^* := \sum_{j=0}^{d-2} q_j^* \cdot \delta^j$
- ▷ Phase 2: Public relation and folding challenge
- 13 : Parse $\text{chall} := \alpha_0 \| \dots \| \alpha_{\beta-1} \| r_0 \| r_1 \| s \| t$ ▷ α_i, r_i, s are λ -bit strings; t is a 64-bit string
- 14 : $(\mathbf{h}_0, \dots, \mathbf{h}_{n-1}) \leftarrow \text{RSD.ExpandPackedH}(\text{seed}_{\text{pk}})$
- 15 : $\alpha_b \leftarrow \text{BitsToFields}(\alpha_b, \lambda)$ for $b \in [0..\beta-1]$ ▷ Folding challenge
- 16 : Parse $\mathbf{y} \| 0^{\lambda\beta-(n-k)}$ as $(y_0, \dots, y_{\beta-1}) \in \mathbb{F}_{2^\lambda}^\beta$ ▷ Packed syndrome
- ▷ Phase 3: Evaluated RSD constraints
- 17 : $\mathbf{a}^* := \epsilon$
- 18 : **for** $q \in [0..w-1]$ **do**
- 19 : **for** $j \in [0..l-1]$ **do**
- 20 : $o_{q,j} := q \sum_{h=0}^{l-1} \eta_h + \sum_{h=0}^{j-1} \eta_h$
- 21 : $\mathbf{s}_q^{(j),*} := (w_{o_{q,j}+r}^*)_{r \in [0..\eta_j-1]}$ ▷ Sub-vector evaluation
- 22 : **for** $T \in \binom{[\eta_j]}{r_j+1}$ **do**
- 23 : $a_{q,j,T}^* := \rho_{j,T}(\mathbf{s}_q^{(j),*})$ ▷ Membership constraint
- 24 : Append $a_{q,j,T}^*$ to $\mathbf{a}^*$
- 25 : **for** $t \in [0..m-1]$ **do** $e_{q,t}^* := f_t(\mathbf{s}_q^{(0),*}, \dots, \mathbf{s}_q^{(l-1),*})$ ▷ Reconstruction polynomial
- 26 : **for** $b \in [0..\beta-1]$ **do**
- 27 : $z_b^* := \sum_{q=0}^{w-1} \sum_{t=0}^{m-1} \mathbf{h}_{qm+t}[b] \cdot e_{q,t}^* + y_b$ ▷ Packed syndrome relation
- 28 : $\mathbf{z}^* := (z_0^*, \dots, z_{\beta-1}^*)$
- ▷ Phase 4: Check value reconstruction
- 29 : Parse $\pi_{\text{RSD}} := \tilde{a}_1 \| \dots \| \tilde{a}_{d-1}$ with $\tilde{a}_r \in \{0, 1\}^\lambda$
- 30 : $\tilde{q} := \text{ZKHash}(\text{chall}, \mathbf{z}^*, \mathbf{a}^*, q^*)$
- 31 : $A := \sum_{r=1}^{d-1} \text{BitsToFields}(\tilde{a}_r, \lambda) \cdot \delta^r$
- 32 : $\text{chk}_{\text{RSD}} := \text{FieldsToBits}(\text{BitsToFields}(\tilde{q}, \lambda) - A, \lambda)$
- 33 : **return** chk_{RSD}

The remaining routines specify the helper subroutines used by key generation, RSD proving, and RSD verification.

- RSD.ExpandWitness takes the secret-key seed $\text{seed}_{\text{sk}} \in \{0, 1\}^\lambda$ as input, expands it into the RSD solution $\mathbf{x} \in \mathbb{F}_2^n$ and witness $\mathbf{w} \in \{0, 1\}^{w \sum_{j=0}^{l-1} \eta_j}$, and outputs $\mathbf{x}, \mathbf{w}$.
- MixedRadixDecomp takes the offset $p \in [0, \prod_{j=0}^{l-1} D_j)$ and the table sizes $D_j \in \mathbb{N}_{>0}$ for $j \in [0..l-1]$ as input, interprets p as the index tuple $(a_0, \dots, a_{l-1}) \in \prod_{j=0}^{l-1} [0, D_j)$ used to select entries from the tables Table_j , and outputs $(a_0, \dots, a_{l-1})$. For $j \in [0..l-1]$, the set used for the j -th block of

the witness is

$$S_j := \{\mathbf{s} \in \{0, 1\}^{\eta_j} : \text{wt}(\mathbf{s}) \leq r_j\},$$

where $\text{wt}(\mathbf{s})$ is the Hamming weight of $\mathbf{s}$, and the size of S_j is $D_j = |S_j| = \sum_{t=0}^{r_j} \binom{\eta_j}{t}$. The table $\text{Table}_j[0..D_j - 1]$ is obtained by listing the elements of S_j in graded lexicographic order induced by the coordinate order $0, \dots, \eta_j - 1$: vectors are ordered first by increasing Hamming weight, and vectors with the same Hamming weight are ordered lexicographically with $0 < 1$ in every coordinate.

- **RejectionSample** takes the seed $\text{seed} \in \{0, 1\}^\lambda$, the counter $c \in \mathbb{N}$, and the range length $L \in \mathbb{N}_{>0}$ as input, samples $p \in [0, L)$ and updates the counter c , and outputs (p, c) .
- **RSD.ExpandPartialH** takes the public-matrix seed $\text{seed}_{pk} \in \{0, 1\}^\lambda$ and the RSD solution vector $\mathbf{x} \in \mathbb{F}_2^n$ as input, computes the syndrome vector $\mathbf{y} \in \mathbb{F}_2^{n-k}$, and outputs $\mathbf{y}$.
- **RSD.ExpandPackedH** takes the public-matrix seed $\text{seed}_{pk} \in \{0, 1\}^\lambda$ as input, expands each column of $\mathbf{H}$ from **SeedExp** and packs it into the field-vector form $(\mathbf{h}_j)_{j \in [0..n-1]} \in (\mathbb{F}_{2^\lambda}^\beta)^n$, and outputs $(\mathbf{h}_j)_{j \in [0..n-1]}$.
- **ZKHash** takes the hash seed $\text{sd} \in \{0, 1\}^{\beta\lambda+3\lambda+64}$, the coefficient vector $\mathbf{x}_0 \in \mathbb{F}_{2^\lambda}^\beta$, the coefficient vector $\mathbf{x}_1 \in \mathbb{F}_{2^\lambda}^m$, and the masking field element $x_2 \in \mathbb{F}_{2^\lambda}$ as input, hashes these values into $h \in \{0, 1\}^\lambda$, and outputs h .

Algorithm 23 **RSD.ExpandWitness**($\text{seed}_{sk}; \text{param}_{\text{RSD}}$)

Input: $\text{seed}_{sk} \in \{0, 1\}^\lambda$

Output: $\mathbf{x} \in \mathbb{F}_2^n$ and $\mathbf{w} \in \{0, 1\}^{w \sum_{j=0}^{l-1} \eta_j}$

```

1 : Parse  $\text{param}_{\text{RSD}} := (n, k, w, d, l, (\eta_j)_{j \in [0..l-1]}, (r_j)_{j \in [0..l-1]})$ 
2 :  $L := n/w$ 
3 :  $c := 0$  ▷ 32-bit block counter
4 : for  $j \in [0..l-1]$ 
5 :    $D_j := \sum_{t=0}^{r_j} \binom{\eta_j}{t}$ 
6 :  $\mathbf{x} := 0^n$ 
7 :  $\mathbf{w} := \epsilon$ 
8 : for  $i \in [0..w-1]$ 
9 :    $(p_i, c) \leftarrow \text{RejectionSample}(\text{seed}_{sk}, c, L)$ 
10 :    $\mathbf{x}[iL + p_i] := 1$ 
11 :    $(a_{i,0}, \dots, a_{i,l-1}) \leftarrow \text{MixedRadixDecomp}(p_i, (D_j)_{j=0}^{l-1})$ 
12 :   for  $j \in [0..l-1]$ 
13 :      $\mathbf{s}_{i,j} := \text{Table}_j[a_{i,j}]$ 
14 :    $\mathbf{w} := \mathbf{w} \parallel \mathbf{s}_{i,0} \parallel \dots \parallel \mathbf{s}_{i,l-1}$ 
15 : return  $(\mathbf{x}, \mathbf{w})$ 

```

Algorithm 24 **MixedRadixDecomp**($p, (D_j)_{j=0}^{l-1}$)

Input: $p \in [0, \prod_{j=0}^{l-1} D_j)$ and $D_j \in \mathbb{N}_{>0}$ for $j \in [0..l-1]$

Output: $(a_0, \dots, a_{l-1}) \in \prod_{j=0}^{l-1} [0, D_j)$

```

1 : for  $j \in [0..l-1]$ 
2 :    $a_j := \left\lfloor p / \prod_{h=j+1}^{l-1} D_h \right\rfloor \bmod D_j$  ▷ The empty product is 1
3 : return  $(a_0, \dots, a_{l-1})$ 

```

Algorithm 25 [RejectionSample](#)(seed, c , L)

Input: seed $\in \{0, 1\}^\lambda$, $c \in \mathbb{N}$, and $L \in \mathbb{N}_{>0}$ **Output:** $p \in [0, L)$ and $c \in \mathbb{N}$

```
1 :  $T := \lfloor 2^{32}/L \rfloor L$ 
2 : repeat
3 :    $b \leftarrow \text{Hash}_3(\text{seed} \parallel \text{BitDec}(c, 32); 32)$ 
4 :    $b := \text{IntRec}(b, 32)$ 
5 :    $c := c + 1$ 
6 : until  $b < T$ 
7 :  $p := b \bmod L$ 
8 : return  $(p, c)$ 
```

Algorithm 26 [RSD.ExpandPartialH](#)(seed _{pk} , $\mathbf{x}$; param_{RSD})

Input: seed _{pk} $\in \{0, 1\}^\lambda$ and $\mathbf{x} \in \mathbb{F}_2^n$ **Output:** $\mathbf{y} \in \mathbb{F}_2^{n-k}$

```
1 : Parse paramRSD :=  $(n, k, w, d, l, (\eta_j)_j, (r_j)_j)$ 
2 : codim :=  $n - k$ 
3 :  $\beta := \lceil \text{codim}/\lambda \rceil$  ▷ Number of  $\lambda$ -bit blocks per column of  $\mathbf{H}$ 
4 :  $\gamma := \lceil \lambda/\nu \rceil$  ▷  $\nu = 128$  for  $\lambda \in \{160, 256\}$ ;  $\nu = 512$  for  $\lambda = 512$ 
5 :  $J := \{j \in [0..n-1] : \mathbf{x}[j] = 1\}$  ▷ Columns selected by the sparse vector  $\mathbf{x}$ 
6 : for  $a \in [0..\beta-1]$ 
7 :    $\mathbf{y}_a := 0^\lambda$ 
8 : for  $j \in J$  ▷ Expand only selected columns from SeedExp
9 :   for  $a \in [0..\beta-1]$  ▷ For each block
10 :     $\mathbf{s} := \epsilon$ 
11 :    for  $q \in [0..\gamma-1]$ 
12 :       $c := (an + j)\gamma + q$  ▷ 32-bit counter
13 :       $\mathbf{s} := \mathbf{s} \parallel \text{SeedExp}(\text{seed}_{pk}, \text{BitDec}(c, 32) \parallel 0^{\nu-32})$ 
14 :       $\mathbf{h}_{a,j} := \mathbf{s}[0..\lambda-1]$  ▷ Keep the first  $\lambda$  bits
15 :       $\mathbf{y}_a := \mathbf{y}_a \oplus \mathbf{h}_{a,j}$  ▷ Accumulate  $\mathbf{y}_a$ 
16 :  $\mathbf{y} := (\mathbf{y}_0 \parallel \dots \parallel \mathbf{y}_{\beta-1})[0..\text{codim}-1]$ 
17 : return  $\mathbf{y}$ 
```

Algorithm 27 $\text{RSD.ExpandPackedH}(\text{seed}_{pk}; \text{param}_{\text{RSD}})$

Input: $\text{seed}_{pk} \in \{0, 1\}^\lambda$ **Output:** $(\mathbf{h}_j)_{j \in [0..n-1]} \in (\mathbb{F}_{2^\lambda}^\beta)^n$

```
1 : Parse  $\text{param}_{\text{RSD}} := (n, k, w, d, l, (\eta_j)_j, (r_j)_j)$ 
2 :  $\text{codim} := n - k$ 
3 :  $\beta := \lceil \text{codim} / \lambda \rceil$  ▷ Number of  $\lambda$ -bit blocks per column of  $\mathbf{H}$ 
4 :  $\gamma := \lceil \lambda / \nu \rceil$  ▷  $\nu = 128$  for  $\lambda \in \{160, 256\}$ ;  $\nu = 512$  for  $\lambda = 512$ 
5 : for  $j \in [0..n-1]$ 
6 :   for  $a \in [0..\beta-1]$ 
7 :      $\ell_a := \min\{\lambda, \text{codim} - a\lambda\}$ 
8 :      $\mathbf{s} := \epsilon$ 
9 :     for  $q \in [0..\gamma-1]$ 
10 :       $c := (an + j)\gamma + q$  ▷ 32-bit counter
11 :       $\mathbf{s} := \mathbf{s} \parallel \text{SeedExp}(\text{seed}_{pk}, \text{BitDec}(c, 32) \parallel 0^{\nu-32})$ 
12 :       $\mathbf{u}_{a,j} := \mathbf{s}[0..\ell_a-1] \parallel 0^{\lambda-\ell_a}$ 
13 :       $\mathbf{h}_j[a] := \text{BitsToFields}(\mathbf{u}_{a,j}, \lambda)$ 
14 : return  $(\mathbf{h}_j)_{j \in [0..n-1]}$ 
```

Algorithm 28 $\text{ZKHash}(\text{sd}, \mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2)$

Input: $\text{sd} \in \{0, 1\}^{\beta\lambda+3\lambda+64}$, $\mathbf{x}_0 \in \mathbb{F}_{2^\lambda}^\beta$, $\mathbf{x}_1 \in \mathbb{F}_{2^\lambda}^m$, and $\mathbf{x}_2 \in \mathbb{F}_{2^\lambda}$ **Output:** $h \in \{0, 1\}^\lambda$

```
1 : Parse  $\text{sd} := (g_0 \parallel \dots \parallel g_{\beta-1} \parallel r_0 \parallel r_1 \parallel s \parallel t)$  ▷  $r_i, g_i, s$  are  $\lambda$ -bit strings;  $t$  is a 64-bit string
2 :  $g_i \leftarrow \text{BitsToFields}(g_i, \lambda)$  for  $i \in [0..\beta-1]$ 
3 :  $r_i \leftarrow \text{BitsToFields}(r_i, \lambda)$  for  $i \in [0..1]$ 
4 :  $s \leftarrow \text{BitsToFields}(s, \lambda)$ 
5 :  $t \leftarrow \text{BitsToFields}(t \parallel 0^{\lambda-64}, \lambda)$ 
6 :  $z := \sum_{i=0}^{\beta-1} g_i \cdot \mathbf{x}_0[i]$ 
7 :  $h_0 := \sum_{i=0}^{m-1} s^{m-1-i} \cdot \mathbf{x}_1[i]$ 
8 :  $h_1 := \sum_{i=0}^{m-1} t^{m-1-i} \cdot \mathbf{x}_1[i]$ 
9 :  $h \leftarrow \text{FieldsToBits}(z + r_0 h_0 + r_1 h_1 + \mathbf{x}_2, \lambda)$ 
10 : return  $h$ 
```

5 Design Rationale

5.1 Hardness Problem

5.1.1 Syndrome Decoding Problem

The core hardness assumption underlying SYDO is the syndrome decoding (SD) problem for random linear codes over the binary field, which we recall below.

Let n, k and w be positive integers such that $n > k$ and $n > w$. The syndrome decoding problem with parameters (n, k, w) is the following problem: Let $\mathbf{H}$, $\mathbf{x}$ and $\mathbf{y}$ be such that:

1. $\mathbf{H}$ is uniformly sampled from $\mathbb{F}_2^{(n-k) \times n}$,
2. $\mathbf{x}$ is uniformly sampled from $\{\mathbf{x} \in \mathbb{F}_2^n : \text{wt}(\mathbf{x}) = w\}$,
3. $\mathbf{y}$ is defined as $\mathbf{y} := \mathbf{H}\mathbf{x}$.

From $(\mathbf{H}, \mathbf{y})$, find $\mathbf{x}$.

Table 5.1. Selection of symmetric primitives in SYDO

Parameter Sets	PRG	SeedExp
SYDO-160	Rijndael-192*	AES-192*
SYDO-256	Rijndael-256	AES-256
SYDO-512	BLAKE2s	BLAKE2s

* Truncated input/output is used.

5.1.2 Cryptographic Primitives

Except for the SD problem, SYDO relies on the security of standard symmetric primitives, including block ciphers and hash functions. The concrete instantiations of cryptographic primitives used in SYDO are summarized in Table 5.1.

Block Ciphers and PRGs. SYDO uses the IV-based PRG [PRG](#) and the seed-expansion interface [SeedExp](#).

[PRG](#) is the IV-based PRG used for the GGM tree expansion, leaf commitments, and VOLE correlation generation. Since the scheme targets security for up to 2^{80} signature queries, the IV must be at least 160 bits. This requires a block cipher with a block size of at least 160 bits. We therefore instantiate [PRG](#) with Rijndael-192 and Rijndael-256, whose block sizes are equal to their key sizes, for the 160- and 256-bit security levels, respectively. For the 512-bit security level, [PRG](#) uses the internal block cipher of BLAKE2s [SA15].

[SeedExp](#) expands seed_H into the random parity-check matrix $\mathbf{H}$ for reducing the public key size. We instantiate [SeedExp](#) using standard AES-192 and AES-256 in CTR mode for the 160- and 256-bit security levels, respectively. For the 512-bit security level, [SeedExp](#) uses the same internal block cipher of BLAKE2s. In the security analysis, we model [SeedExp](#) as an ideal cipher.

Hash Functions. We instantiate the hash functions using the SM3-based pseudoXOF function provided by NGCC. For the security analysis, we model [Hash](#)₀, [Hash](#)₁, and [Hash](#)₂^j as random oracles, while [Hash](#)₃ is modeled as a pseudorandom function where the last λ input bits are the key. Because NGCC currently restricts implementations to the SM3-based pseudoXOF function, the current version uses this function; we will consider other hash functions in the future.

5.2 Parameter Selection

The parameter of SYDO consists of two parts, RSD parameters and proof system parameters. Let λ be the computational security parameter. In this section, we describe our selection of parameters for SYDO- λ for $\lambda \in \{160, 256, 512\}$.

RSD Parameters. The RSD parameters are (n, k, w) , where n is the code length, k is the code dimension, and w is the Hamming weight of the secret vector. Our selection of RSD parameters relies on the reduction from SD to RSD in Theorem 3. To target RSD instances with λ bits of security, we choose parameters such that the associated SD instance has at least λ' bits of security, where

$$\lambda' := \lambda + \log_2 \left(\frac{\binom{n}{w}}{(n/w)^w} \right) + \frac{1}{\theta \ln 2}. \quad (9)$$

Here $\theta \geq 1$ is the density parameter appearing in Theorem 3. Thus, any attack solving the RSD instance with cost below 2^λ would imply an attack against the associated SD instance with cost below $2^{\lambda'}$.

More precisely, we select the RSD parameters according to the following criteria.

- We consider unstructured binary SD instances with parameters (n, k_{SD}, w) , where k_{SD} is chosen so that the corresponding SD instance lies close to the Gilbert–Varshamov bound, namely we select k_{SD} to be the largest integer such that

$$k_{\text{SD}} := n - \left\lfloor \log_2 \binom{n}{w} \right\rfloor.$$

The estimated security of this SD instance is required to be at least λ' bits, with λ' defined in Equation (9).

Table 5.2. Selection of parameters for RSD

Parameter Sets	Security (bits)		RSD Parameters			Modeling		
	Classical	Quantum	n	$(n - k)$	w	ℓ	$(\eta_j)_{j \in [\ell]}$	$(r_j)_{j \in [\ell]}$
SYDO-160	160	80	16 461	480	59	2	(2, 8)	(1, 3)
SYDO-256	256	128	26 226	768	94	2	(2, 8)	(1, 3)
SYDO-512	512	256	49 941	1456	179	2	(2, 8)	(1, 3)

- We choose RSD instances with parameters (n, k, w) , where n and w are the same as in the SD instance, and k is chosen to be below the largest dimension leading to an RSD instance with density $\leq 1/\theta$. Specifically, we select k to be the largest integer such that

$$k_{\text{SD}} \leq k \leq n - w \log_2 \left(\frac{n}{w} \right) - \log_2 \theta.$$

By the reduction in Theorem 3, for such choices of (n, k, w) , any attack solving the selected RSD instance with cost below 2^λ would imply an attack against the associated SD instance below the security threshold $2^{\lambda'}$ above.

The concrete RSD parameters and modeling parameters are chosen empirically by balancing the size of the modeled witness against the growth of the underlying SD instance. In particular, the search aims to keep $|w| + d\lambda$ small, while avoiding unnecessary increases in n and $n - k$, which would make the scheme slower. Concretely, we use scripts to enumerate candidate parameters and select the final choices according to the following procedure. The resulting RSD parameters and modeling parameters are listed in Table 5.2.

1. Enumerate the candidate values of w . For each w , find the smallest value of n that achieves the target level of security and satisfies $w \mid n$. Set $\tilde{\mu} = n/w$.
2. For each fixed pair $(w, \tilde{\mu})$, enumerate the candidate values of d . For each d , search over admissible choices of $(\eta_j, r_j)_{j=1}^\ell$ with $\sum_j r_j = d$ and $r_j \geq 1$. Choose $(\eta_j)_{j=1}^\ell$ to minimize $\sum_j \eta_j$ subject to $\mu \geq \tilde{\mu}$, where μ is computed by Equation (5).
3. For each feasible candidate obtained from either search, compute the cost

$$w\eta' + \lambda(d - 1),$$

where $\eta' = \sum_j \eta_j$.

4. Select the feasible candidate that minimizes $w\eta' + \lambda(d - 1)$. When several candidates have similar costs, prefer the ones with smaller n and $n - k$.

Memory in SD Attacks. Since recent SD attacks [BJMM12b, Hir16, BM18, Ess22] often require exponential memory to reduce running time, concrete security estimates for the selected parameters must account for memory requirements. Following SDitH [MBB⁺25], for each security level λ , we set a memory bound λ_M for the attacks. The running time of any attack is then estimated under the constraint that the attack uses at most 2^{λ_M} memory. For the 160-, 256-, and 512-bit security levels, we use memory bounds of 2^{160} , 2^{192} , and 2^{256} , respectively. These bounds are chosen based on the following considerations.

- SDitH [MBB⁺25], a NIST round-3 additional-signature candidate [NIS26], uses a memory limit of 2^{160} for both the 207- and 272-bit security levels.
- Dilithium [LDK⁺22] reports memory costs of about 2^{138} and 2^{187} for attacks on the MLWE instances at the 207- and 272-bit security levels, respectively.
- For our 512-bit parameter set, allowing more than 2^{256} memory does not significantly reduce the estimated attack time. In our estimates, increasing the memory bound from 2^{256} to 2^{384} reduces the time complexity by only 5 bits, and the resulting time remains above 2^{512} .

Due to the large memory use of these attacks, the cost of memory accesses might also be taken into account when estimating concrete security. The cost of each memory access can be modeled as constant, logarithmic, cubic-root, or square-root in the memory usage. While SDitH uses a logarithmic cost model for memory accesses, we use the constant memory-access cost model to obtain a more conservative estimate of the attack cost. This matches the cost model used in the refined estimates for Dilithium [LDK⁺22].

Table 5.3. Selection of proof system parameters

Parameter Sets	Security (bits)		VOLEitH Parameters			
	Classical	Quantum	N	τ	w_{grind}	T_{open}
SYDO-160s	160	80	2^{11}	14	8	132
SYDO-160f	160	80	2^8	20	2	138
SYDO-256s	256	128	2^{11}	23	5	225
SYDO-256f	256	128	2^8	32	2	236
SYDO-512s	512	256	2^{11}	46	8	445
SYDO-512f	512	256	2^8	64	2	446

Proof System Parameters. The selected VOLEitH parameters are listed in Table 5.3. For each security level, we consider two proof-system variants. The “s” variant uses larger GGM trees, which reduces the number of repetitions τ and gives smaller signatures. The “f” variant uses smaller GGM trees, which makes the GGM expansion procedures faster, at the cost of a larger signature size.

The VOLEitH proof system has the following parameters: the number of leaves N in the GGM tree, the number of repetitions τ , the degree bound d of the constraint system, the maximum allowed number of seeds for the GGM tree opening T_{open} , and the grinding parameter w_{grind} . To target at least λ bits of security, the parameters must satisfy the following inequality:

$$\tau \cdot \log_2 N - \log_2 d + w_{\text{grind}} \geq \lambda.$$

In our parameter sets, we use $N = 2^8$ for the “f” variants and $N = 2^{11}$ for the “s” variants. The constraint system has degree bound $d = 4$. We set the VOLE hash parameter to $B = 16$. For each choice of N , we set $\tau := \lfloor \lambda / \log_2 N \rfloor$ and choose w_{grind} as the smallest value satisfying the security inequality above. Finally, T_{open} is chosen empirically to keep signatures small without adding too much rejection-sampling cost.

Table 5.4. Key and signature sizes in bytes for the selected parameter sets

Parameter Set	sk	pk	\sigma	\sigma + pk
SYDO-160s	174	80	5428	5508
SYDO-160f	174	80	6724	6804
SYDO-256s	278	128	14444	14572
SYDO-256f	278	128	17604	17732
SYDO-512s	534	246	56672	56918
SYDO-512f	534	246	67716	67962

Key and Signature Sizes. Given the parameters above, the sizes of the secret key, public key, and signature are as follows. the public key is $\text{pk} = (\text{seed}_H, y)$ with $|\text{seed}_H| = \lambda$ and $|y| = n - k$.

$$|\text{pk}|_{\text{bit}} = \lambda + (n - k)$$

the secret key is the public key together with the witness

$$|\text{sk}|_{\text{bit}} = \lambda + (n - k) + w \cdot \sum_{j=1}^{\ell} \eta_j$$

the signature can be calculated as follows

$$\begin{aligned} |\text{sig}|_{\text{bit}} &= \tau \cdot (w \cdot \sum_{j=1}^{\ell} \eta_j + d\lambda + B) && \rightarrow (c_1, \dots, c_{\tau-1}, \tilde{u}, d, \pi_{\text{RSD}}) \\ &+ T_{\text{open}} \cdot \lambda + \tau \cdot 2\lambda + 32 && \rightarrow (\text{open}_I, \text{ctr}) \\ &+ \lambda && \rightarrow \text{chall}_3 \\ &+ \lambda && \rightarrow \text{iv}^{\text{pre}}. \end{aligned}$$

The resulting byte sizes are listed in Table 5.4.

5.3 Failure Probability

There are two places where the algorithms may reject an intermediate value and repeat a local sampling step. During signing, the loop in [SYDO.Sign](#) repeats when the final challenge does not satisfy the grinding condition or when [BAVC.Open](#) returns $\perp$ because the selected opening exceeds T_{open} . During key generation, [RejectionSample](#) repeats when the sampled 32-bit value falls outside the interval used for unbiased sampling of an index in $[0, L)$.

Signing. For a fixed counter value, the last w_{grind} bits of chall_3 are zero with probability $2^{-w_{\text{grind}}}$. The prefix of chall_3 is then decoded into the hidden indices $I = (\Delta_0, \dots, \Delta_{\tau-1})$. The final challenge has length λ , and [DecodeAllChall](#) consumes the prefix $\text{chall}_3[0..\lambda - w_{\text{grind}} - 1]$. For the selected parameters, the number of grinding bits used by the signing algorithm is the value w_{grind} listed in Table 5.3. The BAVC opening succeeds exactly when the number of sibling nodes collected by [BAVC.Open](#) is at most T_{open} . We estimate this probability by sampling random hidden indices according to [DecodeAllChall](#) and counting the number of nodes collected by [BAVC.Open](#). The resulting per-counter success probabilities are summarized in Table 5.5.

Table 5.5. Estimated signing-loop success probabilities for the selected parameter sets

Parameter Set	w_{grind}	$\Pr[\text{BAVC.Open} \neq \perp]$	$\Pr[\text{counter succeeds}]$	Expected counters
SYDO-160s	8	$3.09 \cdot 10^{-2}$	$1.21 \cdot 10^{-4}$	$8.29 \cdot 10^3$
SYDO-160f	2	$1.91 \cdot 10^{-1}$	$4.78 \cdot 10^{-2}$	20.9
SYDO-256s	5	$1.06 \cdot 10^{-1}$	$3.31 \cdot 10^{-3}$	302
SYDO-256f	2	$8.82 \cdot 10^{-1}$	$2.21 \cdot 10^{-1}$	4.54
SYDO-512s	8	$1.47 \cdot 10^{-2}$	$5.74 \cdot 10^{-5}$	$1.74 \cdot 10^4$
SYDO-512f	2	$1.54 \cdot 10^{-1}$	$3.85 \cdot 10^{-2}$	26.0

These rejection events are not signature-verification failures. The signer repeats the counter loop until both conditions are satisfied, and the resulting signature is checked by the deterministic verification algorithm. Thus, T_{open} affects both the expected signing time and the signature size through the opening-size bound, while w_{grind} affects the expected signing time through the grinding condition. If an implementation imposes an external limit of M counter values, the probability of not finding a valid counter is approximately $(1 - p)^M$, where p is the per-counter success probability in Table 5.5. This is an implementation-level availability parameter rather than a change in the security claim.

Key generation. In [RSD.ExpandWitness](#), each block position p_i is sampled by [RejectionSample](#). For all selected RSD parameter sets, $L = n/w = 279$. The rejection threshold is

$$T = \left\lfloor \frac{2^{32}}{279} \right\rfloor 279,$$

so one 32-bit draw is rejected with probability

$$\frac{2^{32} \bmod 279}{2^{32}} = \frac{4}{2^{32}} = 2^{-30}.$$

The probability that at least one draw is rejected during one key generation is therefore at most $w \cdot 2^{-30}$ by a union bound. The resulting estimates are listed in Table 5.6.

Table 5.6. Rejection probabilities in [RejectionSample](#)

Parameter Set	w	Pr[at least one rejected draw]	Expected extra draws
SYDO-160	59	$5.5 \cdot 10^{-8}$	$5.5 \cdot 10^{-8}$
SYDO-256	94	$8.8 \cdot 10^{-8}$	$8.8 \cdot 10^{-8}$
SYDO-512	179	$1.7 \cdot 10^{-7}$	$1.7 \cdot 10^{-7}$

This rejection sampling does not bias the key distribution, because rejected 32-bit values are discarded and the loop continues until a value below T is obtained. It also does not create a key-generation failure in the specified algorithm, since the algorithm has no abort condition in this loop. Its only effect is a negligible increase in expected key-generation time.

Overall, the rejection events above affect performance rather than security. The signing loop repeats until it obtains a challenge and opening that are accepted by the public verification procedure, and the key-generation sampler repeats until it obtains an unbiased block position. Consequently, these events change the expected running time but do not introduce an additional correctness error for accepted signatures or generated keys.

6 Theoretical Security Analysis

In this section, we formally prove the security of SYDO. The proofs in this section follow the framework of FAEST [BBB⁺25a]. We reproduce or adapt the arguments needed for SYDO, with the necessary changes to the OWF, PRG instantiation, and leaf-commitment analysis. For details of the original framework, we refer the reader to FAEST [BBB⁺25a, Section 9].

6.1 Preliminaries for Security Proofs

We begin by introducing the necessary cryptographic preliminaries. Most of this subsection is adapted almost verbatim from FAEST [BBB⁺25a], with the notation and parameters adjusted to the SYDO setting.

Notation. For a function $F: \mathcal{X} \rightarrow \mathcal{Y}$, we write $\text{dom}(F) = \mathcal{X}$ for the domain of F , $\text{cod}(F) = \mathcal{Y}$ for the codomain of F , and $\text{im}(F) = \{F(x) \mid x \in \mathcal{X}\} \subseteq \mathcal{Y}$ for the image of F .

Standard Hardness Assumptions. We recall the standard definitions of indistinguishability, pseudorandom functions, and pseudorandom permutations underlying our security analysis.

Definition 11 (Indistinguishability [BBB⁺25a]). Let $\mathcal{D}_0, \mathcal{D}_1$ be two distributions. Let $\mathcal{A}$ be an adversary in the following experiment $\text{ExpDist}_{\mathcal{A}}^{\mathcal{D}_0, \mathcal{D}_1}$:

1. $x_0 \leftarrow \mathcal{D}_0, x_1 \leftarrow \mathcal{D}_1, b \leftarrow \{0, 1\}$.
2. $b' \leftarrow \mathcal{A}(1^\lambda, x_b)$.
3. Return $\mathbf{1}_{[b'=b]}$.

The distinguishing advantage of $\mathcal{A}$ is

$$\text{AdvDist}_{\mathcal{A}}^{\mathcal{D}_0, \mathcal{D}_1} = 2 \cdot \left| \Pr[\text{ExpDist}_{\mathcal{A}}^{\mathcal{D}_0, \mathcal{D}_1}] - \frac{1}{2} \right| = |\Pr[b' = 1 \mid b = 0] - \Pr[b' = 1 \mid b = 1]|.$$

When $\mathcal{A}$ is provided with Q independent samples from either $\mathcal{D}_0$ or $\mathcal{D}_1$, we denote its advantage in the corresponding game by $\text{AdvDist}_{\mathcal{A}}^{\mathcal{D}_0, \mathcal{D}_1}[Q]$. This definition naturally extends to the indistinguishability of oracles rather than distributions.

An efficiently computable keyed function family $\text{PRF} : \mathcal{K}_\lambda \times \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda$ is a pseudorandom function family if, for a uniformly sampled key $k \leftarrow \mathcal{K}_\lambda$, oracle access to $\text{PRF}(k, \cdot)$ is computationally indistinguishable from oracle access to a uniformly random function $H : \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda$ for every efficient adversary. An efficiently computable keyed function family $\text{PRP} : \mathcal{K}_\lambda \times \mathcal{X}_\lambda \rightarrow \mathcal{X}_\lambda$ is a pseudorandom permutation family if $\text{PRP}(k, \cdot)$ is a permutation over $\mathcal{X}_\lambda$ for every $k \in \mathcal{K}_\lambda$, and oracle access to $\text{PRP}(k, \cdot)$ for a uniformly sampled key $k \leftarrow \mathcal{K}_\lambda$ is computationally indistinguishable from oracle access to a uniformly random permutation over $\mathcal{X}_\lambda$.

Divergence. The block cipher underlying our IV-based tweakable PRG is a pseudorandom *permutation* (PRP), but several proofs below—e.g., the hiding of [VOLE.Commit \[BBB+25a\]](#) and PRF-type security of the [PRG \(Lemma 17\)](#)—need to treat it as a pseudorandom *function* (PRF). Switching via statistical distance costs advantage roughly Q^2/D on a domain of size D (the birthday bound), which would degrade the concrete security we can prove—a proof artefact, not a genuine weakness.

We instead relate the two distributions by their ∞ -divergence (Rényi divergence of order ∞), adopting the divergence-based switching machinery of [FAEST \[BBB+25a\]](#): by [Lemma 12](#), a bound ρ on the worst-case ratio of point probabilities gives $\Pr[X \in E] \leq \rho \cdot \Pr[Y \in E]$ for *every* event E , so each game hop costs only the multiplicative factor ρ (no additive loss) and such factors compose across hybrids ([Remark 14](#)). For our parameters ρ is negligible—e.g., $e^{n(n-1)/N}$ when replacing n samples without replacement by uniform ones ([Corollary 13](#))—letting us replace PRP and GGM-tree PRG outputs by truly random values at negligible multiplicative cost.

Lemma 12 (∞ -divergence [BBB+25a]). *Let X and Y be random variables with values in a set Ω and countable support. Let*

$$\rho := \sup_{\omega \in \Omega} \frac{\Pr[X = \omega]}{\Pr[Y = \omega]},$$

where $0/0 := 0$ and $t/0 := \infty$ for any $t > 0$. For any function $f : \Omega \rightarrow \mathbb{R}_{\geq 0}$ we have

$$\mathbb{E}[f(X)] \leq \rho \cdot \mathbb{E}[f(Y)].$$

In particular, for any event $E \subseteq \Omega$, we have

$$\Pr[X \in E] \leq \rho \cdot \Pr[Y \in E].$$

Note that $D_\infty(X \parallel Y) := \log(\rho)$ is called ∞ -divergence or Rényi divergence of order ∞ .

Proof. By the definition of ρ and the non-negativity of f , we have

$$\begin{aligned} \mathbb{E}[f(X)] &= \sum_{\omega \in \Omega} \Pr[X = \omega] \cdot f(\omega) \\ &= \sum_{\omega \in \Omega} \frac{\Pr[X = \omega]}{\Pr[Y = \omega]} \cdot \Pr[Y = \omega] \cdot f(\omega) \\ &\leq \sum_{\omega \in \Omega} \rho \cdot \Pr[Y = \omega] \cdot f(\omega) \\ &= \rho \cdot \mathbb{E}[f(Y)], \end{aligned}$$

which proves the first claim. The second inequality follows by evaluating the expectation with respect to the indicator predicate $f(\omega) = \mathbf{1}_{[\omega \in E]} \in \{0, 1\}$.

Corollary 13 ([BBB+25a]). *Let $\mathcal{X}$ be a set of size N and let $n \in \mathbb{N}$ with $n < N$. Let $\mathcal{A} : \mathcal{X}^n \rightarrow \{0, 1\}$ be any function. Let R be the uniform distribution on $\mathcal{X}^n$ and let P be the distribution of n uniform samples without replacement from $\mathcal{X}$. Then we have*

$$\Pr[\mathcal{A}(P) = 1] \leq e^{\frac{n(n-1)}{N}} \Pr[\mathcal{A}(R) = 1].$$

Proof ([BBB⁺25a]). By Lemma 12, it suffices to establish that $\frac{\Pr[P=x]}{\Pr[R=x]} \leq e^{\frac{n(n-1)}{N}}$. This holds since $\Pr[P=x] \leq \frac{1}{N \cdot (N-1) \cdots (N-n+1)}$ and $\Pr[R=x] = \frac{1}{N^n}$, yielding

$$\begin{aligned} \frac{1/(N \cdot (N-1) \cdots (N-n+1))}{1/N^n} &= \frac{N^n}{N \cdot (N-1) \cdots (N-n+1)} \\ &= \left(\prod_{i=1}^{n-1} \left(1 - \frac{i}{N}\right) \right)^{-1} \\ &\leq e^{\frac{n(n-1)}{N}}, \end{aligned}$$

where the final inequality follows from the bound $e^{-x} \leq 1 - x/2$ for $x \geq 0$.

Consequently, Corollary 13 can be applied to replace the outputs of PRGs within the GGM tree with uniformly random values. These values can subsequently serve as independent random seeds for successive GGM evaluations.

Remark 14 (Hybrids with ∞ -divergence [BBB⁺25a]). For a sequence of hybrids $H_0, H_1, \dots, H_n$, where $\Pr[H_{i-1} = 1] \leq \rho_i \cdot \Pr[H_i = 1] + \varepsilon_i$, with $\rho_i \geq 1$ and $\varepsilon_i \geq 0$, we find

$$\begin{aligned} \Pr[H_0 = 1] &\leq \left(\prod_{j=1}^n \rho_j \right) \cdot \Pr[H_n = 1] + \sum_{i=1}^n \left(\prod_{j=1}^{i-1} \rho_j \right) \cdot \varepsilon_i \\ &\leq \left(\prod_{j=1}^n \rho_j \right) \cdot \left(\Pr[H_n = 1] + \sum_{i=1}^n \varepsilon_i \right). \end{aligned}$$

This is the analogue of the usual hybrid argument.

IV-based tweakable PRGs. As in FAEST, our security analysis relies on pseudorandomness and a non-standard injectivity property of an IV-based tweakable PRG. FAEST instantiates the IV-based tweakable PRG with AES, which is a natural choice for efficiency and security. However, AES has a block size of 128 bits, which is below the block size required for security against 2^{80} signing queries. Therefore, we instantiate the IV-based tweakable PRG using Rijndael with λ -bit block and adapt the security definitions from FAEST to our setting as follows.

Definition 15 (IV-Based Tweakable PRG [BBB⁺25a]). An IV-based tweakable PRG is an efficient map $\text{PRG}: \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{\alpha \cdot \lambda}$, where $\alpha \in \mathbb{N}$ and the output length is $m = \alpha \cdot \lambda$.

The pseudorandomness of the IV-based tweakable PRG is characterized by the following PRP-type and PRF-type security.

Definition 16 (PRP-Type and PRF-Type Security of PRG [BBB⁺25a]). Let $\alpha \in \mathbb{N}$ and let $\text{PRG}: \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{\alpha \cdot \lambda}$ be a deterministic polynomial-time algorithm with output length $m = \alpha \cdot \lambda$. Let $\mathcal{A}$ be a T -query adversary in the following PRP-type game:

1. Sample $k \leftarrow \{0, 1\}^\lambda$ and $\text{iv} \leftarrow \{0, 1\}^\lambda$.
2. Sample $b \leftarrow \{0, 1\}$ and set $\text{RP} := \{\}$ as an empty associative array and $B := \emptyset$.
3. Run $b' \leftarrow \mathcal{A}^{\text{PRG}^b}(\text{iv})$, where $\mathcal{A}$ is limited to T oracle queries and the oracle PRG^b is defined as follows:
 - $\text{PRG}^0(k, \text{iv}, \text{twk})$: output $\text{PRG}(k, \text{iv}, \text{twk})$.
 - $\text{PRG}^1(k, \text{iv}, \text{twk})$: if $\text{RP}[\text{twk}] = \perp$, then for $i = 1, \dots, \alpha$:
 - sample $y_i \leftarrow \{0, 1\}^\lambda \setminus B$.
 - set $B := B \cup \{y_i\}$.
4. Output 1 if $b = b'$, else output 0.

Let p be the probability that $\mathcal{A}$ wins the game. The advantage of a T -query adversary against the PRP-type security of the IV-based tweakable PRG PRG is defined as

$$\text{AdvPRP}_{\mathcal{A}}^{\text{PRG}}[T] = |2p - 1|.$$

The PRF-type game is identical to the above game, except that the ideal oracle is implemented by a lazy sampled random function RF instead of the random permutation RP . We write $\text{AdvPRG}_{\mathcal{A}}^{\text{PRG}}[T] = |2p - 1|$ for the corresponding PRF-type advantage.

It is not hard to see that the IV-based tweakable PRG satisfies PRP-type security of Definition 16 by a straightforward reduction to PRP security of Rijndael. For PRF-type security we reuse the bound and argument of FAEST [BBB⁺25a, Lemma 9.12], which rests on ∞ -divergence (Lemma 12 and Corollary 13); we restate it below with the block size 128 replaced by λ .

Lemma 17 ([BBB⁺25a]). *Let $\beta \in \mathbb{N}$ and let $\text{PRG}: \{0, 1\}^\lambda \times \{0, 1\}^\lambda \times \{0, 1\}^{32} \rightarrow \{0, 1\}^{\beta \cdot \lambda}$ be a deterministic polynomial-time algorithm with $m = \beta \cdot \lambda$. Let $\mathcal{A}$ be a T -query adversary for the PRF-type security of PRG (Definition 16). Let G_{PRP} denote the output of $\mathcal{A}$ in the real PRP-type game and G_{RP} (resp. G_{RF}) be the output in the ideal PRP-type game (resp. PRF-type). Then it holds that*

$$\Pr[\text{G}_{\text{PRP}} = 1] \leq e^{\beta^2 T^2 / 2^\lambda} \cdot \Pr[\text{G}_{\text{RF}} = 1] + \text{AdvPRP}_{\mathcal{A}}^{\text{PRG}}[T].$$

Proof. Let G_{PRP} denote the ideal PRP-type game. Then

$$\Pr[\text{G}_{\text{PRP}} = 1] \leq \Pr[\text{G}_{\text{RP}} = 1] + \text{AdvPRP}_{\mathcal{A}}^{\text{PRG}}[T]$$

follows by definition. By an application of Corollary 13, switching the sampling of blocks without replacement to sampling of blocks with replacement (i.e., truly at random) and therefore to G_{RF} , leads to

$$\Pr[\text{G}_{\text{RP}} = 1] \leq e^{\beta^2 T^2 / 2^\lambda} \cdot \Pr[\text{G}_{\text{RF}} = 1].$$

We adopt the almost-injectivity notion of FAEST [BBB⁺25a, Definition 9.13] for our IV-based tweakable PRG. Intuitively, this property asserts the computational infeasibility of finding an image that exhibits a collision (i.e., possesses two or more distinct preimages), thereby serving as a strengthened variant of standard collision resistance.

Definition 18 (Almost-Injective PRG [BBB⁺25a]). *Let $\text{PRG} : \mathcal{K}_\lambda \times \mathbb{Z}_\lambda \times \mathcal{T}_\lambda \rightarrow \mathcal{Y}_\lambda$ be an IV-based tweakable pseudorandom generator, where $\mathcal{K}_\lambda = \{0, 1\}^\lambda$ is the key space, $\mathbb{Z}_\lambda$ is the IV space, and $\mathcal{T}_\lambda$ is the tweak space. For a positive integer $Q_{\text{out}} = Q_{\text{out}}(\lambda)$, consider the following experiment $\text{Exp}_{\text{PRG}}^{\text{inj}}(\mathcal{A}, Q_{\text{out}})$:*

1. *The adversary $\mathcal{A}$ outputs an IV $\text{iv} \in \mathbb{Z}_\lambda$ and Q_{out} pairs $(\text{twk}_j, y_j)_{j \in [Q_{\text{out}}]}$, where $\text{twk}_j \in \mathcal{T}_\lambda$ and $y_j \in \mathcal{Y}_\lambda$.*
2. *The experiment outputs 1 if there exist $j \in [Q_{\text{out}}]$ and two distinct keys $k, k' \in \mathcal{K}_\lambda$ such that*

$$\text{PRG}(k, \text{iv}, \text{twk}_j) = y_j = \text{PRG}(k', \text{iv}, \text{twk}_j).$$

Otherwise, it outputs 0.

The advantage of $\mathcal{A}$ in this experiment is defined as

$$\text{Adv}_{\text{PRG}, \mathcal{A}}^{\text{inj}}(Q_{\text{out}}) := \Pr[\text{Exp}_{\text{PRG}}^{\text{inj}}(\mathcal{A}, Q_{\text{out}}) = 1].$$

We say that PRG is almost injective if, for every probabilistic polynomial-time adversary $\mathcal{A}$ and every polynomially bounded $Q_{\text{out}} = Q_{\text{out}}(\lambda)$, the advantage $\text{Adv}_{\text{PRG}, \mathcal{A}}^{\text{inj}}(Q_{\text{out}})$ is negligible in λ .

6.2 Properties of the VOLE Commitment

We now state the three security properties of VOLE.Commit on which our later proofs rely: extractable binding, multi-hiding, and unpredictability.

Relative to FAEST [BBB⁺25a], our VOLE.Commit differs only in that the underlying IV-based tweakable PRG is instantiated with Rijndael (block size λ) rather than AES (block size 128); the structure of the commitment scheme, and hence the arguments establishing these three properties, are essentially unchanged. We therefore state the conclusions directly and refer the reader to [BBB⁺25a] for the full proofs, adapting only the concrete advantage bounds to our block size.

6.2.1 Binding

Binding is captured by a straight-line extractor that watches the adversary's random-oracle queries and recovers the committed message except with small probability.

Lemma 19 (Extractable Binding of `VOLE.Commit` [BBB⁺25a]). *The extractable binding game—played between (`VOLE.Commit`, `VOLE.Reconstruct`), a stateful adversary $\mathcal{A}$, and a straight-line extractor Ext —is defined as follows.*

1. **Commit-Phase:** Let $\mathcal{C} = \emptyset$ be an empty associative array.
 - (a) $\mathbf{C} = (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}}) \leftarrow \mathcal{A}^{\text{Hash}_0, \text{Hash}_1, \text{Hash}_4}(\text{commit})$.
 - (b) $\mathbf{D} = (\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)} = \text{Ext}^{\text{Hash}_4}(\mathcal{L}_0, \mathcal{L}_1, \text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$, where for $b \in \{0, 1\}$, $\mathcal{L}_b$ is a set $\{(x_i, \text{Hash}_b(x_i))\}$ of query-response pairs from queries $\mathcal{A}$ made to the random oracle Hash_b .
 - (c) Set $\mathcal{C}[\mathbf{C}] = \mathbf{D}$. Note that $\mathbf{D} = \perp$ is allowed.
 - (d) $\mathcal{A}$ can choose to go to Step 1(a) or it outputs $\mathbf{C}$ in $\mathcal{C}$ and the game continues below.
2. **Decommitment-Phase:** Parse $\mathbf{C} = (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$.
3. $(\text{chall}_3, \text{decom}_I) \leftarrow \mathcal{A}^{\text{Hash}_0, \text{Hash}_1, \text{Hash}_4}(\text{open})$ where $I = (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{VOLE.DecodeAllChall}(\text{chall}_3)$.
4. If $I = \perp$ then output 0 (failure).
5. Let $\text{iv} = \text{Hash}_4(\text{iv}^{\text{pre}})$.
6. For $i \in [0, \dots, \tau)$: $(\delta_{i,0}, \dots, \delta_{i,k_i}) := \text{BitDec}(\Delta_i, k_i)$.
7. $(\overline{\text{com}}, \mathbf{Q}) := \text{VOLE.Reconstruct}^{\text{Hash}_0, \text{Hash}_1}(\text{chall}_3, \text{decom}_I, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}, \hat{\ell}; \text{param})$.
8. Parse $[\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1} \mathbf{0}_{\hat{\ell},w}] = \mathbf{Q}$, where $\mathbf{Q}_i \in \mathbb{F}_2^{\hat{\ell} \times k_i}$.
9. Output 0 (failure) if:
 - (a) $\overline{\text{com}} = \perp$ or $\text{com} \neq \overline{\text{com}}$ (i.e., verifier rejects), or
 - (b) $\mathbf{Q}_i = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot (\mathbf{u}_i^* \oplus \mathbf{c}_i), \dots, \delta_{i,k_i} \cdot (\mathbf{u}_i^* \oplus \mathbf{c}_i)]$, for all $i \in [0, \tau)$ (i.e., extraction correct).
10. Output 1 (success) otherwise.

Assume $\mathcal{A}$ makes at most Q_0 queries to `Hash`₀ and Q_1 queries to `Hash`₁, and that it checks whether `VOLE.Reconstruct` yields $\perp$ for the purported opening decom_I and outputs $\perp$ if that happens. Then there is a deterministic, computationally unbounded straight-line extractor Ext which is allowed to program `Hash`₄ such that the advantage of $\mathcal{A}$ in the above game is bounded by

$$\frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvInj}_B^{\text{PRG}}[Q_4, L],$$

with $\mathcal{B}$ running in time comparable to that of $\mathcal{A}$. Here PRG is the IV-based tweakable PRG defined in Definition 15, called with output length 2λ and with its tweak domain restricted to $[L - 1, L + \tau - 1]$.

6.2.2 Hiding

We next show that `VOLE.Commit` is hiding.

Lemma 20 (`VOLE.Commit` is multi-hiding [BBB⁺25a]). *Modeling `Hash`₀ and `Hash`₁ as random oracles, the multi-hiding experiment for `VOLE.Commit`^{`Hash`₀, `Hash`₁} against a Q -query adversary $\mathcal{A}$ (for arbitrary $\hat{\ell}$ and param) is defined as follows.*

1. Sample $b^* \leftarrow \{0, 1\}$.
2. For each $j \in [1, Q]$:
 - $r^j \leftarrow \{0, 1\}^\lambda$, $\text{iv}^j \leftarrow \{0, 1\}^\lambda$.
 - $(\text{com}^j, \text{decom}^j, (\mathbf{c}_i^j)_{i \in [1, \tau)}, \mathbf{u}^j, \mathbf{V}^j) := \text{VOLE.Commit}(r^j, \text{iv}^j, \hat{\ell}; \text{param})$.
 - Sample $\text{chall}_3^j \leftarrow \{0, 1\}^{\lambda - w_{\text{grind}}} \times 0^{w_{\text{grind}}}$ uniformly, conditioned on `BAVC.Open` not aborting for $I^j = (\Delta_0^j, \dots, \Delta_{\tau-1}^j) := \text{VOLE.DecodeAllChall}(\text{chall}_3^j; \text{param})$.
 - $\text{decom}_{I^j} := \text{BAVC.Open}(\text{decom}^j, I^j)$.
 - Let $\mathbf{u}_s^j \leftarrow \mathbb{F}_2^{\hat{\ell}}$ and define

$$(\bar{\mathbf{u}}^j, \bar{\mathbf{c}}_1^j, \dots, \bar{\mathbf{c}}_{\tau-1}^j) = \begin{cases} (\mathbf{u}^j, \mathbf{c}_1^j, \dots, \mathbf{c}_{\tau-1}^j) & \text{if } b^* = 0, \\ \text{uniformly random in } (\mathbb{F}_2^{\hat{\ell}})^\tau & \text{if } b^* = 1. \end{cases}$$

3. $b \leftarrow \mathcal{A}^{\text{Hash}_0, \text{Hash}_1} \left(\left(\bar{\mathbf{u}}^j, (\bar{\mathbf{c}}_i^j)_{i \in [1, \tau]}, \text{decom}_{I^j}, \text{chall}_3^j, \text{com}^j \right)_{j \in [1, Q]} \right)$.
4. Output 1 (success) if $b = b^*$, and 0 (failure) otherwise.

Writing $\text{PRG}_{\text{len}}(s) = \text{PRG}(s; \text{len})$ and taking $\beta = 1$, and abbreviating the hiding game under $b^* = i$ as $\mathbf{G}_i^*$, we obtain

$$\Pr[\mathbf{G}_0^* = 1] \leq e^{Q\tau(4\lceil \log(L) \rceil \beta^2 + 16\beta^2 N_0^2 + (\lceil \hat{\ell}/\lambda \rceil)^2)/2^\lambda} \cdot \left(\Pr[\mathbf{G}_1^* = 1] + Q\tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}_{\mathcal{B}_2}^{\text{BAVC.LeafCommit}}[N_0] + \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\hat{\ell}}}[1] \right) \right),$$

Here $\text{AdvPRP}_{\mathcal{B}_2}^{\text{BAVC.LeafCommit}}$ refers to the corresponding PRG evaluation inside **BAVC.LeafCommit**, namely $\text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{2\lambda}}$. The bound carries over to quantum adversaries unchanged, with every advantage term then measured against quantum adversaries.

Because **BAVC.Open** rejects challenges based on the challenge alone (independently of the underlying commitment), drawing $(\text{chall}_3^j)_{j \in [1, Q]}$ before the commitment phase leaves the adversary's view untouched. All decommitment positions are thus fixed at commitment time, so only selective security is needed.

6.2.3 Unpredictability

The unpredictability notion is mildly non-standard: $\mathcal{A}$ produces a bit that we overwrite with 0 whenever its guess hit the mark. This matches how the property is invoked later on, where $\mathcal{A}$ is itself a hybrid game outputting 1 precisely when the EUF-CMA adversary wins; overwriting such a 1 with 0 then amounts to ruling that the EUF-CMA adversary loses once it has predicted a commitment.

Lemma 21 (Vole.Commit is unpredictable [BBB⁺25a]). For $\hat{\ell} \in \mathbb{N}$ and admissible param, let $\mathcal{A}$ be an adversary in the experiment $\text{ExpUnpre}_{b, \mathcal{A}}^{\text{Vole.Commit}, \hat{\ell}, \text{param}}(\lambda)$, defined by:

1. $(\mathcal{L}_{\text{com}}, \text{state}) \leftarrow \mathcal{A}^{\text{Hash}_0, \text{Hash}_1}(1^\lambda)$. Here $\mathcal{L}_{\text{com}}$ is a list of commitment guesses.
2. $r \leftarrow \{0, 1\}^\lambda$, $\text{iv} \leftarrow \{0, 1\}^\lambda$.
3. $C = (\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{Vole.Commit}(r, \text{iv}, \hat{\ell}; \text{param})$.
4. Let $b' \leftarrow \mathcal{A}^{\text{Hash}_0, \text{Hash}_1}(\text{state}, C)$.
5. If $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}) \in \mathcal{L}_{\text{com}}$ and $b = 1$, output 0. Otherwise, output b' .

Abbreviating $\text{ExpUnpre}_{b, \mathcal{A}}^{\text{Vole.Commit}, \hat{\ell}, \text{param}}$ as $\mathbf{G}_b$ ($b \in \{0, 1\}$) and setting $\beta = 1$, every $\mathcal{A}$ that issues at most Q_1 queries to **Hash**₁ and lists at most Q_{com} guesses satisfies

$$\Pr[\mathbf{G}_0 = 1] \leq e^{(4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^\lambda} \cdot \left(\Pr[\mathbf{G}_1 = 1] + \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}_{\mathcal{B}}^{\text{BAVC.LeafCommit}}[1] + (2Q_1 + Q_{\text{com}}) \cdot 2^{-2\lambda} \right),$$

where $\text{AdvPRP}_{\mathcal{B}}^{\text{BAVC.LeafCommit}}$ again stands for $\text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}$.

Unpredictability already follows from hiding (by substituting the $\mathbf{c}_i$ with true randomness); Lemma 21 sharpens this to a tighter bound.

6.3 EUF-KO

With the prerequisites in place, we turn to the security of the full signature scheme. Following the FAEST framework, we begin with EUF-KO security—a key-only setting in which the forger sees nothing but the public key—and then strengthen the result to EUF-CMA security, where a signing oracle is added. Most of this subsection is adapted almost verbatim from FAEST [BBB⁺25a, Section 9.5], with the necessary details adjusted to the SYDO setting.

A note on the proof strategy and assumptions. The EUF-KO proof presented below uses a lossy-key argument following the approach of Katz and Wang [KW03], as in FAEST [BBB⁺25a], which in our setting requires the indistinguishability of the RSD instance distribution from the uniform vector distribution. More precisely, the proof uses the assumption that $(\mathbf{H}, \mathbf{H}\mathbf{x})$, for uniform $\mathbf{H}$ and regular $\mathbf{x}$, is computationally indistinguishable from $(\mathbf{H}, \mathbf{u})$, where $\mathbf{u}$ is uniform. It is also possible to avoid this assumption by following the FAEST v1 [BBdSG⁺23a] approach based on the leaf commitment from collision-resistant hash functions. Such a proof can be obtained directly from FAEST v1 [BBdSG⁺23a], and the formal security proof for this variant will be added to a future version of the specification.

We begin by recalling the definition of EUF-CMA and EUF-KO security of a signature scheme.

Definition 22 (EUF-CMA and EUF-KO Security [BBB⁺25a]). A signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is existentially unforgeable against chosen message attacks (EUF-CMA) (resp. against key-only attacks (EUF-KO)) in the random oracle model if any PPT adversary has at most negligible advantage in winning the EUF-CMA (resp. EUF-KO) security game (Figure 6.1). We denote by $\text{AdvEUF-CMA}_{\mathcal{A}}^{\text{SIG}} = \Pr[\text{ExpEUF-CMA}_{\mathcal{A}}^{\text{SIG}} = 1]$ (resp. $\text{AdvEUF-KO}_{\mathcal{A}}^{\text{SIG}} = \Pr[\text{ExpEUF-KO}_{\mathcal{A}}^{\text{SIG}} = 1]$) the advantage of an adversary $\mathcal{A}$.

$\text{ExpEUF-CMA}_{\mathcal{A}}^{\text{SIG}} / \text{ExpEUF-KO}_{\mathcal{A}}^{\text{SIG}}$	$\text{OSign}(\text{msg})$
1: $\mathcal{M} := \emptyset$	1: $\mathcal{M} := \mathcal{M} \cup \{\text{msg}\}$
2: $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}^{\text{H}}(\text{param})$	2: return $\sigma \leftarrow \text{Sign}^{\text{H}}(\text{sk}, \text{msg})$
3: $(\text{msg}, \sigma) \leftarrow \mathcal{A}^{\text{H}, \text{OSign}}(\text{param}, \text{pk})$	
4: return $\text{Verify}^{\text{H}}(\text{msg}, \text{pk}, \sigma; \text{param}) \wedge \text{msg} \notin \mathcal{M}$	

Fig. 6.1. Random-oracle-model experiments for EUF-CMA and EUF-KO security; H stands for whichever random oracles are used. Access to OSign is granted only under EUF-CMA.

For the proof of EUF-KO security, we use the following technical fact from FAEST [BBB⁺25a, Lemma 9.23].

Lemma 23 ([BBB⁺25a]). Let $F: \mathcal{K} \rightarrow \mathcal{Y}$ be a PRG and let $\mathcal{A}$ be any (possibly unbounded) distinguisher, i.e., a predicate $\mathcal{A}: \mathcal{Y} \rightarrow \{0, 1\}$. Then

$$\Pr_{y \leftarrow \text{im}(F)} [\mathcal{A}(y) = 1] \leq \frac{|\text{dom}(F)|}{|\text{im}(F)|} \cdot \Pr_{x \leftarrow \text{dom}(F)} [\mathcal{A}(F(x)) = 1].$$

Informally, Lemma 23 lets us trade the uniform distribution on the image of F for F 's output distribution, bounding the cost of that switch.

Proof ([BBB⁺25a]). Every predicate $\mathcal{A}$ singles out the set $E = \{y \in \mathcal{Y} \mid \mathcal{A}(y) = 1\} \subseteq \mathcal{Y}$, so it suffices to invoke ∞ -divergence (Lemma 12) between two laws on $\mathcal{Y}$: I , uniform on $\text{im}(F)$, and J , the law of $F(x)$ for uniform $x \leftarrow \mathcal{K}$. Whenever $\Pr[I = y] = 0$ we also have $\Pr[J = y] = 0$, so $\Pr[I = y] / \Pr[J = y] \leq \infty$; and for $y \in \text{supp}(I)$, $\Pr[I = y] = \frac{1}{|\text{im}(F)|}$ while $\Pr[J = y] \geq \frac{1}{|\text{dom}(F)|}$. Hence,

$$\frac{\Pr[I = y]}{\Pr[J = y]} \leq \frac{|\text{dom}(F)|}{|\text{im}(F)|},$$

and Lemma 12 delivers the claim.

The following lemma bounds the distinguishing advantage incurred when replacing an honestly generated public key of SYDO by one with a uniformly random vector.

Lemma 24. Let $\mathcal{R}_{\text{RSD}} = \left\{ (\mathbf{x}_1, \dots, \mathbf{x}_w) \in (\mathbb{F}_2^{n/w})^w : \text{wt}(\mathbf{x}_i) = 1 \text{ for all } i \in [w] \right\}$. For any distinguisher $\mathcal{B}$ that receives a pair $(\mathbf{H}, \mathbf{y}) \in \mathbb{F}_2^{(n-k) \times n} \times \mathbb{F}_2^{n-k}$ and outputs a bit, define

$$\begin{aligned} \text{AdvDist}_{\mathcal{B}}^{\text{RSD}} = & \left| \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{H}\mathbf{x}) = 1 : \mathbf{H} \leftarrow_r \mathbb{F}_2^{(n-k) \times n}, \mathbf{x} \leftarrow_r \mathcal{R}_{\text{RSD}} \right] \right. \\ & \left. - \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{y}) = 1 : \mathbf{H} \leftarrow_r \mathbb{F}_2^{(n-k) \times n}, \mathbf{y} \leftarrow_r \mathbb{F}_2^{n-k} \right] \right|. \end{aligned}$$

For every algorithm $\mathcal{A}$ that receives a public key $(\mathbf{H}, \mathbf{y})$, let G_0 be the experiment that samples $\mathbf{H} \leftarrow_r \mathbb{F}_2^{(n-k) \times n}$, $\mathbf{x} \leftarrow_r \mathcal{R}_{\text{RSD}}$, sets $\mathbf{y} = \mathbf{H}\mathbf{x}$, and runs $\mathcal{A}$ on $(\mathbf{H}, \mathbf{y})$. Let G_1 be the experiment that samples $\mathbf{H} \leftarrow_r \mathbb{F}_2^{(n-k) \times n}$ and $\mathbf{y} \leftarrow_r \mathbb{F}_2^{n-k}$, and runs $\mathcal{A}$ on $(\mathbf{H}, \mathbf{y})$. Then there exists a distinguisher $\mathcal{B}$ with running time roughly that of $\mathcal{A}$ such that

$$|\Pr[G_1 = 1] - \Pr[G_0 = 1]| \leq \text{AdvDist}_{\mathcal{B}}^{\text{RSD}}.$$

For a fixed matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$, define the RSD one-way function

$$\text{OWF} : \mathcal{R}_{\text{RSD}} \rightarrow \mathbb{F}_2^{n-k}, \quad \mathbf{x} \mapsto \mathbf{H}\mathbf{x}.$$

Assuming that $\text{AdvDist}_{\mathcal{B}}^{\text{RSD}}$ is negligible, we may treat this OWF as a PRG in the EUF-KO proof and apply Lemma 23 in the subsequent game hop.

We are now ready to prove EUF-KO security of SYDO. The proof follows the structure of FAEST [BBB⁺25a, Theorem 9.24]; we recount it here for completeness.

Theorem 25 (SYDO is EUF-KO). *Let Hash_0 , Hash_1 , Hash_2^0 , Hash_2^1 , Hash_2^2 , Hash_2^3 , Hash_3 , and Hash_4 be modeled as random oracles, and let param be the parameters of the SYDO signature scheme. Let d denote the constraint degree in $\text{param}_{\text{RSD}}$. Let $\mathcal{A}$ be an adversary which, for simplicity, runs SYDO.Verify on its purported forgery and outputs $\perp$ if verification fails. Let $Q_0, Q_1, Q_{2,i}, Q_3, Q_4$, and Q_5 denote upper bounds on the number of queries made by $\mathcal{A}$ to Hash_0 , Hash_1 , Hash_2^i , Hash_3 , and Hash_4 , respectively, and let $Q_2 = Q_{2,1} + Q_{2,2} + Q_{2,3}$. Suppose that $\hat{\ell} \leq 2^{13}$, $\ell' \leq 2^{13}$, and $2 \log_2(\tau) \leq \min\{B, 50\}$. Then*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}} \leq \frac{1}{1 - \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}} \cdot \left(\text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}} + \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}} \right),$$

where OWF is the OWF used in SYDO, and

$$\begin{aligned} \text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}} &\leq (Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2 / 2^{2\lambda} \\ &\quad + \frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvLeafColl}^{\text{BAVC.LeafCommit}} \\ &\quad + (Q_{2,1} + Q_{2,2} + Q_{2,3} + 9) \cdot (d - 1) / 2^\lambda, \end{aligned}$$

with $\text{AdvLeafColl}_{\mathcal{B}'}^{\text{BAVC.LeafCommit}} \leq (Q_{\text{PRG}} + 1)^2 \cdot 2^{-2\lambda}$.

For the OWF used in SYDO, the quotient $\frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}$ is the RSD density

$$\rho_{\text{RSD}} = \frac{(n/w)^w}{2^{n-k}}.$$

Under the parameter condition in Theorem 3, $\rho_{\text{RSD}} \leq 1/\theta$. Thus the loss factor is bounded by

$$\frac{1}{1 - \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}} \leq \frac{1}{1 - 1/\theta}.$$

We first sketch the argument at a high level. The signature is the Fiat-Shamir transform of a VOLE-in-the-head proof for the RSD relation associated with the public key $\text{pk} = (\mathbf{H}, \mathbf{y})$. The witness determines a regular vector $\mathbf{x}$ satisfying $\mathbf{H}\mathbf{x} = \mathbf{y}$ for a random $\mathbf{H}$.

The EUF-KO reduction follows the approach of Katz and Wang [KW03]: after replacing $\mathbf{y} = \mathbf{H}\mathbf{x}$ by a uniformly random vector, the event that the vector has a regular preimage has probability at most the RSD density $\rho_{\text{RSD}} \leq 1/\theta$. Conditioned on the complementary event, the public key is outside the RSD relation, so any accepting forgery violates soundness of the proof system.

We then define the interactive proof underlying SYDO and recast EUF-KO security as the state-restoration security of that proof. Extracting the BAVC commitments turns an interactive-proof adversary into an adversary against an interactive oracle proof that models SYDO with an idealized VOLE.Commit. For this IOP, round-by-round soundness entails state-restoration soundness, and RBR soundness follows from the VOLE consistency check and the polynomial checks for the RSD constraints through ZKHash.

Finally, EUF-KO security is transformed to EUF-CMA security by simulating signatures without the secret key. The simulator uses honest-verifier zero knowledge and programs the Fiat–Shamir oracles Hash_2^1 , Hash_2^2 , and Hash_2^3 ; unpredictability and hiding of the BAVC commitments make the simulated transcripts indistinguishable from real signatures.

Proof (Proof of Theorem 25, adapted from [BBB⁺25a]). The argument is a chain of game hops. We handle the SYDO case first and then note what changes for the other variant. Write G_i for the output of game i .

Game G_0 : This is the genuine EUF-KO game. Denoting the generated challenge secret and public key by sk, pk , we have

$$\Pr[G_0 = 1] = \text{AdvEUFKO}_A^{\text{SYDO}}.$$

Game G_1 : Here the honestly generated pair $(\mathbf{H}, \mathbf{y})$, with $\mathbf{y} = \mathbf{H}\mathbf{x}$ for $\mathbf{x} \leftarrow_r \mathcal{R}_{\text{RSD}}$, is replaced by $(\mathbf{H}, \mathbf{u})$ for $\mathbf{u} \leftarrow_r \mathbb{F}_2^{n-k}$. Lemma 24 gives a distinguisher $\mathcal{B}_1$ running in about the time of the EUF-KO game such that

$$|\Pr[G_1 = 1] - \Pr[G_0 = 1]| \leq \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}}.$$

Splitting Game G_1 : We split $\Pr[G_1 = 1]$ across two events: I , meaning $\mathbf{u} \in \text{im}(\text{OWF})$ so that $(\mathbf{H}, \mathbf{u})$ has a regular preimage, and $\neg I$, in which no $\mathbf{x} \in \mathcal{R}_{\text{RSD}}$ satisfies $\mathbf{H}\mathbf{x} = \mathbf{u}$. By Corollary 35, $\mathcal{A}$'s chance of winning G_1 under $\neg I$ is bounded by the soundness advantage AdvSnd of that corollary—there the EUF-KO adversary breaks soundness of the proof system. Thus,

$$\Pr[G_1 = 1 \mid \neg I] \leq \text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}}.$$

Lemma 23, applied for each fixed $\mathbf{H}$, gives

$$\Pr[G_1 = 1 \mid I, \mathbf{H}] \leq \frac{|\text{dom}(\text{OWF})|}{|\text{im}(\text{OWF})|} \cdot \Pr[G_0 = 1 \mid \mathbf{H}],$$

the left-hand side being G_1 under a uniform image of OWF and the right-hand side the law induced by the RSD instance distribution. Therefore,

$$\begin{aligned} \Pr[G_1 = 1] &= \mathbb{E}_{\mathbf{H}} [\Pr[I \mid \mathbf{H}] \cdot \Pr[G_1 = 1 \mid I, \mathbf{H}] + \Pr[\neg I \mid \mathbf{H}] \cdot \Pr[G_1 = 1 \mid \neg I, \mathbf{H}]] \\ &\leq \mathbb{E}_{\mathbf{H}} \left[\frac{|\text{im}(\text{OWF})|}{|\text{cod}(\text{OWF})|} \cdot \frac{|\text{dom}(\text{OWF})|}{|\text{im}(\text{OWF})|} \Pr[G_0 = 1 \mid \mathbf{H}] \right] + \text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}} \\ &\leq \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|} \Pr[G_0 = 1] + \text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}}, \end{aligned}$$

where we used $\Pr[I \mid \mathbf{H}] = |\text{im}(\text{OWF})|/|\text{cod}(\text{OWF})|$ and $\Pr[\neg I \mid \mathbf{H}] \leq 1$. Folding in the first game hop,

$$\Pr[G_0 = 1] \leq \Pr[G_1 = 1] + \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}} \leq \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|} \Pr[G_0 = 1] + \text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}} + \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}}.$$

Isolating $\Pr[G_0 = 1]$, we obtain

$$\Pr[G_0 = 1] \leq \frac{1}{1 - \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}} \cdot \left(\text{AdvSnd}_{\mathcal{B}_2}^{\text{SYDO}} + \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}} \right), \quad (10)$$

which is the claimed bound once the advantage from Corollary 35 is substituted in.

6.3.1 Interactive Proof

We now recall the definition of an interactive proof (IP) used in [BBB⁺25a].

Definition 26 (Interactive Proof System). (Extracted from [CY24], Section 13.1.) An interactive proof (IP) for relation $\mathcal{R}$ in the random oracle model is a tuple of algorithms $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$, where the prover and verifier interact over $k \in \mathbb{N}$ rounds as follows, with access to one or more random oracles.

- The IP prover $\mathcal{P}_{\text{IP}}$ receives as input the instance $\curvearrowright$ and a witness $\curvearrowright$ such that $(\curvearrowright, \curvearrowright) \in \mathcal{R}$, while the IP verifier $\mathcal{V}_{\text{IP}}$ receives only the instance $\curvearrowright$.
- In each round $i \in [k]$, the prover $\mathcal{P}_{\text{IP}}$, with access to one or more random oracles, sends a message α_i , after which the verifier $\mathcal{V}_{\text{IP}}$ responds with a (possibly empty) message ρ_i .
- Following the interaction, $\mathcal{V}_{\text{IP}}$ outputs a bit, where 1 denotes acceptance and 0 denotes rejection, computed based on the instance $\curvearrowright$, the random oracle queries, the prover messages $(\alpha_i)_{i \in [k]}$, and the verifier messages $(\rho_i)_{i \in [k]}$.

Definition 27. ([CY24], Definition 13.2.1.) The state-restoration experiment for $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$, in the random oracle model, with salt sizes $s_i \in \mathbb{N}$, randomness functions $\text{rnd} = (\text{rnd}_i)_{i \in [k]} \leftarrow \mathcal{U}((r_i)_{i \in [k]})$, and IP state-restoration prover $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ is defined in Figure 6.2. We say that $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ is t -move if it exits the while loop after at most t iterations.

Definition 28. ([CY24], Definition 13.2.2.) The IP $\text{IP} = (\mathcal{P}_{\text{IP}}, \mathcal{V}_{\text{IP}})$ has state-restoration soundness error $\epsilon_{\text{IP}}^{\text{sr}}$ if for every choice of salt sizes $s_i \in \mathbb{N}$, move budget $k \in \mathbb{N}$, and k -round malicious IP state-restoration prover $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$,

$$\Pr \left[\begin{array}{c} \curvearrowright \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathcal{V}_{\text{IP}}(\curvearrowright, (\alpha_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) = 1 \end{array} \middle| \begin{array}{c} \text{rnd} = (\text{rnd}_i)_{i \in [k]} \leftarrow \mathcal{U}((r_i)_{i \in [k]}) \\ (\curvearrowright, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]}, (\rho_i)_{i \in [k]}) \leftarrow \\ \text{Exp}_{\text{IP}}^{\text{sr}}((s_i)_{i \in [k]}, \text{rnd}, \tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}) \end{array} \right] \leq \epsilon_{\text{IP}}^{\text{sr}}((s_i)_{i \in [k]}, k).$$

Experiment $\text{Exp}_{\text{IP}}^{\text{sr}}((s_i)_{i \in [k]}, \text{rnd}, \tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}})$
INPUT: $((s_i)_{i \in [k]}, \text{rnd}, \tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}})$. OUTPUT: $(\curvearrowright, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$. 1 : $H \leftarrow \mathcal{U}(2\lambda)$ 2 : while $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ does not exit do 3 : $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}, H}$ outputs $(\curvearrowright, (\alpha_1, \dots, \alpha_i), (\text{st}_1, \dots, \text{st}_i))$ where $\text{st}_i \in \{0, 1\}^{s_i}$ 4 : Set $\rho_i \leftarrow \text{rnd}_i(\curvearrowright, (\alpha_1, \dots, \alpha_i), (\text{st}_1, \dots, \text{st}_i))$ 5 : Provide ρ_i to $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}, H}$ 6 : $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}, H}$ exits with output $(\curvearrowright, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]})$ 7 : for $i \in [k]$, set $\rho_i \leftarrow \text{rnd}_i(\curvearrowright, (\alpha_1, \dots, \alpha_i), (\text{st}_1, \dots, \text{st}_i))$ 8 : return $(\curvearrowright, (\alpha_i)_{i \in [k]}, (\text{st}_i)_{i \in [k]}, (\rho_i)_{i \in [k]})$

Fig. 6.2. The state-restoration game for interactive proofs.

6.3.2 EUF-KO to SRS Soundness

We now recall a list-game bound that the reduction relies on. The proof details can be found in FAEST [BBB⁺25a, Section 9.5].

Proposition 29 (Random oracle list game [BBB⁺25a]). Let $n \in \mathbb{N}$, and for $i \in \{0, \dots, n+1\}$ let $\mathcal{V}_i, \mathcal{V}'_i \subseteq \{0, 1\}^*$ be finite sets. For each $i \in \{0, \dots, n\}$ let $\text{Hash}_i: \mathcal{V}_i \times \mathcal{V}'_i \rightarrow \mathcal{V}_{i+1}$ be a random oracle, and set $Y := \min_{i=1}^n |\mathcal{V}_i|$. Consider the following game with an adversary $\mathcal{B}$: for $i \in [1, n]$ the game keeps a list $L_i \subseteq \mathcal{V}_i$, initially empty. For $i \in \{0, \dots, n\}$, the adversary may query Hash_i on a pair (x, x') , receiving y :

1. if $y \in L_{i+1}$, the adversary wins;
2. thereafter, if $i \geq 1$, add x to L_i .

If $\mathcal{B}$ makes at most Q queries to $\text{Hash}_0, \dots, \text{Hash}_n$, its winning probability is at most

$$Q^2/(2Y).$$

Verifier $\mathcal{V}_{\text{IP}}^{\text{H}_0, \text{H}_1, \text{H}_4}(\curvearrowright, (\alpha_1, \dots, \alpha_4), (\rho_1, \dots, \rho_4))$ INPUT: $(\curvearrowright, (\alpha_1, \dots, \alpha_4), (\rho_1, \dots, \rho_4))$. OUTPUT: true or false. 1 : // Decode input 2 : Write $\mathbf{x} = (\text{pk}, \text{param})$, $\alpha_1 = (\text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}})$, $\alpha_2 = (\tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d})$, $\alpha_3 = (\tilde{a}_0 \parallel \dots \parallel \tilde{a}_{\text{deg}-1})$, and $\alpha_4 = (\text{decom}_I)$. 3 : Set $\text{chall}_1 := \rho_1$, $\text{chall}_2 := \rho_2$, and $\text{chall}_3 := \rho_3$. 4 : Set $(\delta_0, \dots, \delta_{\lambda-1}) := \text{chall}_3[0..\lambda-1]$. 5 : Set $\text{iv} \leftarrow \text{H}_4(\text{iv}^{\text{pre}})$. 6 : // Reconstruct VOLEs and check commitment 7 : $(\widehat{\text{com}}, \mathbf{Q}) \leftarrow \text{VOLE.Reconstruct}^{\text{H}_0, \text{H}_1}(\text{decom}_I, \text{chall}_3, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}; \text{param})$. 8 : if $\widehat{\text{com}} = \perp$ or $\widehat{\text{com}} \neq \text{com}$ or $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda] \neq 0$ then return false . 9 : // Apply the VOLE correction values and check consistency 10 : $\tilde{\mathbf{Q}} \leftarrow \text{VOLE.Hash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda+B) \times \lambda}$. 11 : if $\mathbf{D} \neq \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda-1} \cdot \tilde{\mathbf{u}}]$ then return false . 12 : // Compute OWF consistency values 13 : $\hat{a}_0 \leftarrow \text{ZK.OWFVerify}(\mathbf{d}, \mathbf{Q} _{[0..\ell + (\text{deg}-1)\lambda-1]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1 \parallel \dots \parallel \tilde{a}_{\text{deg}-1}, \text{pk}; \text{param})$. 14 : return true if $\hat{a}_0 = \tilde{a}_0$ else false .

Fig. 6.3. The SYDO interactive proof verifier.

Lemma 30 ([BBB⁺25a]). *Let $\mathcal{R}$ be the relation with $(\curvearrowright, \lesssim) \in \mathcal{R}$ iff $\curvearrowright$ is a public key and $\lesssim$ its matching signing key for the SYDO scheme (parameter param left implicit). Let $\curvearrowright = \text{pk} = (\mathbf{x}, \mathbf{y})$ be a purported public key, and let $\mathcal{A}$ be a (non-uniform) algorithm with access to Hash_0 , Hash_1 , Hash_2^0 , Hash_2^1 , Hash_2^2 , Hash_2^3 , Hash_3 , and Hash_4 playing the EUF-KO game of Definition 22 on public key pk . Suppose $\mathcal{A}$ wins with probability at least p , and makes at most $Q_{2,0}$, $Q_{2,1}$, $Q_{2,2}$, and $Q_{2,3}$ queries to Hash_2^0 , Hash_2^1 , Hash_2^2 , and Hash_2^3 , respectively.*

Then there exists a 4-round probabilistic algorithm $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ for $\mathcal{R}$, with access to Hash_0 , Hash_1 , Hash_2^0 , Hash_3 , and Hash_4 , that wins the state-restoration game of Definition 27 against $\mathcal{V}_{\text{IP}}$ with probability at least

$$p - (Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2 / 2^{2\lambda}.$$

Moreover, $\tilde{\mathcal{P}}_{\text{IP}}^{\text{sr}}$ makes at most $(Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)$ queries to the SRS game and runs in time roughly that of $\mathcal{A}$.

6.3.3 SRS IP to SRS IOP

The SRS-IP-to-SRS-IOP step in FAEST [BBB⁺25a] uses a streamlined form of interactive oracle proof (IOP), following [BBdSG⁺23b], and idealizes the BAVC commitment layer before reducing SRS-IP soundness to SRS-IOP soundness.

Definition 31 (Interactive Oracle Proof (IOP) [BBB⁺25a]). *A (public-coin) interactive oracle proof (IOP) [BSCS16, RRR16] for an NP-relation $\mathcal{R}$ is a pair of PPT ITMs $\Pi = (\mathcal{P}, \mathcal{V})$, defined as follows: In each round, the prover $\mathcal{P}$ sends a string $\overline{m}_i = (m_i, g_i)$, where $m_i \in \{0, 1\}^*$ is a message and $g_i \in \{0, 1\}^*$ is an oracle. The verifier $\mathcal{V}$ learns m_i in round i but does not learn g_i . In response, it sends a random challenge $\rho_i \in \mathcal{C}_i$ in each round i . In the final round, the verifier learns all g_i and outputs $b = \mathcal{V}_{\text{IOP}}(\curvearrowright, ((m_1, \dots, m_k), (g_1, \dots, g_k), (\rho_1, \dots, \rho_{k-1})))$.*

For state-restoration soundness, the IOP message in round i is treated as the combined message $\alpha_i := (m_i, g_i)$; applying this FAEST step to SYDO gives the IOP verifier $\mathcal{V}_{\text{IOP}}$ in Figure 6.4.

As in FAEST, the commitment com inside α_1 is replaced by the IOP oracle string g_1 , which lists the pairs $(\mathbf{u}_i, \mathbf{V}_i)$ that VOLE.Commit would produce; hence no commitment opening decom_I is needed in α_4 . VOLE reconstruction then reads the data directly from g_1 through $\text{VOLE.IOPReconstruct}$ (Figure 6.5), which differs from VOLE.Reconstruct only in the source of the VOLE data: it reads $\mathbf{u}_i$ and $\mathbf{V}_i$ directly from g_1 and assembles $\mathbf{Q}$ using the relation enforced by ConvertToVOLE (Proposition 5.2).

Verifier $\mathcal{V}_{\text{IOP}}(\swarrow, (\alpha_1, \dots, \alpha_4), (\rho_1, \dots, \rho_4))$
INPUT: $(\swarrow, (\alpha_1, \dots, \alpha_4), (\rho_1, \dots, \rho_4))$. OUTPUT: true or false. 1 : // Decode input 2 : Write $\mathbf{x} = (\text{pk}, \text{param})$, $\alpha_1 = ((\mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}^{\text{pre}}) \parallel g_1)$, $\alpha_2 = (\tilde{\mathbf{u}} \parallel \mathbf{D} \parallel \mathbf{d})$, and $\alpha_3 = (\tilde{a}_0 \parallel \dots \parallel \tilde{a}_{\text{deg}-1})$. 3 : Set $\text{chall}_1 := \rho_1$, $\text{chall}_2 := \rho_2$, and $\text{chall}_3 := \rho_3$. 4 : Set $(\delta_0, \dots, \delta_{\lambda-1}) := \text{chall}_3[0..\lambda-1]$. 5 : // Reconstruct VOLEs from IOP oracle 6 : $\mathbf{Q} \leftarrow \text{VOLEIOPReconstruct}(g_1, \text{chall}_3, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \hat{\ell}; \text{param}, \text{param}_{\text{VOLE}})$. 7 : if $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda) \neq 0$ then return false. 8 : // Apply the VOLE correction values and check consistency 9 : $\tilde{\mathbf{Q}} \leftarrow \text{VOLE.Hash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda+B) \times \lambda}$. 10 : if $\mathbf{D} \neq \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda-1} \cdot \tilde{\mathbf{u}}]$ then return false. 11 : // Compute OWF consistency values 12 : $\hat{a}_0 \leftarrow \text{ZK.OWFVerify}(\mathbf{d}, \mathbf{Q} _{[0..\ell+(\text{deg}-1)\lambda-1]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1 \parallel \dots \parallel \tilde{a}_{\text{deg}-1}, \text{pk}; \text{param})$. 13 : return true if $\hat{a}_0 = \tilde{a}_0$ else false.

Fig. 6.4. The SYDO IOP verifier.

VOLEIOPReconstruct $(g_1, \text{chall}_3, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \hat{\ell}; \text{param}, \text{param}_{\text{VOLE}})$
INPUT: $(g_1, \text{chall}_3, (\mathbf{c}_i)_{i>0}, \hat{\ell})$. OUTPUT: $\mathbf{Q}$. 1 : Parse $(\delta_0, \dots, \delta_{\lambda-1}) := \text{chall}_3[0..\lambda-1]$. 2 : Parse g_1 as $(\mathbf{u}_i, \mathbf{V}_i)_{i \in [0, \tau)}$. 3 : for $i = 0$ to $\tau - 1$ do 4 : $\mathbf{Q}_i \leftarrow \mathbf{V}_i \oplus [\delta_{i,0} \cdot (\mathbf{u}_i \oplus \mathbf{c}_i) \dots \delta_{i,k_i-1} \cdot (\mathbf{u}_i \oplus \mathbf{c}_i)]$. 5 : return $\mathbf{Q} = [\mathbf{Q}_0 \dots \mathbf{Q}_{\tau-1} \mathbf{0}_{\hat{\ell}, w}]$.

Fig. 6.5. VOLE reconstruction for the SYDO IOP.

Lemma 32 ([BBB⁺25a]). Let $\mathcal{R}$ be the relation such that $(\curvearrowright, \curvearrowleft) \in \mathcal{R}$ if $\curvearrowright$ is a public key and $\curvearrowleft$ the corresponding signing key for the SYDO signature scheme (with parameter param , which we leave implicit). Let $\mathcal{A}$ be a probabilistic polynomial-time algorithm with access to random oracles H_0, H_1, H_4 that, on input an invalid public key $\curvearrowright = \text{pk}$, wins the SRS soundness game defined in Definition 28 against $\mathcal{V}_{\text{P}}$ with probability at least p . Suppose $\mathcal{A}$ makes at most Q_0 queries to the random oracle H_0 and Q_1 queries to H_1 .

Then there exists a 4-round probabilistic algorithm $\tilde{P}_{\text{IOP}}$ that wins the state-restoration game in Definition 27 against $\mathcal{V}_{\text{IOP}}$ with probability at least

$$p - \left(\frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[Q_4, L] \right),$$

using the extractor from Lemma 19, which reprograms H_4 . If $\mathcal{A}$ makes β queries to the SRS game, then $\tilde{P}_{\text{IOP}}$ makes $\beta + 1$ queries.

6.3.4 RBR Soundness

Here we recall the definition of round-by-round soundness from [BBB⁺25a]. In the setting used below, round-by-round soundness implies state-restoration soundness. It requires that, whenever RBRState is still 0 after a transcript prefix, any next prover message can make RBRState become 1 after the next random challenge only with probability at most the round-by-round soundness error.

Definition 33 (Round-by-Round Soundness [BBB⁺25a]). For an interactive proof Π for a language $\mathcal{L}$, let RBRState be a state function mapping an instance and a transcript prefix to a Boolean value, satisfying the following properties. For $\text{inst} \notin \mathcal{L}$:

1. $\text{RBRState}(\text{inst}, \emptyset) = 0$, where $\emptyset$ denotes the empty transcript prefix.
2. If for some instance inst and a transcript prefix trc that is either empty ($i = 0$) or ends with the i -th challenge, and $\text{RBRState}(\text{inst}, \text{trc}) = 0$, then for any prover message pmsg_{i+1} , it holds that

$$\Pr_{\text{chall}_{i+1}} [\text{RBRState}(\text{inst}, \text{trc} \parallel \text{pmsg}_{i+1} \parallel \text{chall}_{i+1}) = 1] \leq \epsilon,$$

where chall_{i+1} is sampled uniformly from the appropriate challenge space, and the round-by-round soundness error ϵ depends only on the security parameter.

3. For any complete transcript trc , if $\text{RBRState}(\text{inst}, \text{trc}) = 0$, then the verifier rejects.

We say that Π has round-by-round (RBR) soundness if for all no-instances $\text{inst} \notin \mathcal{L}$ and all transcripts trc , ϵ is bounded above by a negligible function of the security parameter.

We say that a challenge chall_{i+1} is isolated if, given $\text{RBRState}(\text{inst}, \text{trc}) = 0$, the event $\text{RBRState}(\text{inst}, \text{trc} \parallel \text{pmsg}_{i+1} \parallel \text{chall}_{i+1}) = 1$ depends only on chall_{i+1} .

We say that Π is RBR-sound with passive prefix up to chall_i if, for a partial transcript trc ending in chall_{j-1} , given $\text{RBRState}(\text{inst}, \text{trc}) = 0$, the event

$$\text{RBRState}(\text{inst}, \text{trc} \parallel \text{pmsg}_{i+1} \parallel \text{chall}_{i+1}) = 1$$

is independent of the prefix of trc ending in chall_m for $m = \min(i, j - 1)$.

Lemma 34 (adapted from [BBB⁺25a]). Let $\mathcal{R}$ be the relation such that $(\curvearrowright, \curvearrowleft) \in \mathcal{R}$ if $\curvearrowright$ is a public key and $\curvearrowleft$ the corresponding signing key for the SYDO signature scheme (with parameter param , which we leave implicit).

Let $\hat{\ell} \leq 2^{13}$, $\ell' \leq 2^{13}$, $\epsilon_v = 2^{-\lambda-B}(1 + 2^{B-50})$, $\epsilon_{zk} = 2^{-\lambda}(1 + 2^{-38})$, and $2 \log_2(\tau) \leq \min\{B, 50\}$. Then the SYDO IOP with verifier $\mathcal{V}_{\text{IOP}}$ has round-by-round soundness with soundness error $(d-1)/2^\lambda$ for every no-instance $\curvearrowright \notin \mathcal{R}$. For the selected parameters in this specification, $d = 4$, and this bound is $3/2^\lambda$.

We assemble the SRS-to-RBR chain in the corollary below.

Corollary 35 (adapted from [BBB⁺25a]). Consider the setting of Lemma 30 with an invalid public key, i.e., a public key outside the image of SYDO.KeyGen. In particular, $\mathcal{A}$ makes at most Q_0 , Q_1 , $Q_{2,i}$ for $i \in \{0, 1, 2, 3\}$, and Q_4 queries to the random oracles Hash_0 , Hash_1 , Hash_2^i , and Hash_4 ,

respectively. Suppose that $\hat{\ell} \leq 2^{13}$, $\ell' \leq 2^{13}$ and $2\log_2(\tau) \leq \min\{B, 50\}$. Let d denote the constraint degree in $\text{param}_{\text{RSD}}$. Then we have

$$\begin{aligned} p &\leq (Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2 / 2^{2\lambda} \\ &\quad + \frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvLeafColl}_{\text{B}'}^{\text{BAVC.LeafCommit}} \\ &\quad + (Q_{2,1} + Q_{2,2} + Q_{2,3} + 9) \cdot (d - 1) / 2^\lambda, \end{aligned}$$

where $\text{AdvLeafColl}_{\text{B}'}^{\text{BAVC.LeafCommit}} \leq \text{AdvInj}_{\text{B}}^{\text{PRG}_{2\lambda}}[Q_4, L]$.

6.4 EUF-CMA

We next show that SYDO is EUF-CMA secure, assuming the security of the underlying primitives.

Theorem 36 (SYDO is EUF-CMA secure). *Assume that $H_0, H_1, H_2^1, H_2^2, H_2^3$ and H_4 are modeled as random oracles, while H_3 is treated as a PRF. For any PPT adversary $\mathcal{A}$ making Q_{sig} queries to the signing oracle $\mathcal{O}_{\text{sig}}$, and making $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to H_0, H_1, H_2^1, H_2^2 and H_2^3 , respectively, its EUF-CMA advantage against SYDO satisfies*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{SYDO}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-KO}^{\text{SYDO}} \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{BAVC.LeafCommit}}[1] \right) \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}} \right). \end{aligned}$$

Therefore, the above reduction and security bound apply to SYDO.

The proof is essentially the same as the corresponding argument in FAEST [BBB⁺25a, Section 9.6]; for completeness, we reproduce it here almost verbatim, with the necessary adaptations to the SYDO setting.

Proof (Proof of Theorem 36, adapted from [BBB⁺25a]). The proof replaces the real signing oracle by a simulator through a sequence of hybrid games. The sequence begins with the ordinary EUF-CMA experiment, where signatures are generated with the secret key sk . It ends with a game in which signing responses no longer depend on sk . In the final game, a successful forgery produced by $\mathcal{A}$ can be used directly as a forgery in the EUF-KO experiment. Hence, after the signing oracle has been simulated, the remaining success probability is bounded by the EUF-KO advantage.

Game G_1 : This is the real EUF-CMA experiment. The adversary $\mathcal{A}$ has access to the genuine signing oracle $\mathcal{O}_{\text{sig}}$, which signs using the actual secret key. Therefore,

$$\Pr[G_1 = 1] = \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{SYDO}}[Q_{\text{sig}}]. \quad (11)$$

Game G_2 : In this game, the randomness used at the beginning of each signing query is detached from the secret key. More precisely, the signing oracle samples $r \leftarrow \{0, 1\}^\lambda$ and $\text{iv}^{\text{pre}} \leftarrow \{0, 1\}^\lambda$ uniformly at random, instead of computing them from $H_3(\text{sk} \parallel \mu \parallel \rho)$.

The change is justified by the PRF security of H_3 . Since the replacement is performed independently for each signing query, a standard hybrid argument gives

$$\Pr[G_1 = 1] \leq \Pr[G_2 = 1] + Q_{\text{sig}} \cdot \text{AdvPRF}_{\mathcal{A}_{\text{PRF}}}^{H_3}. \quad (12)$$

Game G_3 : The first Fiat–Shamir challenge is now sampled before programming the corresponding random oracle value. For every signing query, the signing oracle samples $\text{chall}_1 \leftarrow \{0, 1\}^{5\lambda+64}$ uniformly at random. When the input

$$\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \cdots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}$$

is submitted to H_2^1 , the game aborts if this query has already appeared; otherwise, H_2^1 is programmed to output chall_1 on this input.

The abort event corresponds to predicting the programmed input before the signing oracle reaches it. This probability is bounded using the unpredictability of [VOLE.Commit](#). For the hybrid argument, let $G_{3,i}$ denote the variant in which the abort check is enabled only for the first i signing queries. Then $G_{3,0} = G_2$ and $G_{3,Q_{\text{sig}}} = G_3$, while two consecutive hybrids differ only in the i -th signing query.

The reduction $\mathcal{A}_{\text{unpred},i}$ to Lemma 21 emulates $G_{3,i}$. On the i -th query to $\mathcal{O}_{\text{sig}}$, it provides the unpredictability experiment with its current state and the list $\mathcal{L}_{2,1}$ of previous queries to H_2^1 . It then receives $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1})$ from the experiment instead of executing [VOLE.Commit](#), and continues the simulation. By construction,

$$G_{3,i-1} = 1 \iff \text{ExpUnpre}_{0, \mathcal{A}_{\text{unpred}}}^{\text{VOLE.Commit}, \hat{\ell}, \text{param}} = 1$$

and

$$G_{3,i} = 1 \iff \text{ExpUnpre}_{1, \mathcal{A}_{\text{unpred},i}}^{\text{VOLE.Commit}, \hat{\ell}, \text{param}} = 1.$$

Applying Lemma 21 over all signing queries gives

$$\begin{aligned} \Pr[G_3 = 1] &\leq e^{Q_{\text{sig}} \cdot (4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^\lambda} \cdot \left(\Pr[G_1 = 1] \right. \\ &\quad \left. + Q_{\text{sig}} \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_B^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{BAVC.LeafCommit}}[1] + \frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} \right) \right). \end{aligned}$$

Game G_4 : The second Fiat–Shamir challenge is sampled directly. In each signing query, the signing oracle samples $\text{chall}_2 \leftarrow \{0, 1\}^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}$ uniformly at random. When $\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel h_V \parallel \mathbf{d}$ is queried to H_2^2 , the game aborts if this query has already appeared; otherwise, H_2^2 is programmed to output chall_2 .

A failure can occur only if a prior H_2^2 query already begins with the fresh value chall_1 . Thus,

$$\Pr[G_3 = 1] \leq \Pr[G_4 = 1] + Q_{\text{sig}} \cdot \frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda + 64}}.$$

Game G_5 : The third challenge is handled analogously, with the rejection condition from the signing algorithm preserved. During each signing query and each loop iteration of [SYDO.Sign](#), the oracle samples $\text{chall}_3 \leftarrow \{0, 1\}^\lambda$ uniformly at random until the loop terminates with $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda) = 0^{w_{\text{grind}}}$ and $I = \text{VOLE.DecodeAllChall}(\text{chall}_3; \text{param})$ such that $\text{decom}_I \neq \perp$. When $\text{chall}_2 \parallel \tilde{a} \parallel \tilde{b} \parallel \text{ctr}$ is queried to H_2^3 , the game aborts if this query has already occurred; otherwise, H_2^3 is programmed to return chall_3 .

Since chall_2 is fresh after G_4 , the only relevant bad event is a previous query to H_2^3 with this prefix. Consequently,

$$\Pr[G_4 = 1] \leq \Pr[G_5 = 1] + Q_{\text{sig}} \cdot \frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}}.$$

Game G_6 : At this point, the transcript components relevant to [VOLE.Commit](#) match the multi-hiding experiment from Lemma 20. The actual call to [VOLE.Commit](#) is replaced as follows. For every signing query, the oracle samples $\mathbf{u} \leftarrow \mathbb{F}_2^{\hat{\ell}}$ and $(\mathbf{c}_i)_{i \in [1, \tau]}$ uniformly, and then chooses $\mathbf{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda}$ so that the VOLE relation holds with the values Δ_i and $\mathbf{q}_i$ determined by chall_3 .

The embedding of the hiding experiment uses the same sampling process for chall_3 as in G_5 , except that the successful challenge satisfying $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda) = 0^{w_{\text{grind}}}$ and $\text{decom}_I \neq \perp$ is taken from the hiding experiment. Therefore, the successful counter ctr is well-defined, and the required challenges may be sampled in advance. The non-adaptive hiding property in Lemma 20 is sufficient for this step.

Using Lemma 20 together with a hybrid argument yields

$$\begin{aligned} \Pr[G_5 = 1] &\leq e^{Q_{\text{sig}} \tau (4\lceil \log(L) \rceil \beta^2 + 16\beta^2 N_0^2 + (\hat{\ell}/\lambda)^2)/2^\lambda} \cdot \left(\Pr[G_6 = 1] \right. \\ &\quad \left. + Q_{\text{sig}} \tau \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}_{B_1}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}_{B_2}^{\text{BAVC.LeafCommit}}[N_0] + \text{AdvPRP}_{B_3}^{\text{PRG}_{\hat{\ell}}}[1] \right) \right). \end{aligned}$$

In G_6 , the pair $(\mathbf{u}, \mathbf{V})$ is uniformly distributed subject to the VOLE relation and is independent of the VOLE commitments $(\mathbf{c}_i)_{i \in [1, \tau]}$.

Game G₇: The next change removes the dependence on the actual evaluation of [VOLE.Hash](#) during signing. For each signing query, the oracle samples $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$ uniformly at random, subject to the VOLE relation with $\tilde{\mathbf{Q}}$ and Δ . It then modifies the last $\lambda + B$ entries of $\mathbf{u}$ and the corresponding rows of $\mathbf{V}$ so that applying [VOLE.Hash](#) to $\mathbf{u}, \mathbf{V}$ under challenge chall_1 yields $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{V}}$.

By G_6 , the modified rows are uniform subject to the VOLE relation. Together with the $\mathbb{F}_2^{\ell+\lambda}$ -hiding property of [VOLE.Hash](#), this implies perfect indistinguishability from the previous game.

Game G₈: This game treats the [ZKHash](#) output in the same simulator-oriented manner. In each signing query, $\tilde{a}_0, \dots, \tilde{a}_{d-1}$ are sampled uniformly at random instead of being computed by [ZKHash](#) inside [RSD.Prove](#). The oracle still constructs the RSD coefficient vectors, but modifies $\mathbf{u}, \mathbf{V}$ in rows $[\ell_w, \ell_w + (d-1)\lambda)$ so that applying [ZKHash](#) under challenge chall_2 to $(\mathbf{z}_r, \mathbf{a}_r, b_r)$ gives $\tilde{a}_r$ for every $r \in [0..d-1]$.

Since the affected rows are uniform by G_6 , and since [ZKHash](#) is hiding by Lemma 8, the distribution of $(\tilde{a}_0, \dots, \tilde{a}_{d-1})$ is unchanged. Hence, this game is perfectly indistinguishable from G_7 .

Game G₉: Finally, the mask of the witness is replaced by a uniform value. Since $\tilde{a}_0, \dots, \tilde{a}_{d-1}$ have already been generated independently of $\preceq$, the oracle samples $\mathbf{d}$ uniformly at random in each query instead of computing $\mathbf{d} := \preceq \oplus \mathbf{u}_{[0, \ell)}$.

This does not alter the distribution of $\mathbf{d}$, because $\mathbf{u}$ is uniform by G_6 and its first ℓ entries are not used in the transformations of G_7 and G_8 . Thus, the signature σ output in G_9 is independent of the secret key sk .

Combining the losses from the intermediate games gives

$$\begin{aligned} \Pr[G_0 = 1] &\leq \rho \cdot \left(\Pr[G_9 = 1] \right. \\ &\quad + Q_{\text{sig}} \cdot \text{AdvPRF}^{\text{H}_3} \\ &\quad + Q_{\text{sig}} \cdot \left(\lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{BAVC.LeafCommit}}[1] + \frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} \right) \\ &\quad \left. + Q_{\text{sig}} \cdot \left(\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}} \right) \right) \\ &= \rho \cdot \Pr[G_9 = 1] \\ &\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{\text{H}_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{BAVC.LeafCommit}}[1] \right) \\ &\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}} + \frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}} \right), \end{aligned}$$

where

$$\rho = e^{Q_{\text{sig}} \cdot (4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^\lambda + Q_{\text{sig}} \tau (4\lceil \log(L) \rceil \beta^2 + 16\beta^2 N_0^2 + (\lceil \ell/\lambda \rceil)^2)/2^\lambda}$$

is at most 2 for all proposed parameter sets. Therefore, the factor ρ is absorbed into the leading constant in the theorem statement.

It remains to relate the last game to EUF-KO security. In G_9 , the signing oracle can be simulated without using sk . A reduction can therefore answer all signing queries internally and submit the final forgery of $\mathcal{A}$ as its own EUF-KO forgery. The formal issue caused by programming random oracles does not change the bound: expanding the EUF-KO proof inside this reduction would reproduce the same collision, preimage and random-oracle guessing terms. This completes the proof.

6.5 QROM proof

This subsection follows the QROM proof of FAEST [\[BBB⁺25a\]](#), with the necessary details adjusted to the SYDO setting. For completeness, we state the main QROM reductions and bounds used for SYDO, while omitting the intermediate proofs of the expanded interactive proof, its RBR soundness, and the compressed-oracle search reduction. We refer the reader to FAEST [\[BBB⁺25a\]](#) for the full details.

The QROM proof follows the same outline as the ROM proof: EUF-CMA security is reduced to EUF-KO security, and EUF-KO security is reduced to the soundness of the Fiat–Shamir transform

of the expanded interactive proof. The QROM-specific step is handled by the compressed-oracle search bound and the quantum-guess version of the unpredictability property for [VOLE.Commit](#), as in [FAEST](#).

Lemma 37 (adapted from [FAEST \[BBB+25a\]](#)). *Let $\mathcal{A}^{H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_3, H_4}$ be a QROM EUF-KO adversary against SYDO that makes a total of $Q_0, Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_3, Q_4$ queries to its oracles. Then there exist a quantum distinguisher $\mathcal{B}_1$ for the RSD distribution and a QROM adversary $\mathcal{B}^{H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4}$ against $\text{SYDO}_r^{\text{elc}}$ that makes at most $Q_0 + 1, Q_1 + \tau + 1, Q_{2,0} + 1, Q_{2,1} + 1, Q_{2,2} + 1, Q_{2,3} + 1, Q_4 + 1$ queries to the corresponding oracles, such that*

$$\text{AdvEUFKO}_{\mathcal{A}}^{\text{SYDO}} \leq \frac{1}{1 - \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}} \cdot \left(\text{AdvSnd}_{\mathcal{B}}^{\text{SYDO}_r^{\text{elc}}} + \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}} + \text{AdvInj}_{\mathcal{C}}^{\text{PRG}_{2\lambda}}[L] \right).$$

Proof (Proof of Lemma 37). The proof is the corresponding [FAEST](#) argument with the verification procedure and explicit leaf commitments instantiated for SYDO. Let $\mathcal{A}'^{H_0, H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4}$ be the adversary that runs $\mathcal{A}$ while simulating H_3 locally. Since [SYDO.Verify](#) never queries H_3 , this change is perfectly indistinguishable, and hence

$$\text{AdvSnd}_{\mathcal{A}}^{\text{SYDO}} = \text{AdvSnd}_{\mathcal{A}}^{\text{SYDO}_r} = \text{AdvSnd}_{\mathcal{A}'}^{\text{SYDO}_r}.$$

From $\mathcal{A}'$, construct a prover $\mathcal{B}$ for $\text{SYDO}_r^{\text{elc}}$. Given pk , the prover runs $(\text{msg}, \sigma) \leftarrow \mathcal{A}'(\text{pk})$ and executes [SYDO.Verify](#)($\text{msg}, \text{pk}, \sigma$), retaining the intermediate values needed to form the explicit prover messages. The message pmsg_{-1} is obtained from the values $h_{i,j}$ hashed in Line 36 of [BAVC.Reconstruct](#), as invoked by [VOLE.Reconstruct](#) and then by [SYDO.Verify](#), and these values are used in place of $\text{com}_{i,j}$. For pmsg_i with $i \in \{1, 2, 3\}$, the corresponding hash inputs from Lines 10, 15, and 18 of [SYDO.Verify](#) are used. Thus, every accepting execution of [SYDO.Verify](#) gives an accepting proof for $\text{SYDO}_r^{\text{elc}}$, and

$$\text{AdvSnd}_{\mathcal{A}'}^{\text{SYDO}_r} = \text{AdvSnd}_{\mathcal{B}}^{\text{SYDO}_r^{\text{elc}}}.$$

For SYDO, the same construction first gives

$$\text{AdvSnd}_{\mathcal{A}'}^{\text{SYDO}_r} = \text{AdvSnd}_{\mathcal{B}}^{\widetilde{\text{SYDO}_r^{\text{elc}}}},$$

where $\widetilde{\text{SYDO}_r^{\text{elc}}}$ is the same as $\text{SYDO}_r^{\text{elc}}$, except that its verifier does not reject when some $\text{com}_{i,j}$ has multiple preimages. The gap to the actual verifier is bounded by the almost-injectivity advantage of the IV-based tweakable PRG:

$$\text{AdvSnd}_{\mathcal{B}}^{\widetilde{\text{SYDO}_r^{\text{elc}}}} - \text{AdvSnd}_{\mathcal{B}}^{\text{SYDO}_r^{\text{elc}}} \leq \text{AdvInj}_{\mathcal{C}}^{\text{PRG}_{2\lambda}}[L].$$

The stated bound then follows from Equation (10), which holds in the QROM as well.

Theorem 38 (adapted from [\[BBB+25a\]](#)). *Assume that $H_0, H_1, H_2^1, H_2^2, H_2^3$ and H_4 are modeled as quantum-accessible random oracles. Assume also that H_3 is a PRF. For any QPT adversary $\mathcal{A}$ making Q_{sig} queries to $\mathcal{O}_{\text{sig}}$, and making $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to H_0, H_1, H_2^1, H_2^2 and H_2^3 , respectively, the following bound holds:*

$$\begin{aligned} \text{AdvEUFMA}_{\mathcal{A}}^{\text{SYDO}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUFKO}_{\mathcal{A}}^{\text{SYDO}} \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{BAVC.LeafCommit}}[1] \right) \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}} \right) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right. \\ &\quad \left. + \frac{3}{2} \cdot \sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}} + \frac{3}{2} \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}}} \right), \end{aligned}$$

where $\text{AdvPRP}^{\text{BAVC.LeafCommit}}$ denotes the PRP advantage of the leaf-commitment primitive instantiated in SYDO.

Proof (Proof of Theorem 38). The proof follows the same hybrid sequence as Theorem 36; the only changes are the game hops whose bounds rely on quantum-accessible random oracles. The hop programming H_2^1 is bounded by the quantum-guess unpredictability argument for VOLE.Commit from FAEST [BBB⁺25a], giving the term

$$\left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}}\right) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda}.$$

The hops programming H_2^2 and H_2^3 use adaptive reprogramming in the QROM, giving respectively

$$\frac{3}{2} \cdot \sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}} \quad \text{and} \quad \frac{3}{2} \cdot \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}}}.$$

All remaining game hops are unchanged from Theorem 36, with computational advantage terms interpreted against quantum algorithms. Combining these losses gives the claimed bound.

The following corollary combines the QROM EUF-CMA-to-EUF-KO reduction with the EUF-KO bound.

Corollary 39 (adapted from [BBB⁺25a]). *Assume that $H_0, H_1, H_2^1, H_2^2, H_2^3$ and H_4 are modeled as quantum-accessible random oracles. Assume also that H_3 is a PRF. For any QPT adversary $\mathcal{A}$ making Q_{sig} queries to $\mathcal{O}_{\text{sig}}$, and making $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to H_0, H_1, H_2^1, H_2^2 and H_2^3 , respectively,*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{SYDO}}[Q_{\text{sig}}] &\leq 2 \cdot \text{AdvEUF-KO}_{\mathcal{A}}^{\text{SYDO}} \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] + \text{AdvPRP}^{\text{BAVC.LeafCommit}}[1] \right) \\ &\quad + 2 \cdot Q_{\text{sig}} \cdot \left(\left(3\sqrt{Q_1} + 2\sqrt{Q_{2,1}} \right) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right) \\ &\quad + 3 \cdot Q_{\text{sig}} \cdot \left(\sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda+64}}} + \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{\lceil (n-k)/\lambda \rceil \lambda + 3\lambda + 64}}} \right), \end{aligned}$$

where the EUF-KO term is bounded by

$$\begin{aligned} \text{AdvEUF-KO}_{\mathcal{A}}^{\text{SYDO}} &\leq \frac{1}{1 - \frac{|\text{dom}(\text{OWF})|}{|\text{cod}(\text{OWF})|}} \cdot \left(10 \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot 2^{-\lambda} \right. \\ &\quad \left. + \text{AdvDist}_{\mathcal{B}_1}^{\text{RSD}} + \text{AdvInj}_{\mathcal{C}}^{\text{PRG}_{2\lambda}}[L] \right). \end{aligned}$$

Here,

$$Q = Q_0 + Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4 + 3\tau + 19.$$

7 Concrete Security Analysis

We analyze three classical security levels for SYDO: 160, 256, and 512 bits. Following the standard Grover-type quadratic speedup for exhaustive search [Gro96], these correspond to target quantum security levels of 80, 128, and 256 bits, respectively. Throughout this section, whenever a parameter choice or primitive is selected to provide about 2^c classical security against search-dominated attacks, we interpret it as providing about $2^{c/2}$ quantum security under this Grover-type estimate. This convention is applied both to the selected SD/RSD instances and to the symmetric primitives used in the construction.

7.1 Attacks Against the SD problem

For the selected RSD parameters in Table 5.2, Table 7.1 reports the concrete security estimates under classical attacks. We consider the standard line of binary SD attacks: Prange [Pra62], Stern [Ste89], Dumer [Dum89], BJMM [BJMM12a], May–Ozerov [MO15], and Both–May [BM18]. We also include

their variants [Pet10, MMT11, Hir16, Ess22]. We estimate their costs using Cryptographic Estimators¹ [EVZB24]. The choice of memory-access model and memory bounds follows the memory analysis in Section 5.2. Concretely, for the parameter sets selected in Table 5.2, we use constant-cost memory accesses and memory bounds of 2^{160} , 2^{192} , and 2^{256} for the 160-, 256-, and 512-bit parameter sets, respectively.

Table 7.1. Concrete security estimates for the selected SD parameters under classical attacks, time and memory bounds are given as base-2 exponents

Algorithm	Cost	Parameter Sets					
		SYDO-160s	SYDO-160f	SYDO-256s	SYDO-256f	SYDO-512s	SYDO-512f
BallCollision	Time	195.5	195.5	305.2	305.2	580.9	580.9
	Memory	141.6	141.6	189.5	189.5	246.7	246.7
BJMM-DW	Time	167.3	167.3	271.9	271.9	545.2	545.2
	Memory	155.8	155.8	191.6	191.6	234.7	234.7
BJMM-PDW	Time	167.1	167.1	275.6	275.6	538.5	538.5
	Memory	158.1	158.1	183.4	183.4	250.5	250.5
BJMM	Time	166.8	166.8	275.9	275.9	538.6	538.6
	Memory	157.3	157.3	185.6	185.6	252.8	252.8
BJMM-D2	Time	172.5	172.5	275.9	275.9	538.6	538.6
	Memory	145.9	145.9	185.6	185.6	252.8	252.8
BJMM-D3	Time	166.8	166.8	276.3	276.3	546.1	546.1
	Memory	157.3	157.3	183.2	183.2	242.6	242.6
BJMM+	Time	164.7	164.7	262.5	262.5	517.3	517.3
	Memory	159.4	159.4	191.4	191.4	255.3	255.3
Both-May	Time	164.4	164.4	274.7	274.7	559.2	559.2
	Memory	153.1	153.1	183.2	183.2	190.5	190.5
Dumer	Time	194.2	194.2	305.1	305.1	580.8	580.8
	Memory	151.1	151.1	189.6	189.6	246.8	246.8
May-Ozerov	Time	164.1	164.1	261.8	261.8	524.9	524.9
	Memory	151.7	151.7	189.7	189.7	246.8	246.8
May-Ozerov-D2	Time	164.1	164.1	261.8	261.8	524.9	524.9
	Memory	151.7	151.7	189.7	189.7	246.8	246.8
May-Ozerov-D3	Time	166.6	166.6	281.6	281.6	567.2	567.2
	Memory	147.6	147.6	157.0	157.0	151.4	151.4
Prange	Time	242.7	242.7	368.6	368.6	669.3	669.3
	Memory	23.1	23.1	24.5	24.5	26.3	26.3
Stern	Time	194.3	194.3	305.2	305.2	580.9	580.9
	Memory	150.9	150.9	189.5	189.5	246.7	246.7

7.2 Other Known Attacks

We now analyze the security of SYDO with respect to known attacks.

For SYDO, the public key is an SD/RSD instance, and recovering the signing key from the public key amounts to finding a valid low-weight solution for that instance. The concrete cost of this attack

¹ <https://github.com/Crypto-TII/CryptographicEstimators>

is estimated in Table 7.1. The remaining attack surface comes from the symmetric primitives used for GGM tree expansion, leaf commitments, VOLE correlation generation, Fiat–Shamir challenges, and transcript hashing. As summarized in Table 5.1, PRG is instantiated with Rijndael-192, Rijndael-256, or the internal block cipher of BLAKE2s with a 512-bit key and a 512-bit block. The hash functions are instantiated using the pseudo-XOF function provided by NGCC. We next discuss the relevant attack considerations for these components and explain how the selected parameters meet the intended security levels.

7.2.1 Brute Force the PRG

The tweakable PRG PRG, with inputs (sd, iv, twk), is used in BAVC.Commit and VOLE.BuildVOLE. As specified in Table 5.1, it is instantiated by Rijndael-192 or Rijndael-256 for the 160- and 256-bit security levels, and by the internal block cipher of BLAKE2s with a 512-bit key and a 512-bit block for the 512-bit security level. The analysis follows the corresponding discussion for FAEST [BBB+25a, Section 9.1.2]: brute-force attacks on the PRG do not benefit from combining different calls, because of the way the twk values are chosen. In SYDO, the calls used for GGM tree expansion, leaf commitments, and VOLE correlation generation are separated by their twk values. Thus a key guess for one call cannot be tested simultaneously against many unrelated PRG targets. Consequently, recovering the signing key through these PRG calls costs essentially the same as attacking a single PRG target, namely about 2^λ evaluations of the underlying primitive in the classical setting.

Given Multiple Signatures. Each signature samples an independent $iv \in \{0, 1\}^\lambda$. For Q_{sig} signing queries, the corresponding iv-collision term is bounded by the usual birthday estimate, about $Q_{\text{sig}}^2/2^\lambda$. Thus a λ -bit iv supports up to about $2^{\lambda/2}$ signing queries at this level of analysis. Even the smallest parameter sets use $\lambda = 160$, which gives the required margin for $Q_{\text{sig}} \leq 2^{80}$; the 256- and 512-bit parameter sets provide larger margins. Together with the separation of the twk values between different PRG call sites, this prevents a brute-force PRG attack from combining different calls or different signatures into a cheaper target. This in particular meets the NGCC signing-query requirement.

7.2.2 Attack Soundness of the ZK Protocol

Instead of recovering the signing key, an attacker may try to forge a signature by using an invalid witness while still passing the checks of the ZK protocol. This case is already covered by the EUF-KO analysis in Theorem 25, which accounts for the soundness of the proof system together with the assumptions on the commitment and hash components. We therefore do not treat attack soundness as a separate concrete attack beyond the formal security analysis.

7.2.3 Multi-User Attacks

The multi-user considerations are the same as in FAEST [BBB+25a, Section 9.1.4], except that the public-key relation in SYDO is an RSD instance rather than a symmetric-key OWF. Each public key fixes its own independently sampled pair $(\mathbf{H}, \mathbf{y})$, where $\mathbf{y} = \mathbf{H}\mathbf{x}$ for the corresponding secret solution. Therefore, recovering one user’s secret key still requires solving that user’s RSD instance; other public keys do not give a shared target for a cheaper multi-target search. The random iv sampled for each signature also keeps the PRG calls independent across signatures and users, as discussed above.

Key substitution attacks are handled as in FAEST. An attacker given a signature σ under pk may try to make a signature verify under another public key pk' for the same message [MS04]. In SYDO, the message digest is computed as $\mu = \text{Hash}_0(\text{pk} \parallel \text{msg}; 2\lambda)$, so the public key is bound to the Fiat–Shamir transcript. This prevents reusing a signature under a different public key.

8 Performance Evaluation

8.1 Benchmark Environment

Table 8.1 summarizes the benchmarking setup, including the language, compiler, and instruction set extensions used for each implementation, as well as the common processor, clock frequency, memory, operating system, and library configuration.

Table 8.1. Benchmark environment

Item	Reference implementation	AVX2-optimized implementation
Language	ISO C	C++
Compiler	GCC 14.3.0	G++ 14.3.0
Instruction set extensions	No platform-specific extensions	AVX2, AES-NI, PCLMULQDQ, SSSE3
Processor	AMD Ryzen 9 3900X CPU, single core only	
Clock frequency	3.8 GHz, with turbo boost disabled	
Memory	64 GB	
Operating system	Ubuntu 22.04	
Library	None	

8.2 Benchmark Results

This subsection reports the benchmark results for the selected SYDO instances. For each instance, we report the average running times of key generation, signing, and verification in milliseconds and million cycles, together with the secret-key, public-key, signature, and public-key-plus-signature sizes in bytes. We evaluate two implementations under the environment in Table 8.1.

- The AVX2-optimized implementation is written in C++, compiled with G++ 14.3.0, and targets x86-64 architectures with the AVX2, AES-NI, PCLMULQDQ, and SSSE3 instruction set extensions. Table 8.2 reports the benchmark results for this implementation.
- The reference implementation is written in ISO C, compiled with GCC 14.3.0, and does not use platform-specific instruction-set extensions. Table 8.3 reports the benchmark results for this implementation.

Table 8.2. Benchmark results for the architecture specific implementation for x86-64 with AVX2

Instance	Runtimes in ms, Mcyc						Sizes (bytes)			
	KeyGen		Sign		Verify		sk	pk	sig	sig + pk
SYDO-160s	0.014	0.054	23.823	90.358	20.408	77.406	174	80	5428	5508
SYDO-160f	0.014	0.054	4.936	18.721	4.638	17.590	174	80	6724	6804
SYDO-256s	0.022	0.086	45.091	171.026	44.872	170.195	278	128	14444	14572
SYDO-256f	0.023	0.086	9.919	37.621	9.934	37.679	278	128	17604	17732
SYDO-512s	0.105	0.398	348.389	1321.396	322.189	1222.021	534	246	56672	56918
SYDO-512f	0.105	0.397	68.056	258.128	66.717	253.048	534	246	67716	67962

Table 8.3. Benchmark results for the reference implementation

Instance	Runtimes in ms, Mcyc						Sizes (bytes)			
	KeyGen		Sign		Verify		sk	pk	sig	sig + pk
SYDO-160s	0.250	0.950	186.793	708.516	173.597	658.465	174	80	5428	5508
SYDO-160f	0.250	0.949	72.388	274.576	63.830	242.117	174	80	6724	6804
SYDO-256s	0.426	1.619	339.465	1287.649	320.749	1216.661	278	128	14444	14572
SYDO-256f	0.427	1.621	143.055	542.629	123.590	468.795	278	128	17604	17732
SYDO-512s	0.146	0.557	990.741	3758.104	895.061	3395.164	534	246	56672	56918
SYDO-512f	0.145	0.553	454.441	1723.787	392.854	1490.142	534	246	67716	67962

8.3 Performance Analysis

The results in Tables 8.2 and 8.3 are measured for the parameter sets and primitive choices required by NGCC. These results are useful for evaluating the submitted implementation, but they are not directly comparable with existing post-quantum signature schemes. There are two reasons. First, the NGCC security levels are different from the NIST security categories used by standardized signatures and current NIST additional-signature candidates. Second, NGCC requires the hash interface in `auxfunc.h/c`, namely the SM3-based pseudo-XOF, whereas most public implementations of post-quantum signatures use standardized primitives such as SHAKE. This difference affects both the cryptographic baseline and the measured running time, since the SM3-based pseudo-XOF is not the primitive used in the NIST comparison setting and may become a noticeable part of the implementation cost.

For this reason, a separate comparison is given in Appendix A. There we instantiate SYDO at NIST security levels L1, L3, and L5, and use standardized symmetric primitives: AES for the PRG and SHAKE for the XOF. This gives a cleaner comparison with standardized signatures and with the NIST additional-signature submissions SDitH, FAEST, and MQOM. At NIST L1, the combined public-key and signature size of the short variant is 3572 bytes, smaller than ML-DSA and SLH-DSA in the comparison table. Moreover, SYDO improves over SDitH at all three NIST security levels: signatures are smaller by 4.5%–6.1%, signing is $1.51\times$ – $2.25\times$ faster, and verification is $1.24\times$ – $2.24\times$ faster. Compared with FAEST, SYDO gives substantially smaller signatures, while FAEST remains faster; compared with MQOM, SYDO has larger signatures in the reported parameter sets but is faster for the fast variants at the higher security levels. The detailed parameter choices, primitive choices, and benchmark numbers are reported in Appendix A.

References

- AFS03. D. Augot, M. Finiasz, and N. Sendrier. A fast provably secure cryptographic hash function. Cryptology ePrint Archive, Paper 2003/230, 2003.
- BBB⁺25a. C. Baum, W. Beullens, L. Braun, C. D. de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. FAEST v2: Algorithm specifications. NIST’s Post-Quantum Cryptography Standardization of Additional Digital Signature Schemes Project, Round 2, Version 2.0, 2025.
- BBB⁺25b. C. Baum, W. Beullens, L. Braun, C. D. de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. Shorter, tighter, FAESTer: Optimizations and improved (QROM) analysis for VOLE-in-the-head signatures. In *CRYPTO 2025, Part VI, Lecture Notes in Computer Science* 16005, pages 124–156. Springer, Cham, 2025.
- BBdSG⁺23a. C. Baum, L. Braun, C. D. de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, S. Ramacher, C. Rechberger, E. Orsini, L. Roy, and P. Scholl. FAEST: Algorithm specifications. NIST’s Post-Quantum Cryptography Standardization of Additional Digital Signature Schemes Project, Round 1, Version 1.0, 2023.
- BBdSG⁺23b. C. Baum, L. Braun, C. D. de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, and P. Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In *CRYPTO 2023, Part V, LNCS* 14085, pages 581–615. Springer, 2023.
- BBFR25. R. Benadjila, C. Bouillaguet, T. Feneuil, and M. Rivain. MQOM. NIST’s Post-Quantum Cryptography Standardization of Additional Digital Signature Schemes Project (Round 2), <https://mqom.org/>, 2025.
- BBGK24. S. Bettaieb, L. Bidoux, P. Gaborit, and M. Kulkarni. Modelings for generic PoK and applications: Shorter SD and PKP based signatures. Cryptology ePrint Archive, Paper 2024/1668, 2024.
- BBM⁺24. C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In *ASIACRYPT 2024, Part I, LNCS* 15484, pages 463–493. Springer, 2024.
- BGI16. E. Boyle, N. Gilboa, and Y. Ishai. Function secret sharing: Improvements and extensions. In *ACM CCS 2016*, pages 1292–1303. ACM, 2016.
- BJMM12a. A. Becker, A. Joux, A. May, and A. Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In *EUROCRYPT 2012, LNCS* 7237, pages 520–536. Springer, Berlin, Heidelberg, April 2012.
- BJMM12b. A. Becker, A. Joux, A. May, and A. Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. Cryptology ePrint Archive, Report 2012/026, 2012.

- BM18. L. Both and A. May. Decoding linear codes with high error rate and its impact for LPN security. In *Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018*, pages 25–46. Springer, Cham, 2018.
- BMvT78. E. R. Berlekamp, R. J. McEliece, and H. C. A. van Tilborg. On the inherent intractability of certain coding problems. *IEEE Transactions on Information Theory*, 24(3):384–386, 1978.
- BSCS16. E. Ben-Sasson, A. Chiesa, and N. Spooner. Interactive oracle proofs. In *TCC 2016-B, Part II, LNCS 9986*, pages 31–60, Berlin, Heidelberg, 2016. Springer.
- CCJ23. E. Carozza, G. Couteau, and A. Joux. Short signatures from regular syndrome decoding in the head. In *EUROCRYPT 2023, Part V, LNCS 14008*, pages 532–563. Springer, 2023.
- CLY⁺24. H. Cui, H. Liu, D. Yan, K. Yang, Y. Yu, and K. Zhang. ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-head. In *PKC 2024, Part I, LNCS 14601*, pages 229–258. Springer, 2024.
- CY24. A. Chiesa and E. Yogev. Building cryptographic proofs from hash functions. <https://github.com/hash-based-snargs-book>, 2024.
- Dum89. I. Dumer. Two decoding algorithms for linear codes. *Problems of Information Transmission*, 25(1):17–23, 1989.
- Ess22. A. Esser. Revisiting nearest-neighbor-based information set decoding. Cryptology ePrint Archive, Report 2022/1328, 2022.
- EVZB24. A. Esser, J. A. Verbel, F. Zveydinger, and E. Bellini. SoK: **CryptographicEstimators** – a software library for cryptographic hardness estimation. In *ACM ASIACCS 2024*, pages 560–574. ACM, 2024.
- FR25. T. Feneuil and M. Rivain. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Journal of Cryptology*, 38(3):28, 2025.
- FS87. A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO 1986, LNCS 263*, pages 186–194, Berlin, Heidelberg, 1987. Springer.
- Gro96. L. K. Grover. A fast quantum mechanical algorithm for database search. In *28th ACM STOC*, pages 212–219. ACM Press, May 1996.
- Hir16. S. Hirose. May-ozarov algorithm for nearest-neighbor problem over $\mathbb{F}_q$ and its application to information set decoding. Cryptology ePrint Archive, Report 2016/237, 2016.
- KW03. J. Katz and N. Wang. Efficiency improvements for signature schemes with tight security reductions. In *ACM CCS 2003*, pages 155–164. ACM, 2003.
- LDK⁺22. V. Lyubashevsky, L. Ducas, E. Kiltz, T. Lepoint, P. Schwabe, G. Seiler, D. Stehlé, and S. Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- MBB⁺25. C. A. Melchor, S. Bettaleb, L. Bidoux, T. Feneuil, P. Gaborit, N. Gama, S. Gueron, J. Howe, A. Hülsing, D. Joseph, A. Joux, M. Kulkarni, E. Persichetti, T. H. Randrianarisoa, M. Rivain, and D. Yue. SDitH. NIST’s Post-Quantum Cryptography Standardization of Additional Digital Signature Schemes Project (Round 2), <https://sdith.org/>, 2025.
- MMT11. A. May, A. Meurer, and E. Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In *ASIACRYPT 2011, LNCS 7073*, pages 107–124. Springer, Berlin, Heidelberg, December 2011.
- MO15. A. May and I. Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In *EUROCRYPT 2015, Part I, LNCS 9056*, pages 203–228. Springer, Berlin, Heidelberg, April 2015.
- MS04. A. Menezes and N. P. Smart. Security of signature schemes in a multi-user setting. *Designs, Codes and Cryptography*, 33(3):261–274, 2004.
- NIS22. NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf>, 2022.
- NIS26. NIST. Post-quantum cryptography: Additional digital signature schemes – round 3 additional signatures. <https://csrc.nist.gov/Projects/pqc-dig-sig/round-3-additional-signatures>, 2026.
- OTX24. Y. Ouyang, D. Tang, and Y. Xu. Code-based zero-knowledge from VOLE-in-the-head and their applications: Simpler, faster, and smaller. In *ASIACRYPT 2024, Part V, LNCS 15488*, pages 436–470. Springer, 2024.
- Pet10. C. Peters. Information-set decoding for linear codes over F_q . In *The Third International Workshop on Post-Quantum Cryptography, PQCRYPTO 2010*, pages 81–94. Springer, Berlin, Heidelberg, May 2010.
- Pra62. E. Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962.
- RRR16. O. Reingold, G. N. Rothblum, and R. D. Rothblum. Constant-round interactive proofs for delegating computation. In *STOC 2016*, pages 49–62. ACM, 2016.
- SA15. M.-J. O. Saarinen and J.-P. Aumasson. The BLAKE2 cryptographic hash and message authentication code (MAC). RFC 7693, 2015.

- Ste89. J. Stern. A method for finding codewords of small weight. In *Coding Theory and Applications, Lecture Notes in Computer Science* 388, pages 106–113. Springer, 1989.
- YSWW21. K. Yang, P. Sarkar, C. Weng, and X. Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *ACM CCS 2021*, pages 2986–3001. ACM, 2021.

A Performance Comparison with Existing Post-Quantum Signatures

This section reports the selected parameters and benchmark results for SYDO at the NIST security levels L1, L3, and L5. We compare SYDO with existing post-quantum signature schemes, including NIST standardized signatures² and the third-round candidates based on MPC/VOLEitH techniques in NIST’s Additional Digital Signature Standardization Process³.

A.1 Implementation Methodology

We implement SYDO based on the AVX2-optimized implementation of FAEST v2⁴.

RSD parameter selection. Table A.1 lists the RSD and modeling parameters used in this comparison.

Table A.1. Selection of RSD parameters

NIST Security		RSD Parameters			Modeling		
Category	Bits	n	$(n - k)$	w	ℓ	$(\eta_j)_{j \in [\ell]}$	$(r_j)_{j \in [\ell]}$ [†]
I	143	16 200	384	45	2	(3, 8)	(1, 3)
III	207	23 298	576	66	2	(3, 8)	(1, 3)
V	272	25 392	752	92	2	(2, 8)	(1, 3)

VOLEitH parameter selection. For a fair comparison with SDitH, we keep the original commitment scheme from FAEST v1 [BBM⁺24] rather than the improved all-but-one vector commitment from [BBB⁺25b, BBB⁺25a]. This choice makes the leaf commitment scheme close to that of SDitH, since both use collision-resistant hashing for leaf commitments. For the same reason, we adopt the parameters used by the VOLEitH proof system in SDitH, as reported in Table A.2. These choices are meant to provide a fair comparison with SDitH, and do not rule out better trade-offs for SYDO under a dedicated parameter optimization. We run FAEST with its own recommended parameters from [BBB⁺25a].

Table A.2. VOLEitH parameter choices for different security levels and trade-offs

Security	Trade-off	VOLEitH Parameters			
		τ	k	w_{grind}	T_{open}
NIST I	Short	11	11	9	107
	Fast	16	8	2	101
NIST III	Short	16	12	2	157
	Fast	24	8	2	153
NIST V	Short	21	12	6	216
	Fast	32	8	2	207

Selection of symmetric primitives. For the NIST L1, L3, and L5 parameter sets, the PRG is instantiated with AES-128, AES-192, and AES-256, respectively. The XOF is instantiated with SHAKE128 for NIST L1 and SHAKE256 for NIST L3 and L5.

² <https://csrc.nist.gov/news/2022/pqc-candidates-to-be-standardized-and-round-4>

³ <https://csrc.nist.gov/Projects/pqc-dig-sig/round-3-additional-signatures>

⁴ <https://github.com/faest-sign/faest-arch-opt>

A.2 Comparisons with Other Post-Quantum Signatures

We conduct experiments to evaluate SYDO and compare it with existing post-quantum signatures. To ensure a fair comparison, we benchmark the most optimized openly available implementation of each signature scheme. All experiments use the same benchmark environment as the AVX2-optimized implementation in Table 8.1. All reported numbers are averaged over 1000 runs.

Table A.3. Comparisons of sizes and runtimes for SYDO and existing post-quantum signatures at the NIST L1 security level

Scheme	Sizes (bytes)			Runtimes (ms)		Assumption
	pk	sig	sig + pk	sign	verify	
ML-DSA	1312	2420	3732	0.071	0.026	Lattice
FN-DSA	897	666	1563	0.225	0.049	Lattice
SLH-DSA-SHAKE (short)	32	7856	7888	320.440	0.400	Hash
SLH-DSA-SHAKE (fast)	32	17088	17120	15.517	1.058	Hash
SLH-DSA-SHA2 (short)	32	7856	7888	148.477	0.224	Hash
SLH-DSA-SHA2 (fast)	32	17088	17120	7.249	0.591	Hash
FAEST [BBB ⁺ 25a] (short)	32	4506	4538	4.277	3.298	AES
FAEST [BBB ⁺ 25a] (fast)	32	5924	5956	0.568	0.463	AES
MQOM-gf2 [BBFR25] (short)	52	2820	2872	3.789	3.655	MQ over $\mathbb{F}_2$
MQOM-gf2 [BBFR25] (fast)	52	3144	3196	1.685	1.579	MQ over $\mathbb{F}_2$
MQOM-gf16 [BBFR25] (short)	60	2916	2976	2.063	1.926	MQ over $\mathbb{F}_{16}$
MQOM-gf16 [BBFR25] (fast)	60	3280	3340	0.875	0.694	MQ over $\mathbb{F}_{16}$
SDitH-gf2 [MBB ⁺ 25] (short)	70	3705	3775	9.809	7.950	SD over $\mathbb{F}_2$
SDitH-gf2 [MBB ⁺ 25] (fast)	70	4484	4554	2.973	2.590	SD over $\mathbb{F}_2$
SYDO (short)	64	3508	3572	6.513	6.394	SD over $\mathbb{F}_2$
SYDO (fast)	64	4212	4276	1.474	1.363	SD over $\mathbb{F}_2$

As shown in Table A.3, for the combined size of a public key and a signature, SYDO achieves only 3572 bytes, which is smaller than the 3732 bytes of ML-DSA and less than half of the 7888 bytes of SLH-DSA. The running times are also competitive, with signing and verification taking about 1.5 ms in the fast variant and about 6.5 ms in the short variant. Compared with SLH-DSA-SHA2, signing is about $4.9\times$ and $22.8\times$ faster for the fast and short variants, respectively, while verification is about $2.3\times$ and $28.5\times$ slower.

Table A.4 compares SYDO with FAEST [BBB⁺25a], SDitH [MBB⁺25], and MQOM [BBFR25]. These schemes are the only third-round candidates based on MPCitH/VOLEitH techniques in NIST’s Additional Digital Signature Standardization Process.

Table A.4. Comparisons of SYDO, FAEST v2 [BBB⁺25a], SDitH v2 [MBB⁺25], and MQOM v2 [BBFR25] at NIST security levels L1, L3, and L5

Scheme	Sizes (bytes)			Runtimes (ms)		Assumption
	pk	sig	sig + pk	sign	verify	
SYDO-L1-short	64	3508	3572	6.513	6.394	SD over $\mathbb{F}_2$
SYDO-L1-fast	64	4212	4276	1.474	1.363	SD over $\mathbb{F}_2$
SYDO-L3-short	96	7604	7700	24.781	22.908	SD over $\mathbb{F}_2$
SYDO-L3-fast	96	9404	9500	3.429	3.040	SD over $\mathbb{F}_2$
SYDO-L5-short	126	13453	13579	35.731	35.624	SD over $\mathbb{F}_2$
SYDO-L5-fast	126	16564	16690	5.674	5.069	SD over $\mathbb{F}_2$
SDitH-L1-gf2-short	70	3705	3775	9.809	7.950	SD over $\mathbb{F}_2$
SDitH-L1-gf2-fast	70	4484	4554	2.973	2.590	SD over $\mathbb{F}_2$
SDitH-L3-gf2-short	98	7964	8062	44.183	39.679	SD over $\mathbb{F}_2$
SDitH-L3-gf2-fast	98	9916	10014	7.719	6.803	SD over $\mathbb{F}_2$
SDitH-L5-gf2-short	132	14121	14253	62.886	57.097	SD over $\mathbb{F}_2$
SDitH-L5-gf2-fast	132	17540	17672	11.210	10.065	SD over $\mathbb{F}_2$
FAEST-128-short	32	4506	4538	4.277	3.298	AES
FAEST-128-fast	32	5924	5956	0.568	0.463	AES
FAEST-192-short	48	11260	11308	18.194	14.349	AES
FAEST-192-fast	48	14948	14996	2.610	2.352	AES
FAEST-256-short	48	20696	20744	23.252	23.070	AES
FAEST-256-fast	48	26548	26596	3.329	3.154	AES
MQOM-L1-gf2-short	52	2820	2872	3.789	3.655	MQ over $\mathbb{F}_2$
MQOM-L1-gf2-fast	52	3144	3196	1.685	1.579	MQ over $\mathbb{F}_2$
MQOM-L3-gf2-short	78	6280	6358	16.484	15.690	MQ over $\mathbb{F}_2$
MQOM-L3-gf2-fast	78	7414	7492	8.085	8.029	MQ over $\mathbb{F}_2$
MQOM-L5-gf2-short	104	11564	11668	35.533	35.112	MQ over $\mathbb{F}_2$
MQOM-L5-gf2-fast	104	13124	13228	19.856	19.998	MQ over $\mathbb{F}_2$
MQOM-L1-gf16-short	60	2916	2976	2.063	1.926	MQ over $\mathbb{F}_{16}$
MQOM-L1-gf16-fast	60	3280	3340	0.875	0.694	MQ over $\mathbb{F}_{16}$
MQOM-L3-gf16-short	90	6496	6586	9.873	8.718	MQ over $\mathbb{F}_{16}$
MQOM-L3-gf16-fast	90	7738	7828	3.119	2.974	MQ over $\mathbb{F}_{16}$
MQOM-L5-gf16-short	122	12014	12136	15.606	14.965	MQ over $\mathbb{F}_{16}$
MQOM-L5-gf16-fast	122	13772	13894	6.941	6.563	MQ over $\mathbb{F}_{16}$

Comparison with SDitH. SDitH also uses VOLEitH and relies on syndrome decoding, so this gives the closest comparison. The signatures of SYDO are smaller at all three security levels by 4.5%–6.1%. The same advantage appears for public keys, which are smaller by 2.0%–8.6%. For all parameter sets, signing in SYDO is $1.51\times$ – $2.25\times$ faster, while verification is $1.24\times$ – $2.24\times$ faster.

Comparison with FAEST. VOLEitH-based FAEST instantiates the underlying OWF with AES. The comparison with FAEST shows a clear size-time trade-off. For all three security levels and both variants, the signatures of SYDO are smaller than those of FAEST by 22.1%–37.6%. FAEST is faster in every case, with signing about $1.31\times$ – $2.60\times$ faster and verification about $1.29\times$ – $2.94\times$ faster.

Comparison with MQOM. MQOM uses TCitH [FR25], a variant of MPCitH, and its security relies on the multivariate quadratic problem. The signatures of SYDO are larger than those of MQOM by 12.0%–34.0%. Compared with MQOM over $\mathbb{F}_2$, MQOM is faster for the short variants by up to $1.72\times$ in signing and $1.75\times$ in verification, while the fast variants of SYDO are faster at levels L3 and L5 by up to $3.50\times$ in signing and $3.94\times$ in verification. Compared with MQOM over $\mathbb{F}_{16}$, MQOM is generally faster, with the exception of signing and verification for the L5 fast variant.